

Introduction to Programmable Logic Controllers (PLCs)

Dr. Mazouz Salahat



Objectives

- Understand general PLC issues.

Topics

- Definition of a PLC
- History of PLC
- PLCs vs. PCs and other controls
- PLC applications

PURPOSE OF PLCs

- Initially designed to replace relay logic control:
 - Sequence device actuation.
 - Coordinate activities.
- PLCs were invented to replace **electro-mechanical control parts** with **solid-state ones**.
- PLC Accepts input from a series of switches.
- PLC Sends output to devices or relays.

Definition of a PLC (1/2)

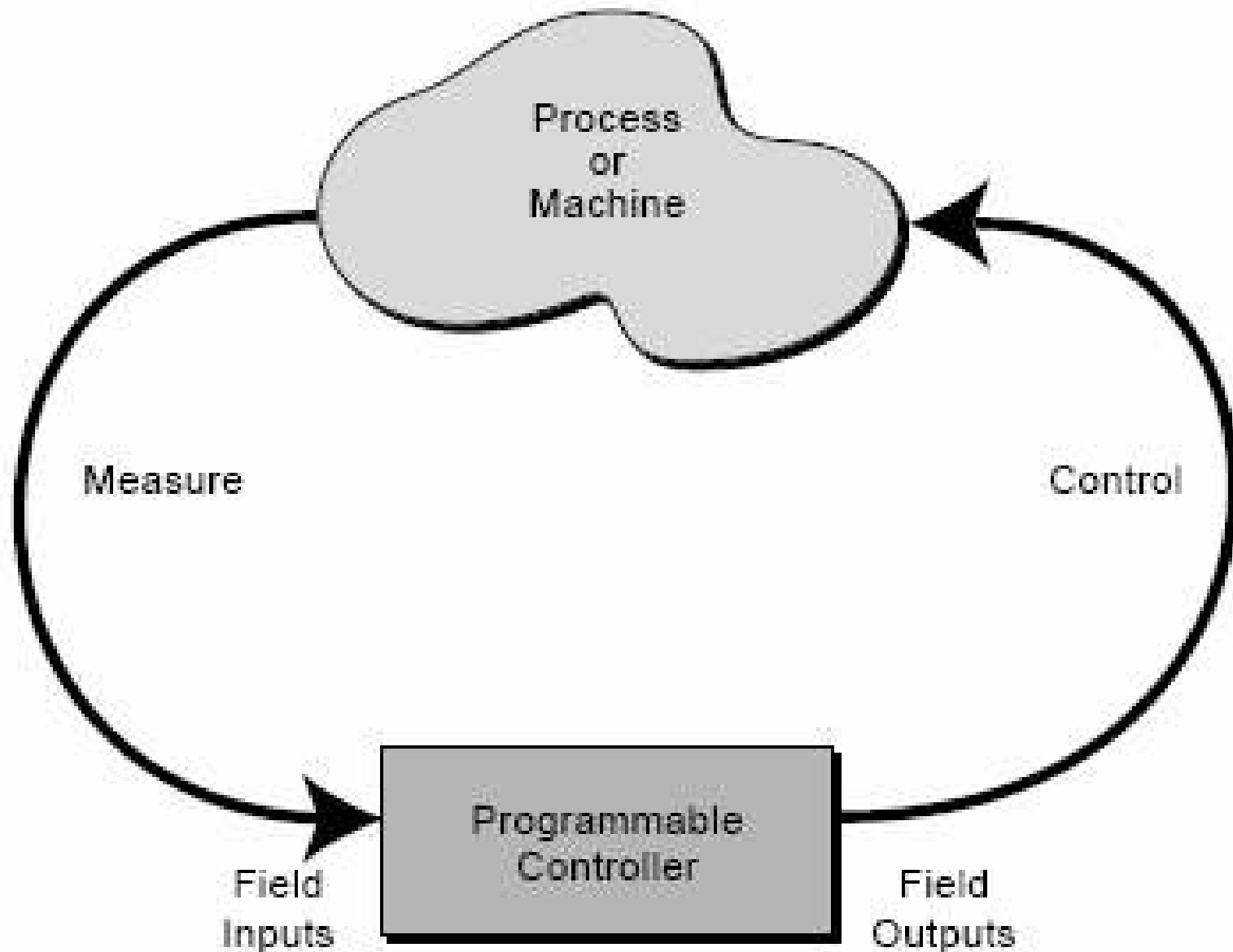
- A PLC is an **industrial computer** used to **monitor inputs**, and depending upon their state **make decisions** based on its program or logic, to **control (turn on/off) its outputs** to automate a machine or a process.
- **PLC's capabilities** are determined by the type of commands a user can instruct it to execute.

Definition of a PLC (2/2)

- *"A digitally operating electronic apparatus which uses a programmable memory for the internal storage of instructions by implementing specific functions such as logic sequencing, timing, counting, and arithmetic to control, through digital or analog input/output modules, various types of machines or processes".*

- *National Electrical Manufacturing Association (NEMA)*

PLC Application Diagram



History of PLCs (1/4)

- PLC development began in 1968 in response to a request from a major US car manufacturer (GE).
- The first PLCs were installed in industry in 1969.

History of PLCs (2/4)

- **PLC's Communications abilities began to appear in 1973.**
- **PLCs could also be used in the 70's to send and receive varying voltages to allow them to enter the analog control.**

History of PLCs (3/4)

- The **80's** saw an attempt to **standardize** communications with General Motor's manufacturing automation protocol (MAP).
- The **80's** also was a time for **reducing the size** of the PLC and making them software programmable through **symbolic programming on personal computers** instead of dedicated programming terminals or handheld programmers.

History of PLCs (4/4)

- The **90's** have seen a gradual reduction in the **introduction of new protocols**, and the **modernization** of the physical layers of some of the more popular protocols that survived the 1980's.
- The latest standard (IEC 1131-3) has tried to **merge plc programming languages** under one international standard.

Control Systems Types

Industrial applications may be able to be controlled by:

- Relays;
- PLCs;
- SBCs (single board controllers);
- PCs.

Disadvantages of PLC control

- Too much work required in connecting wires.
- Difficulty with changes or replacements.
- Difficulty in finding errors; requiring skillful work force.
- When a problem occurs, hold-up time is indefinite, usually long.

Advantages of PLC control

- Rugged and designed to withstand vibrations, temperature, humidity, and noise.
- Have interfacing for inputs and outputs already inside the controller.
- Easily programmed and have an easily understood programming language.

PC-Based Control

The PC presented three main challenges:

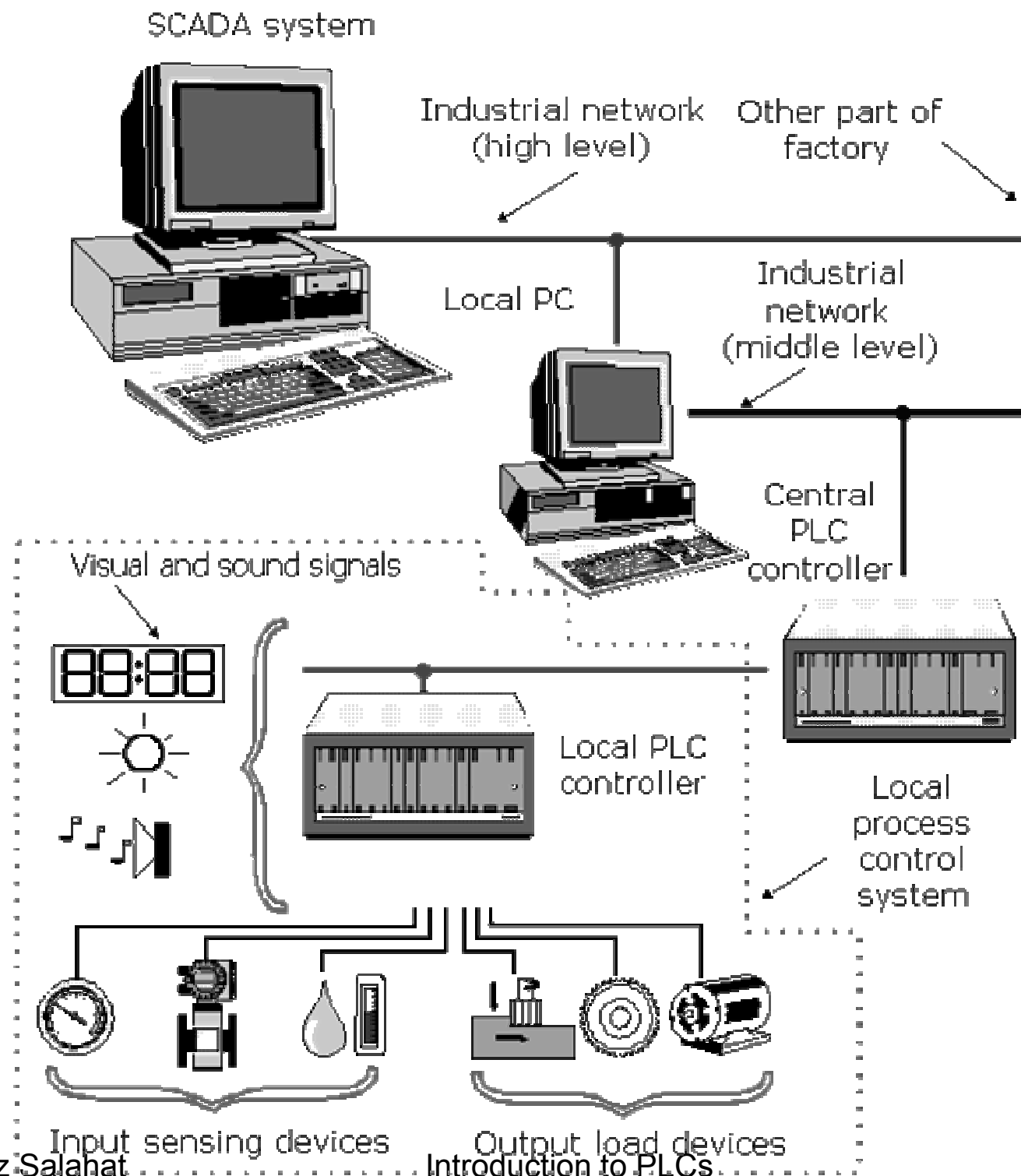
- **Stability:** PC-controlled installations were forced to handle system crashes and unplanned rebooting.
- **Reliability:** With rotating magnetic hard drives and non-industrially hardened components, such as power supplies, PCs were more prone to failure.
- **Unfamiliar Programming Environment:** Plant operators need the ability to override a system for maintenance or troubleshooting.

Comparison between Control Systems

Characteristic	Relay Systems	Digital Logic	Computers	PLC systems
Price per function	Fairly Low	Low	High	low
Physical size	Bulky	Very compact	Fairly compact	Very compact
Operating speed	Slow	Very Fast	Fairly fast	Fast
Electrical noise immunity	Excellent	Good	Quite Good	Good
Installation	Time consuming to design & install	Design is time consuming	Programming extremely time consuming	Simple to program & install
Capable of complicated operations	No	Yes	Yes	Yes
Ease of changing function	Very Difficult	Difficult	Quite Simple	Very Simple
Ease of maintenance	Poor - large number of contacts	Poor if ICs soldered	Poor - Several custom boards	Good - few standard cards

Traditional PLC Applications

- In automated system, PLC controller is usually the central part of a process control system.
- To run more complex processes it is possible to connect more PLC controllers to a central computer.
- It can be included in SCADA systems.



The 80-20 Rule

Experts estimate that:

- 77% of PLCs are used in small applications (less than 128 I/O)
- 72% of PLC I/O is digital
- **80% of PLC application** challenges are solved with a set of 20 ladder-logic instructions.

- Because 80 percent of industrial applications are solved with traditional tools, there is strong demand for simple low-cost micro PLCs with digital I/O that use ladder logic.

- In the 80s and 90s, other “20 percenters” evaluated PCs for industrial control.
- This is the reason many factory floors today have PLCs used in conjunction with PCs for data logging, connecting to bar code scanners, inserting information into databases, and publishing data to the Web.

- Engineers with complex applications requested the ability to combine the advanced **software capabilities** of the PC with the **reliability** of the PLC.
- The software capabilities required not only advanced software, but also an increase in the hardware capabilities of the controllers.

Combination of PCs & PLCs

- The main problem with PC-based control was that standard PCs were not designed for rugged environments.
- The big problem with this type of setup is that these systems are often difficult to **construct, troubleshoot and maintain.**

Programmable Automation Controllers

- Programmable Automation Controllers (PACs) that combine the **functionality** of a PC and **reliability** of a PLC today are increasingly being incorporated into control systems.

➤ Next Class

PLC Hardware

PLC

HARDWARE



Dr. Mazouz Salahat

•
•
•

OBJECTIVES

- Be familiar with mechanical design of PLC systems.
- Understand function and role of each PLC component.

•
•
•

Topics

- Mechanical design
of a PLC
- Hardware
components of a PLC

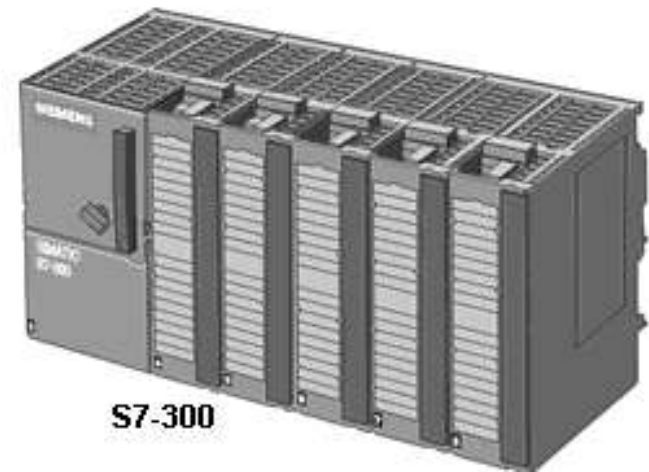
•
•
•

PLC Categories

- Micro (I/O up to 32) ;
- Small (I/O counts 32-256) ;
- Medium (I/O counts 256-1024) ;
- Large (I/O more than 1024) .

Mechanical Design

- Packaged (Fixed)
- Modular (Rack)



•
•
•

Packaged Type

The packaged type is a commonly used for small PLCs and is self-contained unit supplied as an integral compact package complete with all units.

•
•
•

Modular Type

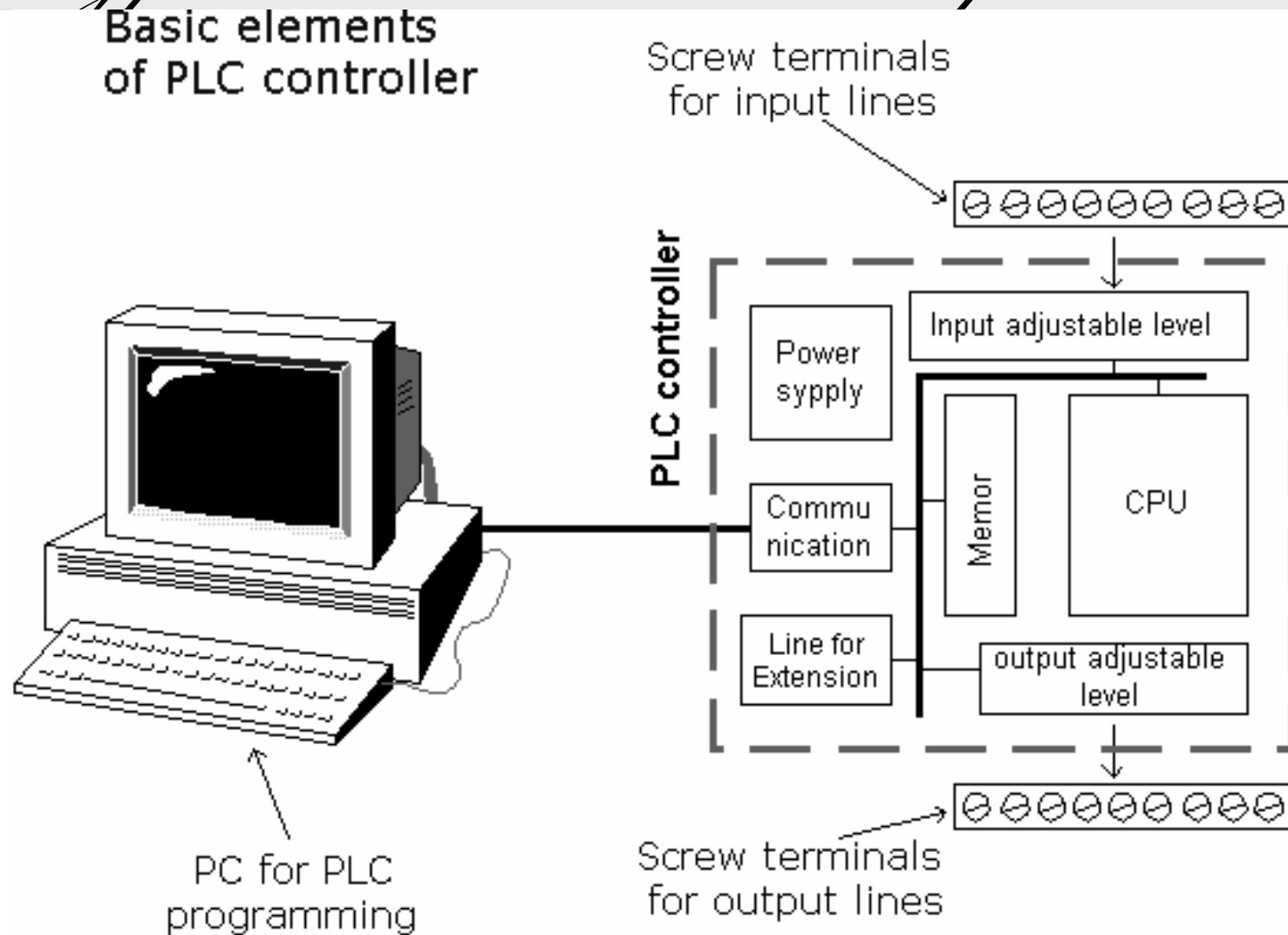
- Can be used for all sizes of PLCs.
- Has the various functional units, **the mix of which decided by the user**, packaged in individual modules which can be plugged into sockets in a base rack (chassis).

•
•
•

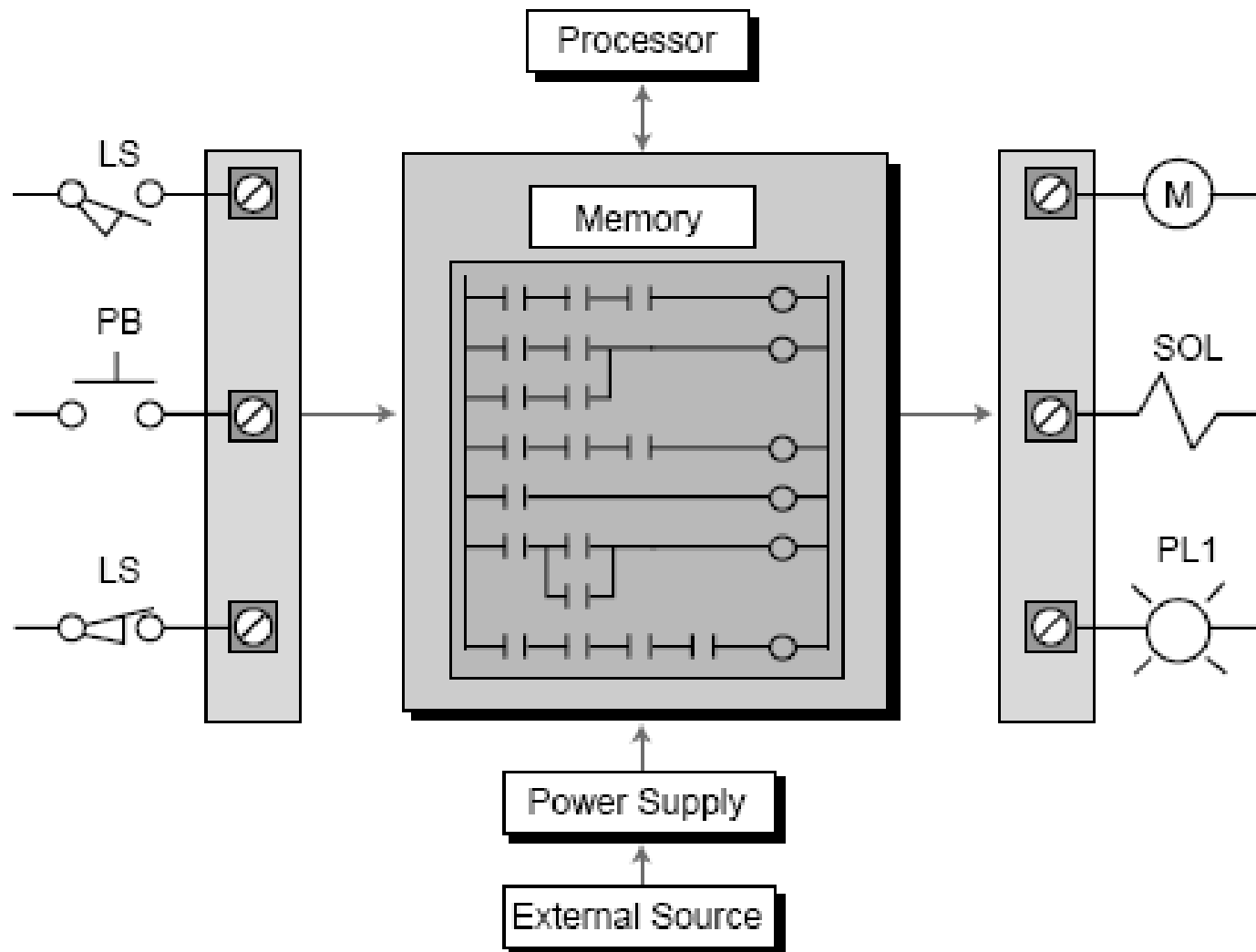
Hardware Components of a PLC System

1. Processor unit (CPU)
2. Memory
3. Input/Output section
4. Power supply unit
5. Programming device
6. Special devices,
Optional modules

Typical PLC Components



Functional interaction of a PLC system



•
•
•

Central Processing Unit

- CPU - Microprocessor based, may allow arithmetic operations, program execution, block memory moves, computer interface, local area network, functions, .
- CPU makes a great number of check-ups of the PLC controller itself so eventual errors would be discovered early.

•
•
•

System Busses

- The internal paths along which the digital signals flow within the PLC are called busses.
- The system has **four busses**.
 - The **data** bus for sending data between the different elements,
 - The **address** bus to send the addresses of locations for accessing stored data,
 - The **control** bus for signals relating to internal control actions,
 - The **system** bus is used for communications between the I/O ports and the I/O unit.

•
•
•

Memory

- System (ROM) to give permanent storage for the operating system and the fixed data used by the CPU.
- RAM for the user's program.
- RAM for data where information is stored on the status of input and output devices and the values of timers and counters and other internal devices.
- EPROM for ROM's that can be programmed and then the program made permanent [possibly, as an option module].

•
•
•

- To prevent the loss of programs when the power supply is switched off, a battery is used in the PLC to maintain the RAM contents for a period of time.
- After a program has been developed in RAM it may be loaded into an EPROM memory chip and so made permanent.
- In addition there are temporary buffer stores for the I/O channels.

•
•
•

IO Sections

- Inputs monitor field devices, such as switches and sensors.
- Outputs control other devices, such as motors, pumps, solenoid valves, and lights.

•
•
•

- Input and output blocks include protection needed in isolating a CPU blocks from damaging influences that industrial environment can bring to a CPU via input lines.

•
•
•

Power Supply

- Most PLC controllers work either at 24 VDC or 220 VAC.
- Some PLC controllers have electrical supply as a separate module, while small and medium series already contain the supply module.

•
•
•

- User has to determine how much current to take from I/O module to ensure that electrical supply provides appropriate amount of current.
- This electrical supply is usually not used to start external inputs or outputs.
- Some of the smaller PLC controllers supply their inputs with voltage from a supply source already incorporated into a PLC.

•
•
•

Programming Device

- The programming device is used to enter the required program into the memory of the processor.
- The program is developed in the programming device and then transferred to the memory unit of the PLC.
- For the operation of the PLC it is not necessary to for its programming device to be connected to the PLC.

-
-
-
- Programming devices can be:
 - Hand-held device (HHT)
 - Desktop console
 - Computer.
- Some PLCs only require the computer to have appropriate software, others special communication cards to interface with the PLC.

•
•
•

Special Devices, optional

- Some of these devices are: CRT programmer; Operator console; Printer; Simulator; EPROM loader; Graphics processor; and Network communication interface.

-
-
-
- Next Class

Wiring of a PLC System

WIRING OF PLC SYSTEM

Dr. Mazouz Salahat

Objectives

- Be able to understand and carry PLC wiring.
- Understand functions of PLC I/O sections.

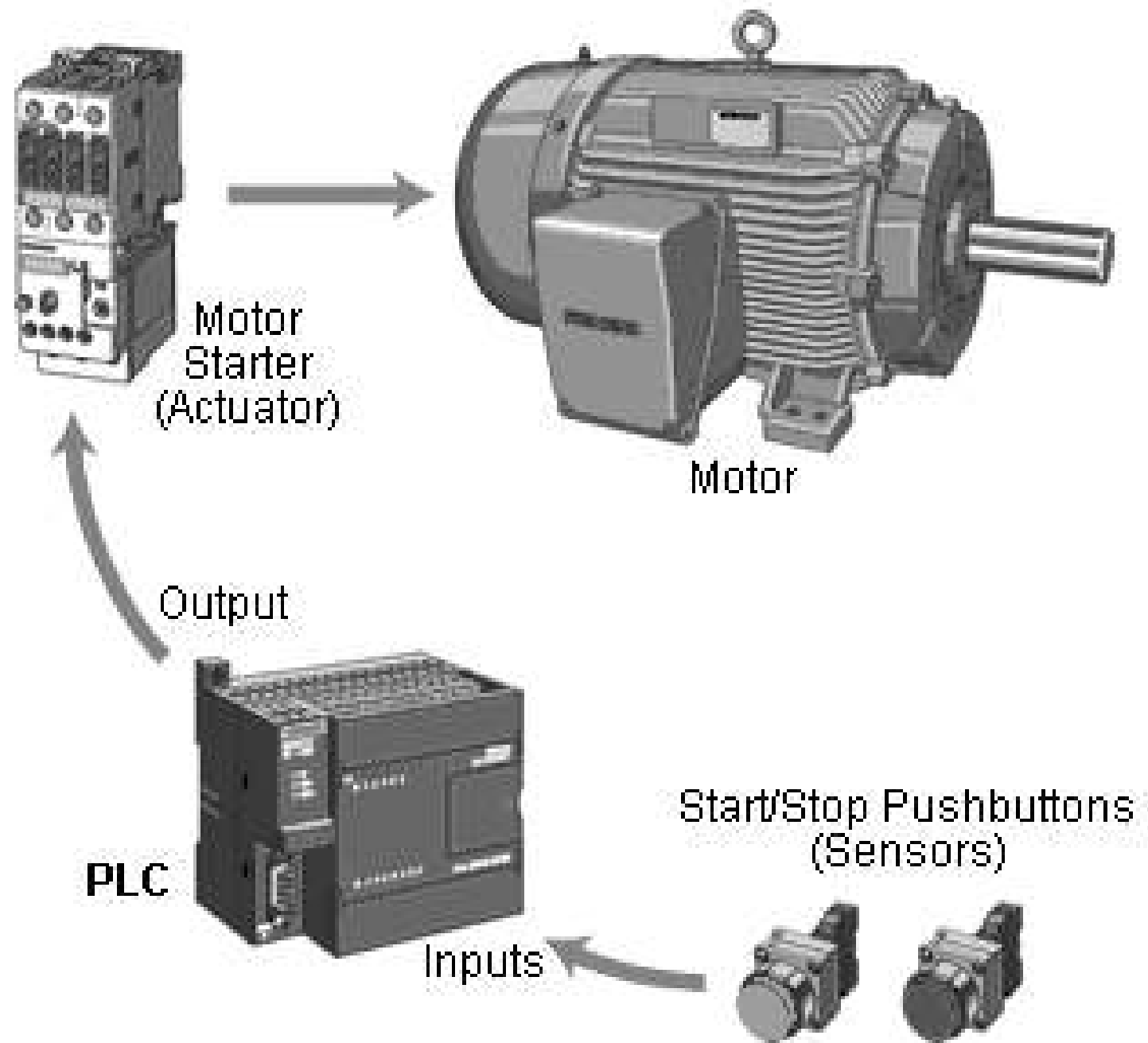
Topics

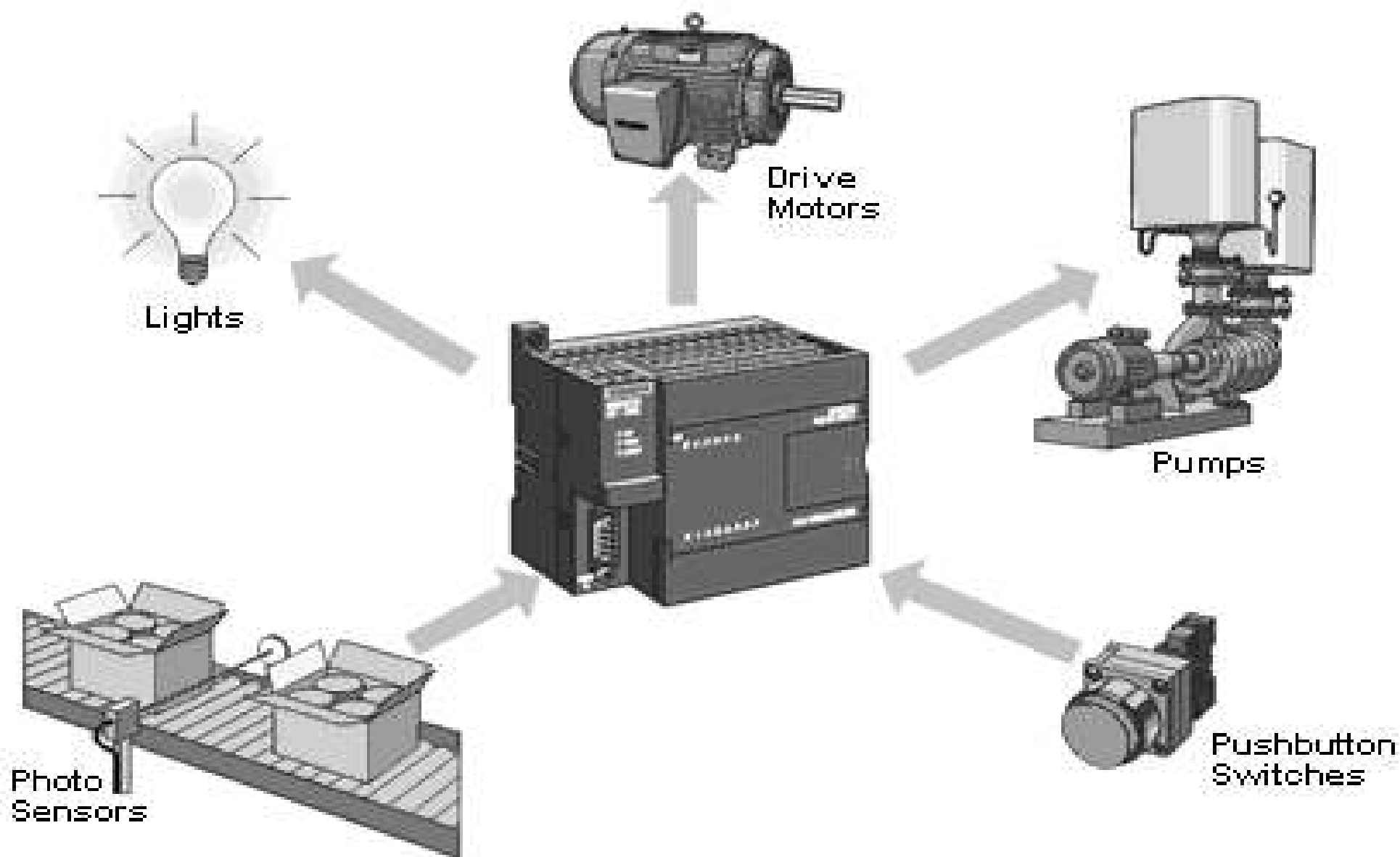
- ❑ I/O field devices
- ❑ I/O units
- ❑ Sinking-Sourcing concept
- ❑ I/O Wiring
- ❑ I/O addressing

I/O devices

In order to function, PLC needs :

- sensors to obtain information from environment, and
- execution devices so it could turn the programmed change into a reality.





I/O Signals

Input signals from sensors and outputs to actuators can be:

- ❑ Analogue, the signal size is related to the size of quantity being sensed.
- ❑ Discrete, just an on-off signal.
- ❑ Digital, a sequence of pulses.

INPUT FIELD DEVICES

- Keys, keyboards and functional switches for man versus machine relationship.
- Sensors & specific automatic devices to detect a working piece, view a mechanism in motion, check pressure or fluid level.
- Intelligent devices such as robots, video systems, etc.

OUTPUT DEVICES

- The digital signal from an output channel of a PLC is used to control an actuator which in turn controls some process.
- The following are some examples: contactors, directional control valves, motors, indication lamps, visual and audio signaling devices.

-
- ❑ Analog output devices can be analog meters, analog valves and actuators, DC and AC motor drives.
 - ❑ The output ports of the PLC are of type depending on the device connected to them which are to be switched on or off.

Ratings of Input/Output devices

- **TTL level;**
- **24 V AC/DC;**
- **48 V AC/DC;**

- **120 V AC/DC;**
- **230 V AC/DC.**

PLC I/O UNITS

- The CPU must have an **input** of digital signals normally **0 to 5 V DC**.
- The **output** from the CPU is digital, also **0 to 5 V DC**.
 - There is a need to **manipulate input and output signals** so that they are in the required form.
 - The I/O units are designed so that a range of input signals can be changed into 5 V digital signals and so that a range of outputs are available to drive external devices.

I/O sections different types

- ❑ AC voltage input and output,
- ❑ DC voltage input and output,
- ❑ Analog (voltage or current) input and output,
- ❑ Special purpose modules: high speed timers, stepping motor controllers, PID controllers, and Motion controllers.

PLC Input Units

- **Field devices typically send to the PLC unfiltered 120/220 VAC or 24 VDC.**
- **Smaller PLC controllers usually have only digital input lines.**
- **Larger accept analogue inputs through special units attached to PLC controller.**

Analog Input

- Analog signals can be inputted to a PLC if the input channel is able to convert the signal to a digital signal using an ADC.
- With a modular system this may be achieved by a suitable analog input card.
 - Cards are typically available giving 4, 8, or 16 analog inputs.

PLC Output Units

- Output can be analog or digital type.
 - Digital output works as a switch - it connects and disconnects line.
 - Analog output is used to generate the analogue signal. Analog outputs can be provided by DAC at the output channel.
 - Analog output modules are usually provided in a number of outputs, e.g. 4 to 20 mA, -5 to +5 V Dc, 0 to +10 V DC.

functions of PLC I/O Sections

- The input screw terminals on a PLC form the interface by which field devices are connected to the PLC.
- The input circuitry on the PLC conditions this voltage to be usable by the PLC (5 VDC).

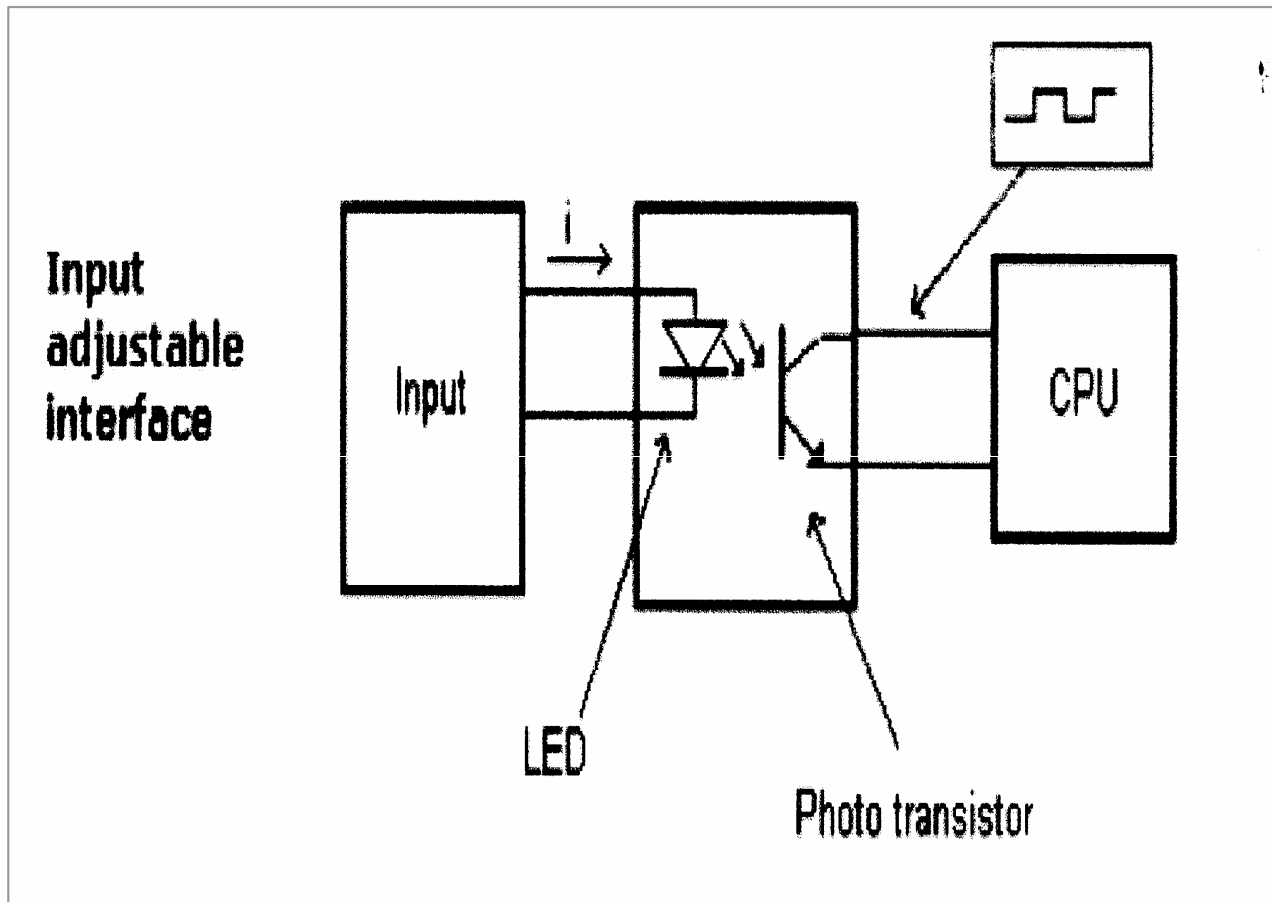
filtering

- The PLC's input circuitry "**filters**" field voltage signals to qualify them as a signal from a sensor, or as a high-frequency electrical noise or static.
- Input filters determine the validity of a signal by its duration. A typical filter time **is 8 ms.**
 - A longer response time provides better filtering of electrical noise.
 - A shorter response time helps in applications that require high-speed operation.

Protection

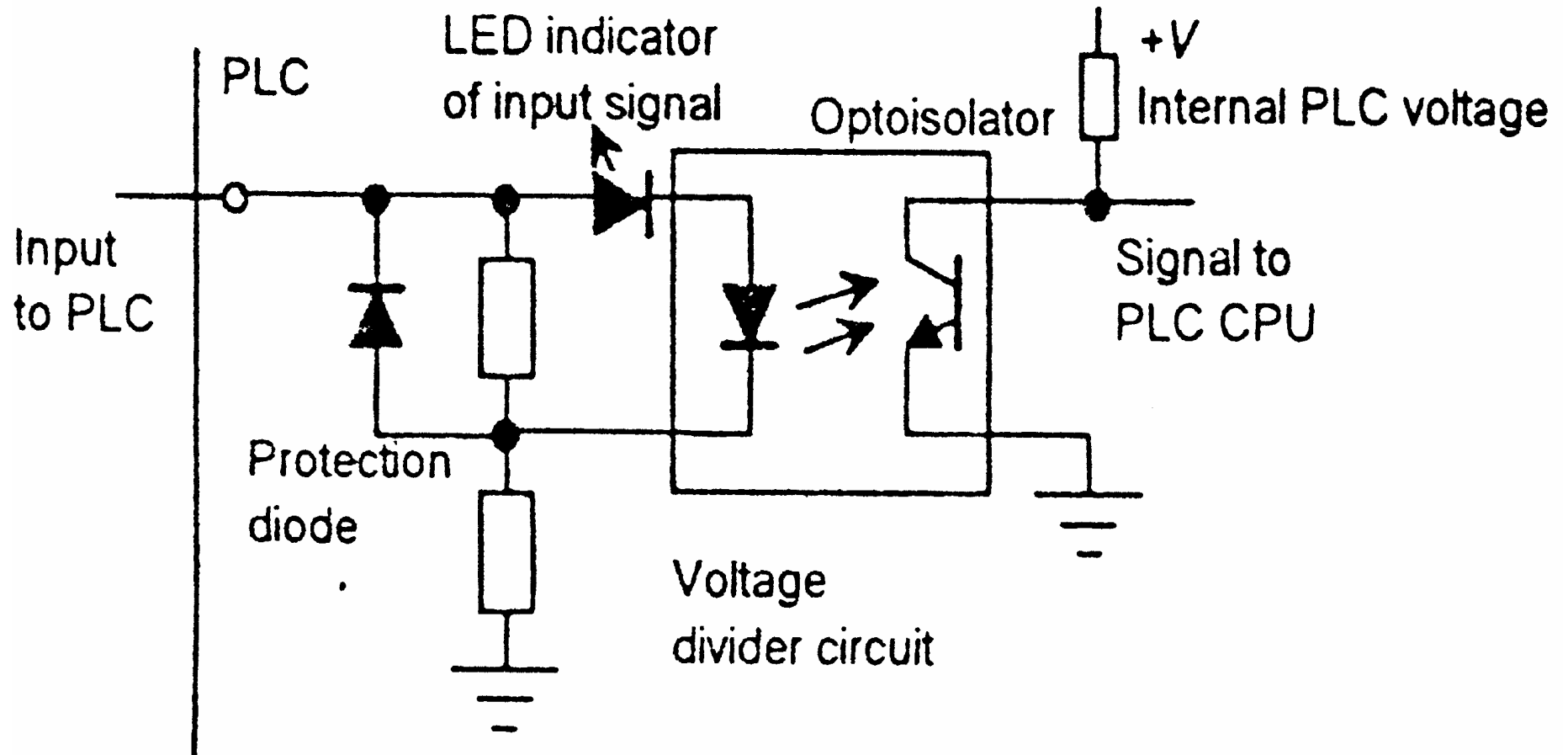
- An interface is placed between input lines and a CPU to **protect** a CPU from disproportionate signals from an outside world.
 - This is typically done through opto-isolation.

INPUT INTERFACE CIRCUIT

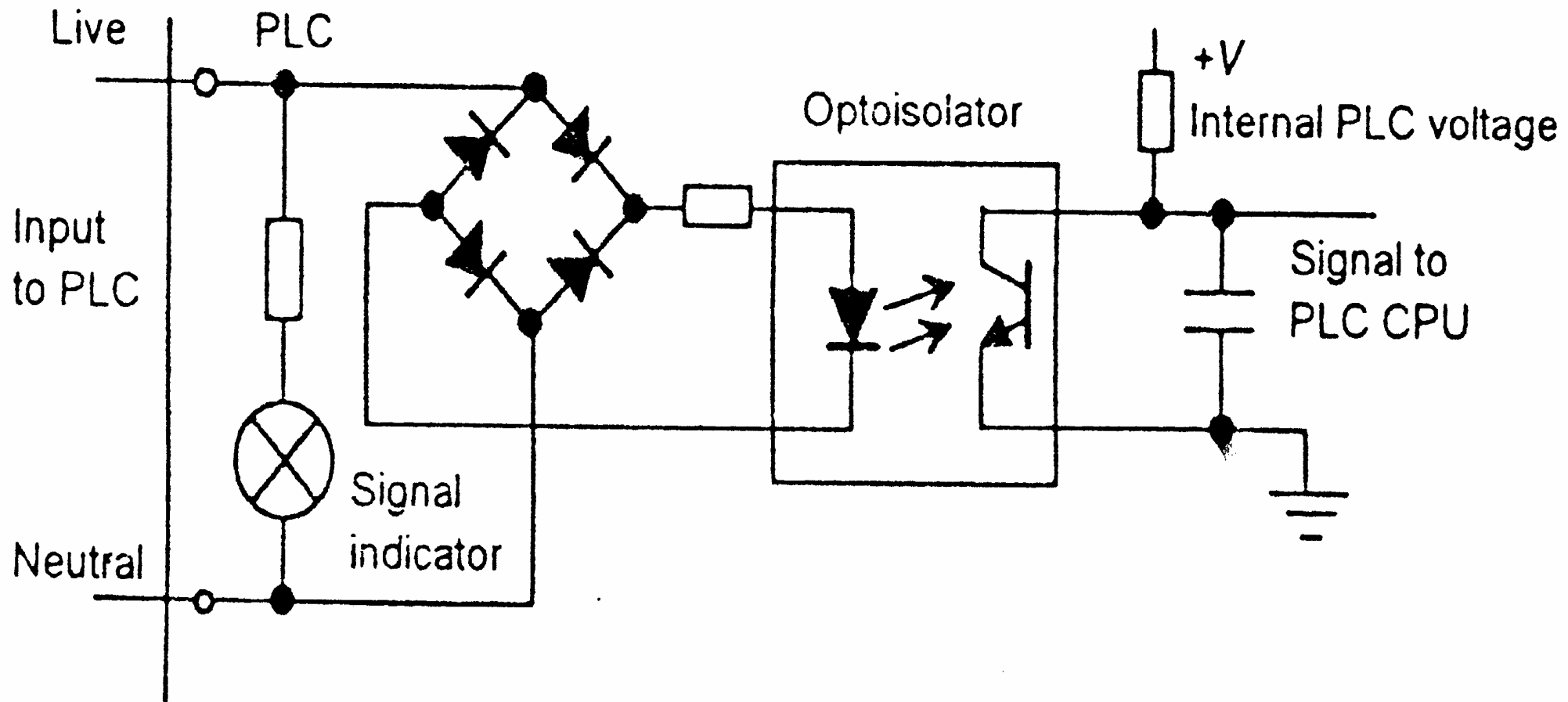


- External device brings a signal which turns LED on, whose light in turn incites photo transistor which in turn starts conducting, and a CPU sees this as logic zero.
- When input signal stops LED diode turns off, transistor stops conducting, collector voltage increases, and CPU receives logic as information.

D.C. Input Unit

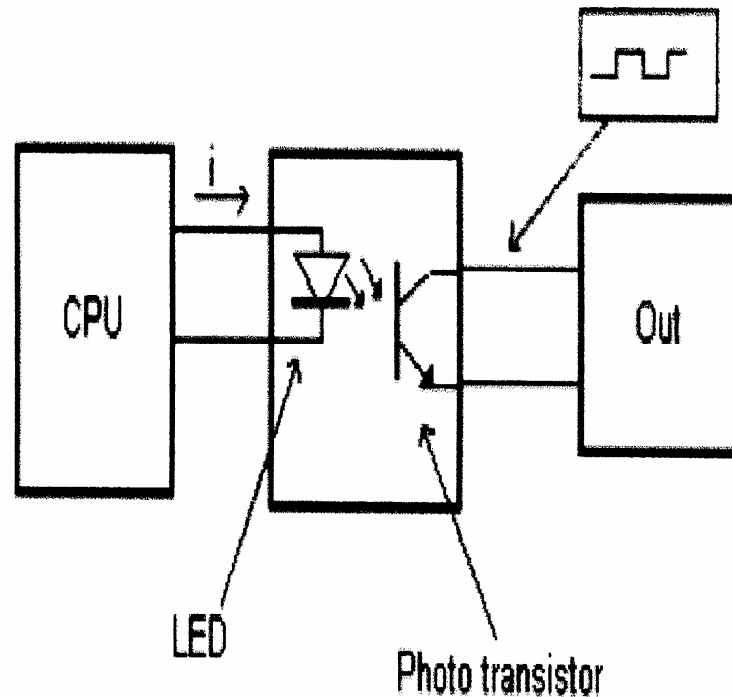


A.C. Input unit



Output Interface Circuit

Output
adjustable
interface



- CPU brings a signal to photo diode and turns it on.
- Light incites a photo transistor which begins to conduct electricity, and thus the voltage between collector and emitter falls to 0.7V, and a device attached to this output sees this as a logic zero.
- Inversely it means that a signal at the output exists and is interpreted as logic one.

-
- Photo transistor is not directly connected to a PLC controller output.
 - Between photo transistor and an output usually there is a relay or a stronger transistor capable of interrupting stronger signals.

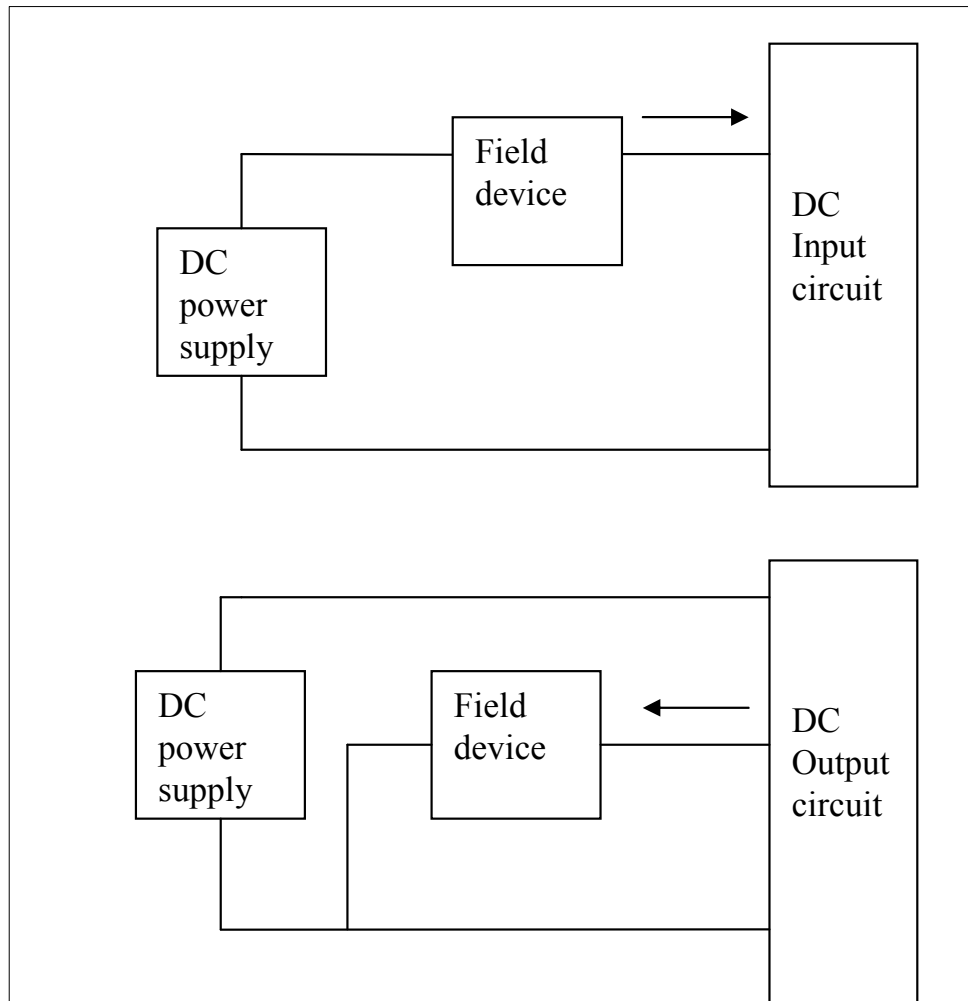
SINKING-SOURCING CONCEPT

- “*Sinking*” and “*Sourcing*” terms are very important in connecting a PLC correctly with external environment.
- These terms are applied only for DC modules.
 - The most brief definition of these two concepts would be:

SINKING = Common GND line (-)
SOURCING = Common VCC line (+)

Most commonly used DC module options in PLCs are:

- **Sinking input and**
- **Sourcing output module**



- **Sinking I/O circuits on the I/O modules receive (sink) current from sourcing field devices.** ■

- Sinking output modules used for interfacing with electronic equipment.

■ **Sourcing I/O**

- Sourcing output modules used for interfacing with solenoids.

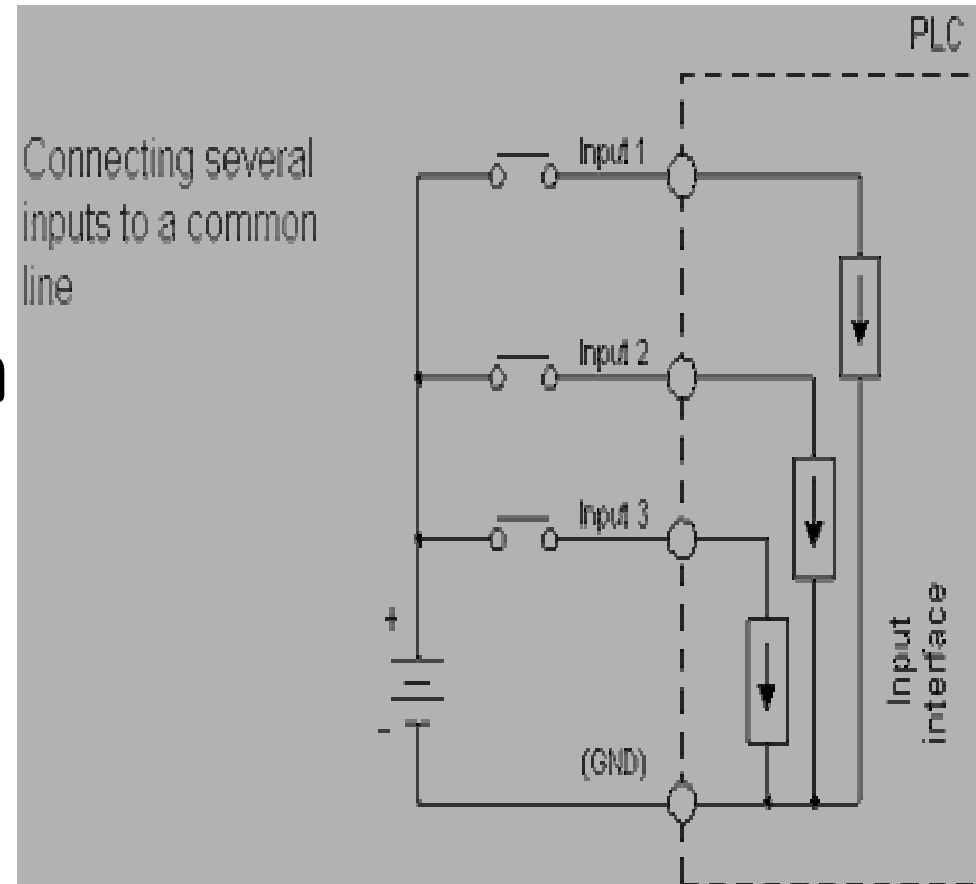
-
- **PLC AC I/O circuits accommodate either sinking or sourcing field devices.**
 - **Solid-state DC I/O circuits require that they used in a specific sinking or sourcing circuit depending on the internal circuitry.**
 - **PLC contact (relay) output circuits AC or DC accommodate either sinking or sourcing field devices.**

I/O WIRING

PLC Input Units

- **Example of input lines can be connection of external input device.**
- **Sensor outputs can be different depending on a sensor itself and also on a particular application.**

- In practice we use a system of connecting several inputs (or outputs) to one return line.
- These common lines are usually marked "COMM" on the PLC controller housing



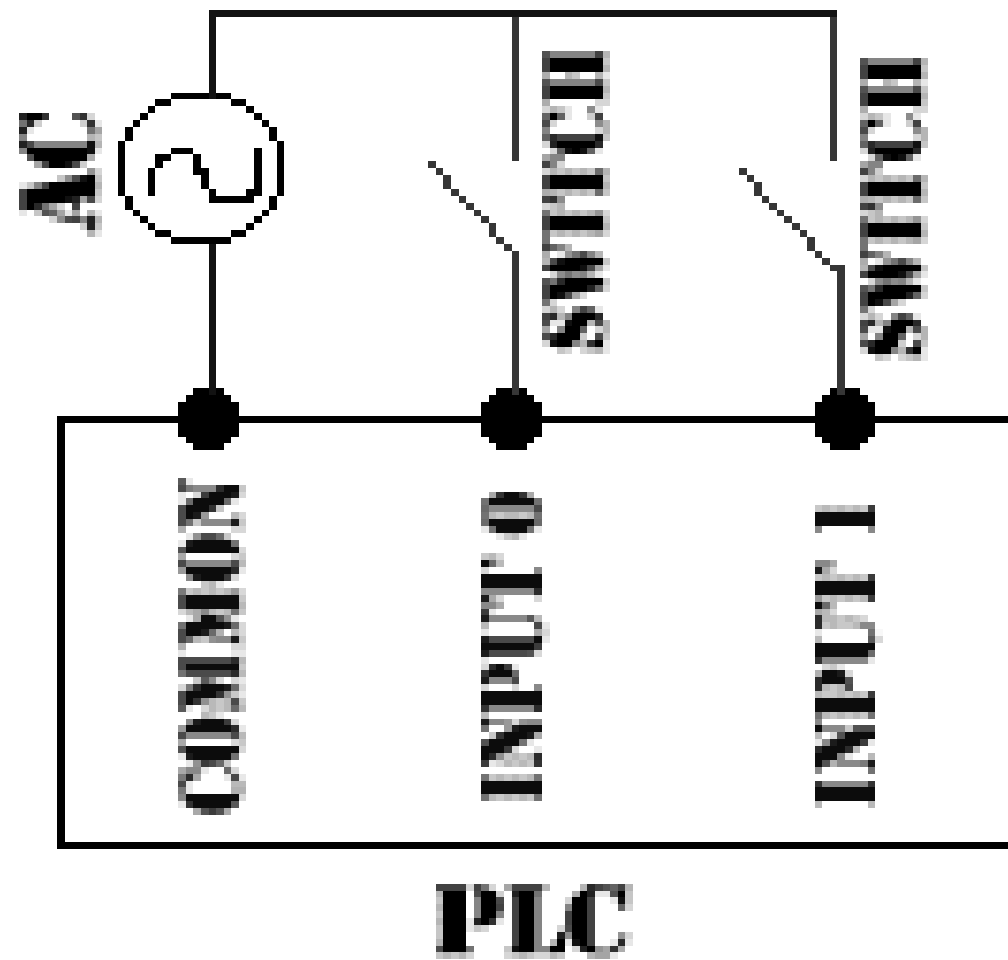
DC Inputs

- DC input modules allow to connect either PNP (sourcing) or NPN (sinking) transistor type devices to them.
 - When we are using a sensor have to worry about its output configuration.
- If we are using a regular switch (toggle or pushbutton) we typically don't have to worry about whether we wire it as NPN or PNP.

AC Inputs

- An ac voltage is non-polarized.
- Most commonly, the AC voltage is being switched through a limit switch or other switch type.
- AC input modules are less common than DC input modules, because today's sensors typically have transistor outputs.
- If application is using a sensor it probably is operating on a DC voltage.

Typical connection of an AC device to PLC input module



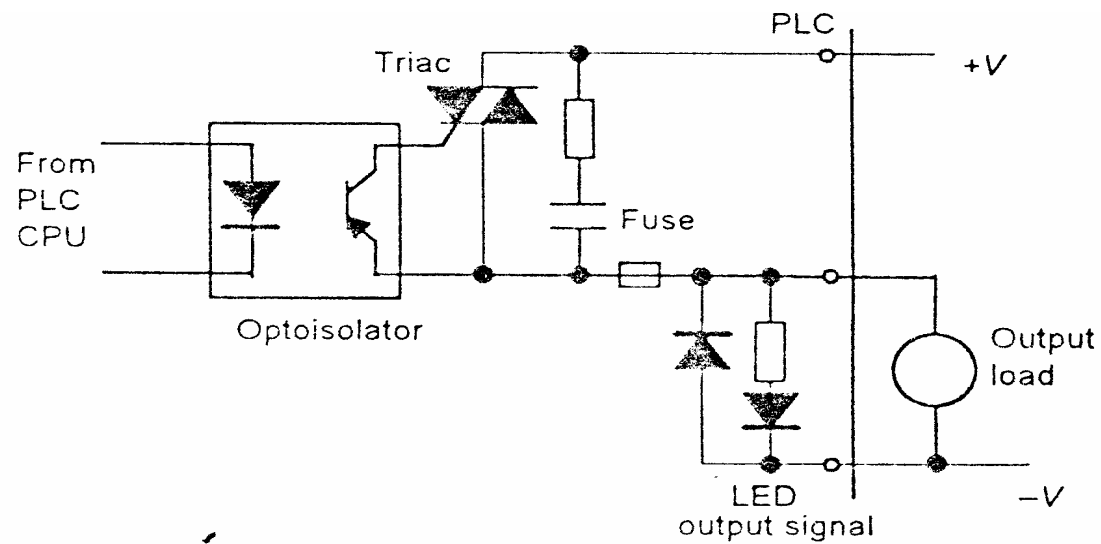
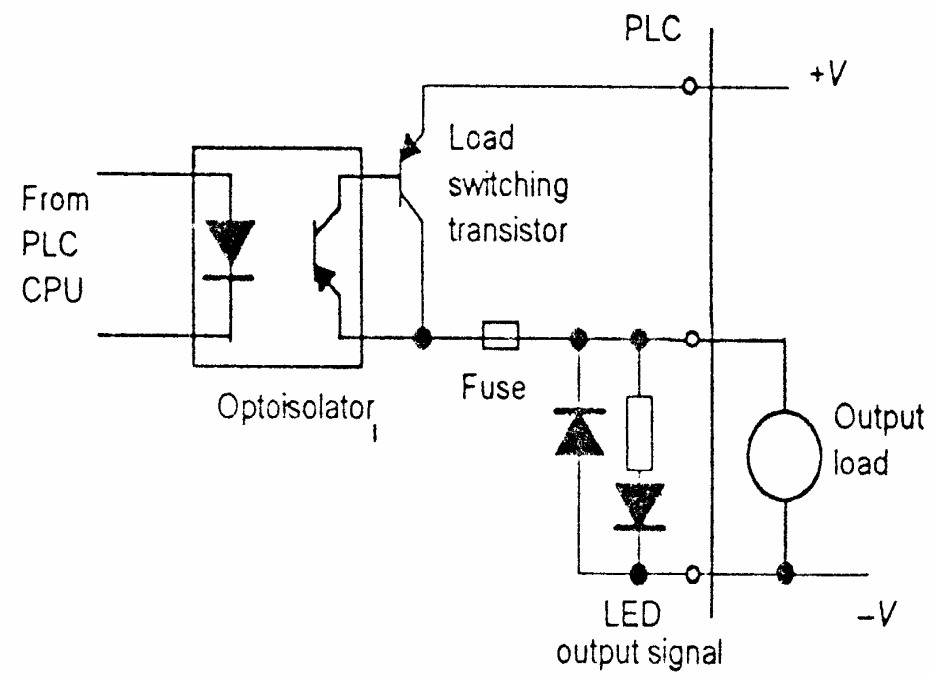
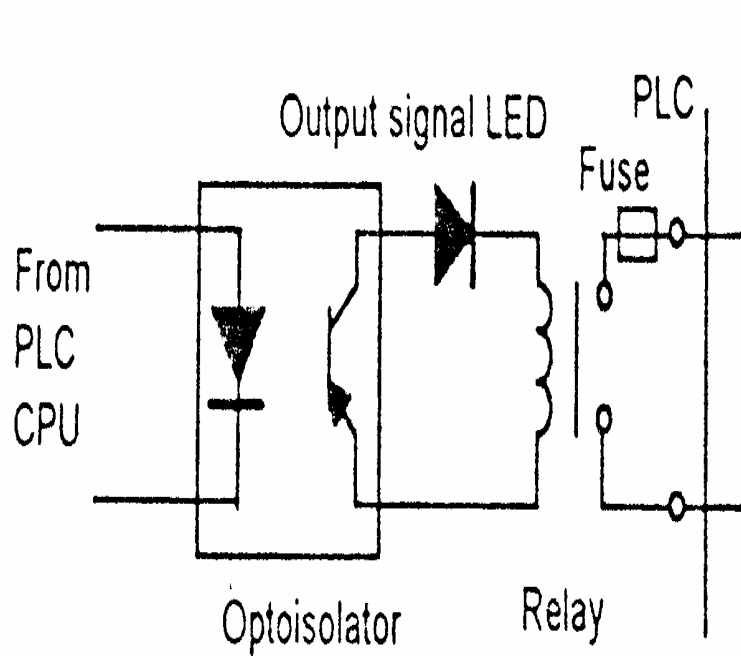
-
- Typically an AC input takes longer than a DC input for the PLC to see.
 - In most cases it doesn't matter to the programmer because an AC input device is typically a mechanical switch and mechanical devices are slow.

-
- It's quite common for a plc to require that the input be on for 25 ms (or more) before it's seen.
 - This delay is required because of the filtering which is needed by the PLC internal circuit.

PLC Output units

PLC Output units can be:

- Relay,**
- Transistor, or**
- Triac.**



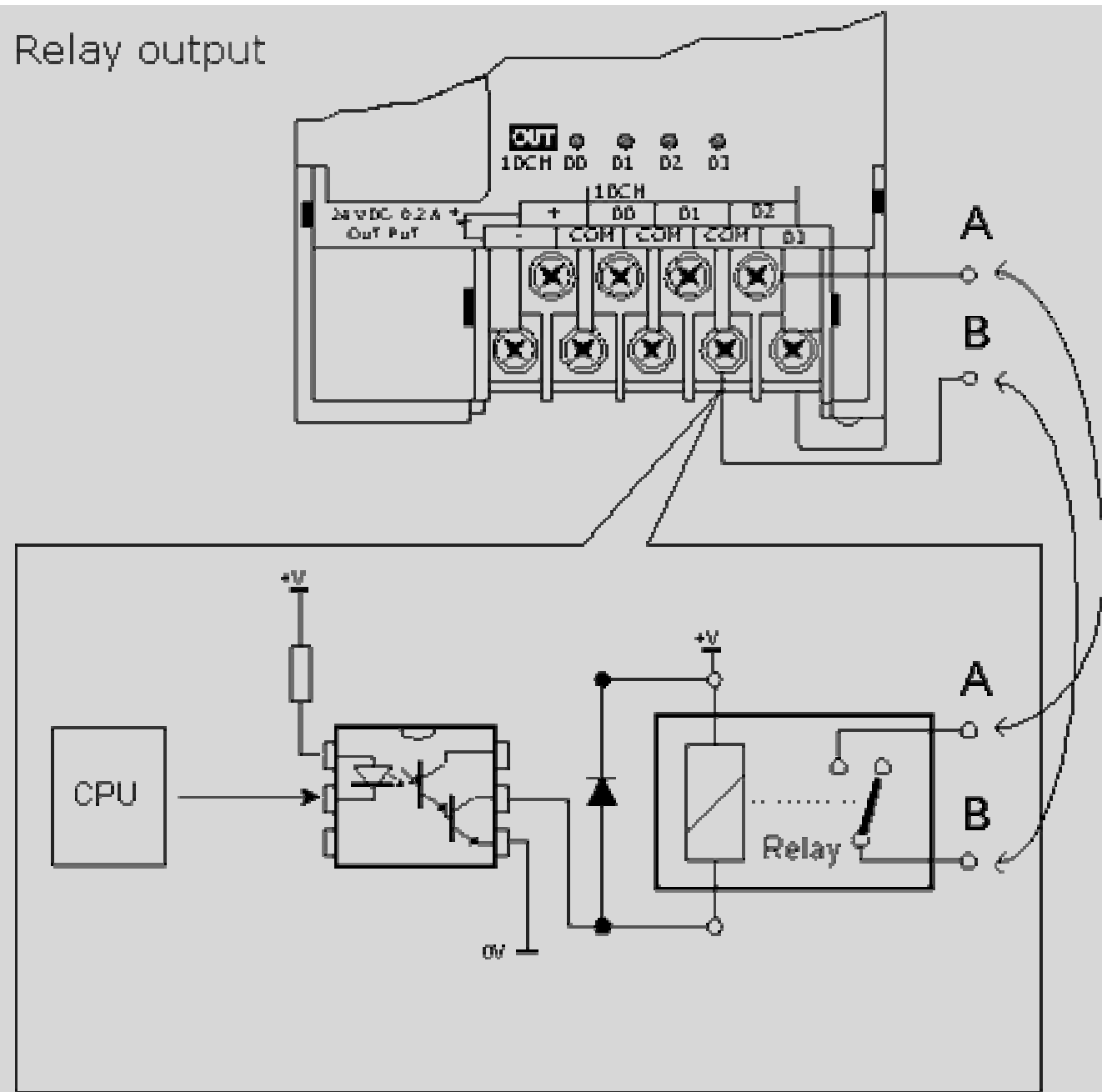
-
- **Check the specifications of load before connecting it to the plc output.**
 - **Make sure that the maximum current it will consume is within the specifications of the plc output.**

Relay Outputs

- One of the most common types of outputs available is the relay output.
- Existence of relays as outputs makes it easier to connect with external devices.

A relay is non-polarized and typically it can switch either AC or DC.

Relay output



Transistor Outputs

- Transistor type outputs can *only switch a dc current*.
- The PLC applies a small current to the transistor base and the transistor output "closes". When it's closed, the device connected to the PLC output will be turned on.

-
- **A transistor typically cannot switch as large a load as a relay.**
 - **If the load current you need to switch exceeds the specification of the output, you can connect the plc output to an external relay, then connect the relay to the large load.**

-
- *Typically a PLC will have either NPN or PNP transistor type outputs.*
 - **Some of the common types available are BJT and MOSFET.**
 - A BJT type often has less switching capacity than a MOSFET type.
 - The BJT also has a slightly faster switching time.

-
- *A transistor is fast, switches a small current, has a long lifetime and works with dc only.*
 - *A relay is slow, can switch a large current, has a shorter lifetime and works with ac or dc.*

Triac Output

- Triac output can be used to control AC loads only.
- Triac output is faster in operation and has longer life than relay output.

-
- ***Inductive loads*** have a tendency to deliver a "back current" when they turn on.
 - This back current is like a voltage spike coming through the system. This could be dangerous to output relays.
 - Typically a diode, varistor, or other "snubber" circuit should be used to protect the PLC output from any damage.
-

INPUT AND OUTPUT ADDRESSING

- **The PLC has to be able to identify each particular input and output.**
 - **It does this by allocating addresses to each input and output.**
 - **I/O addresses usually assigned to their related I/O terminals.**
-

-
- **With small PLC this is likely to be just a number, prefixed by a letter to indicate whether it is an input or an output.**
 - **Larger PLCs have its input and output addresses in terms of the module number and the terminal number within that module.**
-

- Next Class

PLC Communications

PLC OPERATION

Dr. Mazouz Salahat

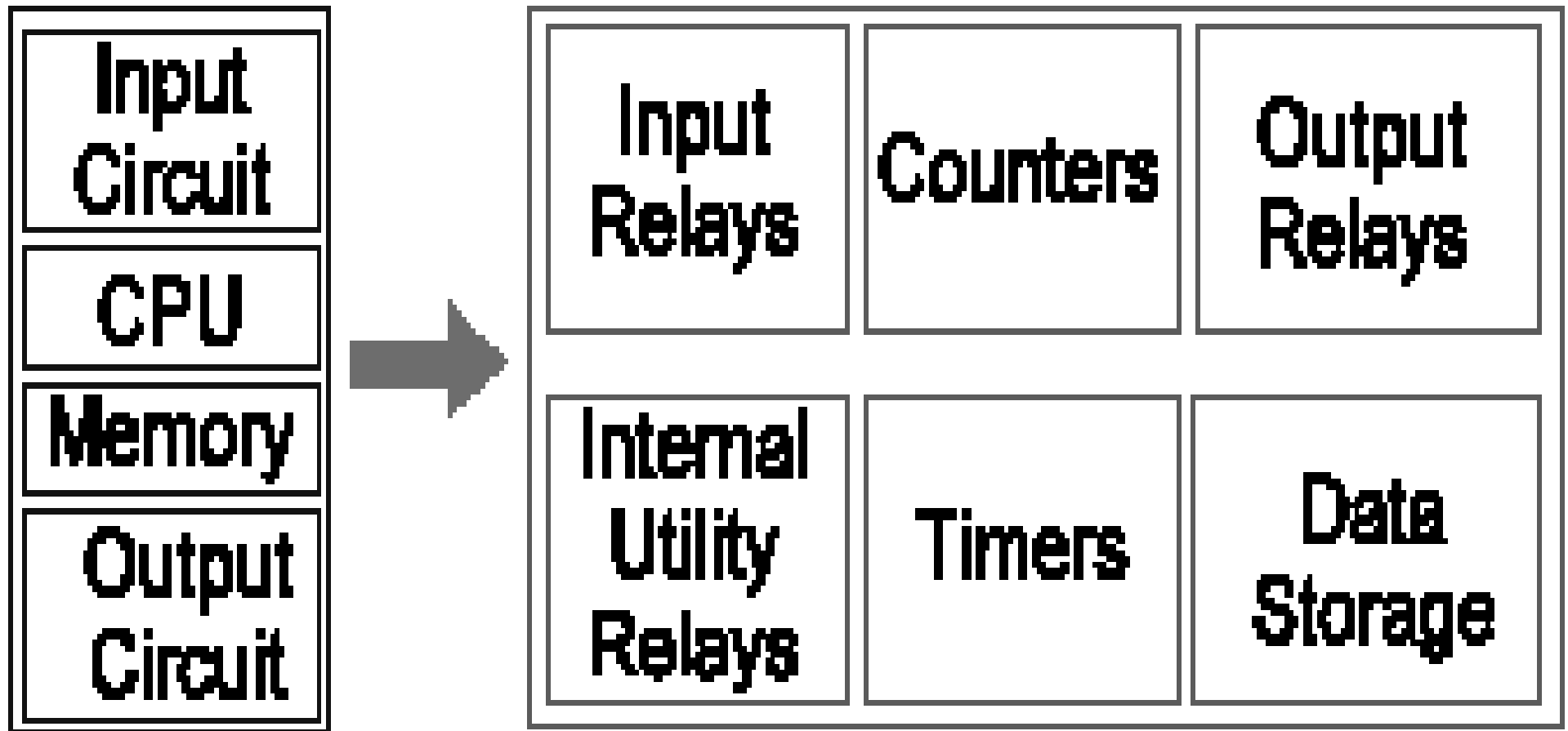
Objectives

- Understand the operation of a PLC.
- Estimate scan time & response time.
- Understand different operation modes.
- Understand codes & formats used in PLCs.

Topics

- PLC Internals
 - PLC Operation (Scan)
 - Response Time
 - PLC Working Modes
-
- PLC Addressing
 - Coding Systems.

PLC Internals



INPUT RELAYS

- Typically they are not relays but rather they are transistors.
- These are connected to the outside world.
- They physically exist and receive signals from switches, sensors, etc.

OUTPUT RELAYS

- These are connected to the outside world.
- They physically exist and send on/off signals to solenoids, lights, etc.
- They can be relays, transistors, or triacs depending upon the model chosen.

INTERNAL UTILITY RELAYS

- These do not receive signals from the outside world nor do they physically exist.
- They are simulated relays and are what enables a PLC to eliminate external relays.
- There are also some special relays that are dedicated to performing only one task.

COUNTERS

- These do not physically exist.
- They are simulated counters and they can be programmed to count pulses.
- Typically these counters can count up, down or both up and down.
- Since they are simulated they are limited in their counting speed.
- Some manufacturers also include high-speed counters that are hardware based.

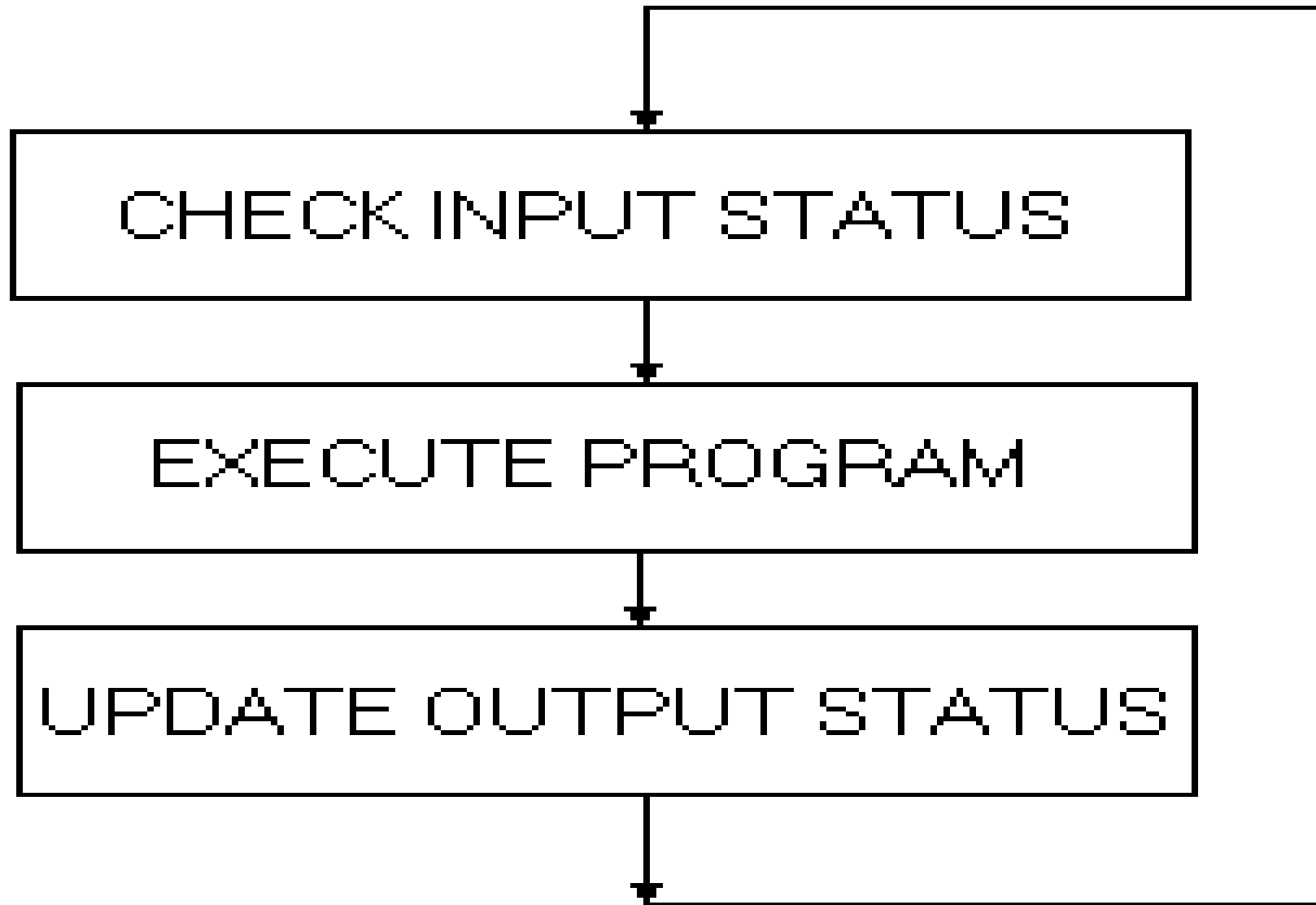
TIMERS

- These also do not physically exist.
- They come in many varieties and increments.
 - Retentive and non-retentive types;
 - On-delay, off-delay types;
 - Increments vary from 1ms through 1s.
- The most common type is an on-delay type.

DATA STORAGE

- Typically there are registers assigned to simply store data.
- They are usually used as temporary storage for math or data manipulation.
- They can also typically be used to store data when power is removed from the PLC.

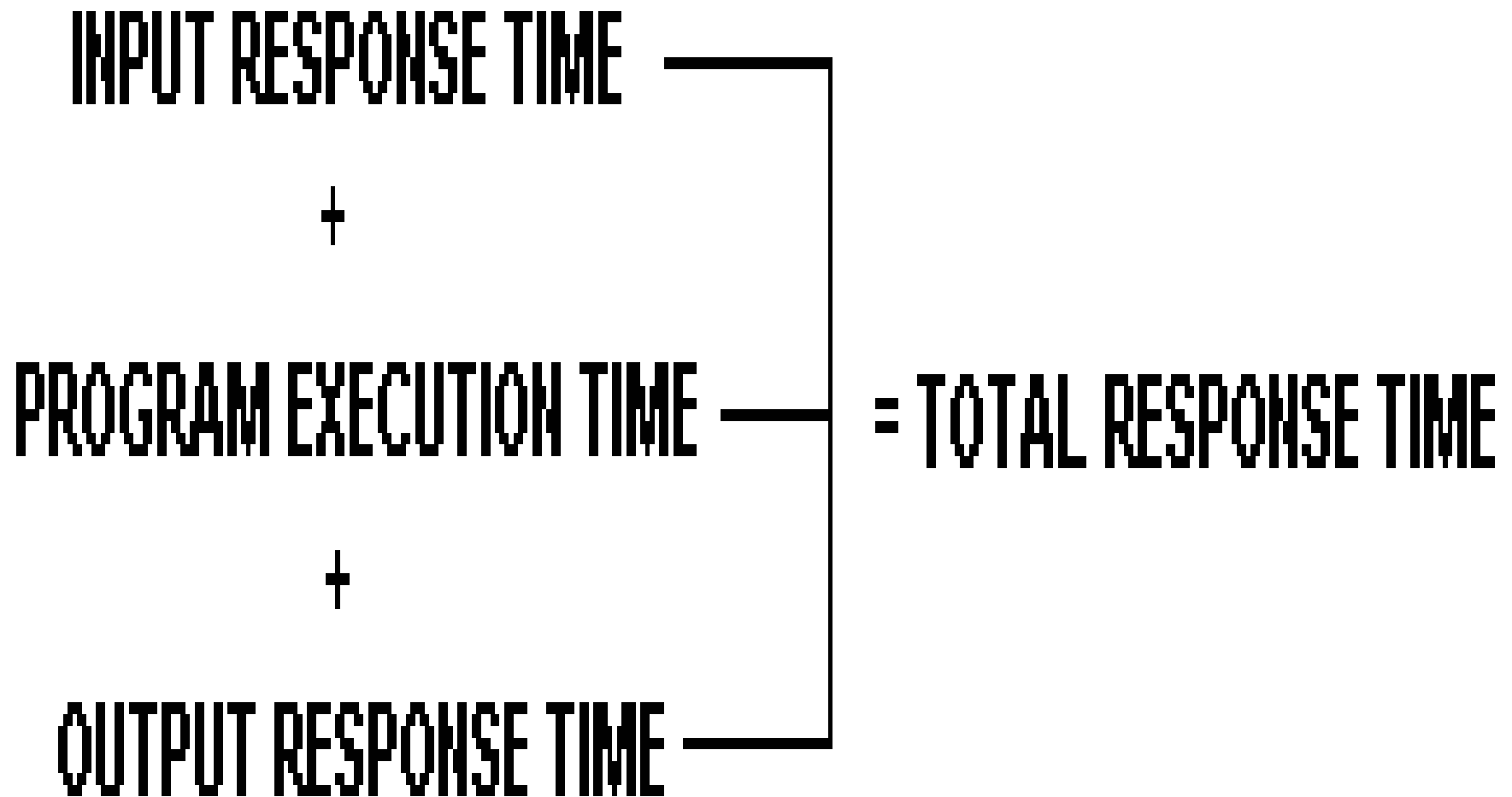
PLC Operation (Scan)



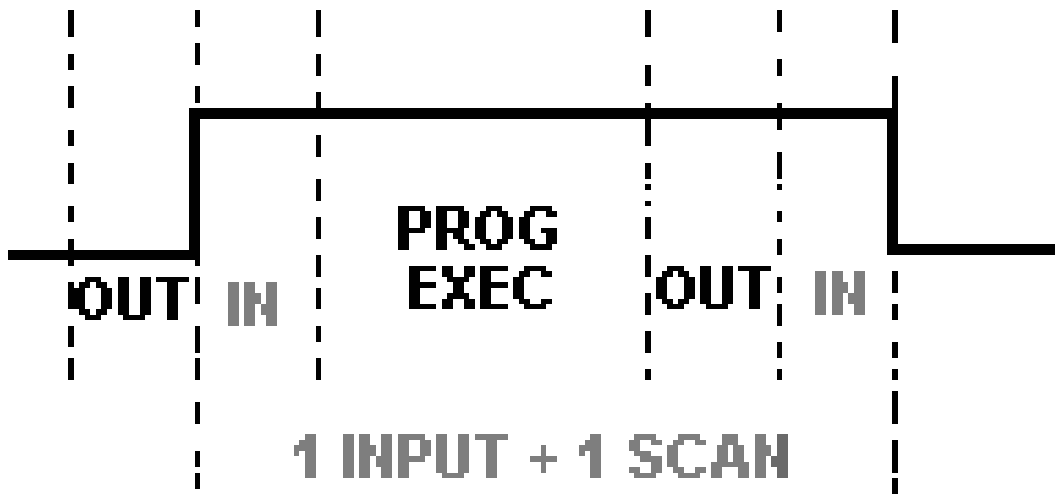
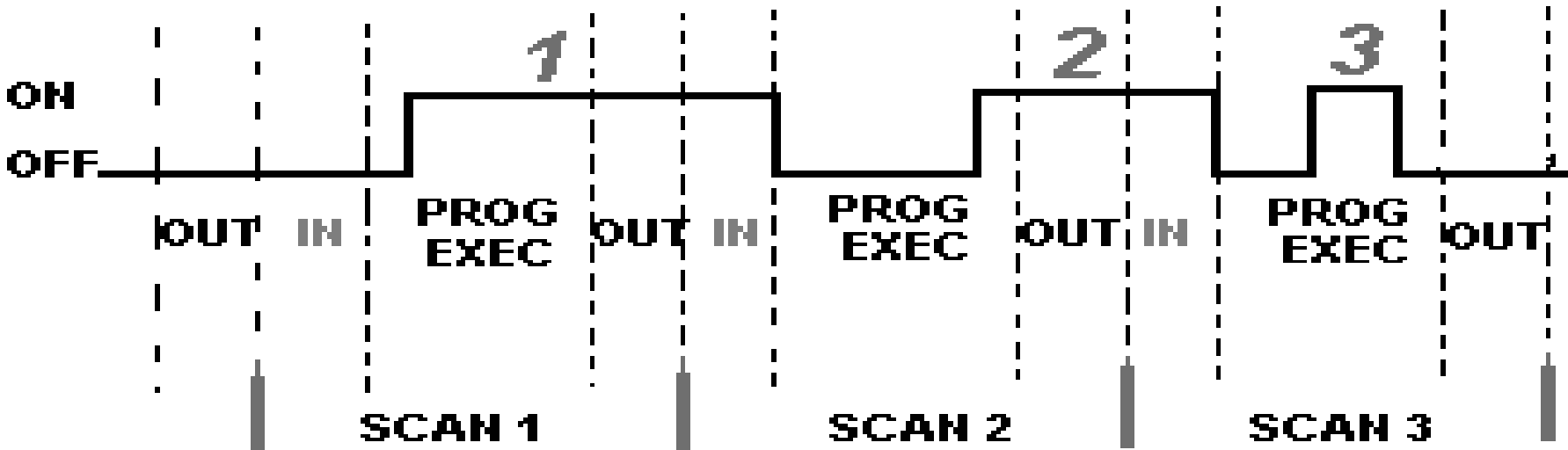
scan time

- One scan time is defined as the time it takes to execute the 3 steps listed previously.
- The scan time varies depends on the program length.
- This process is run in an endless cycle.

Response Time

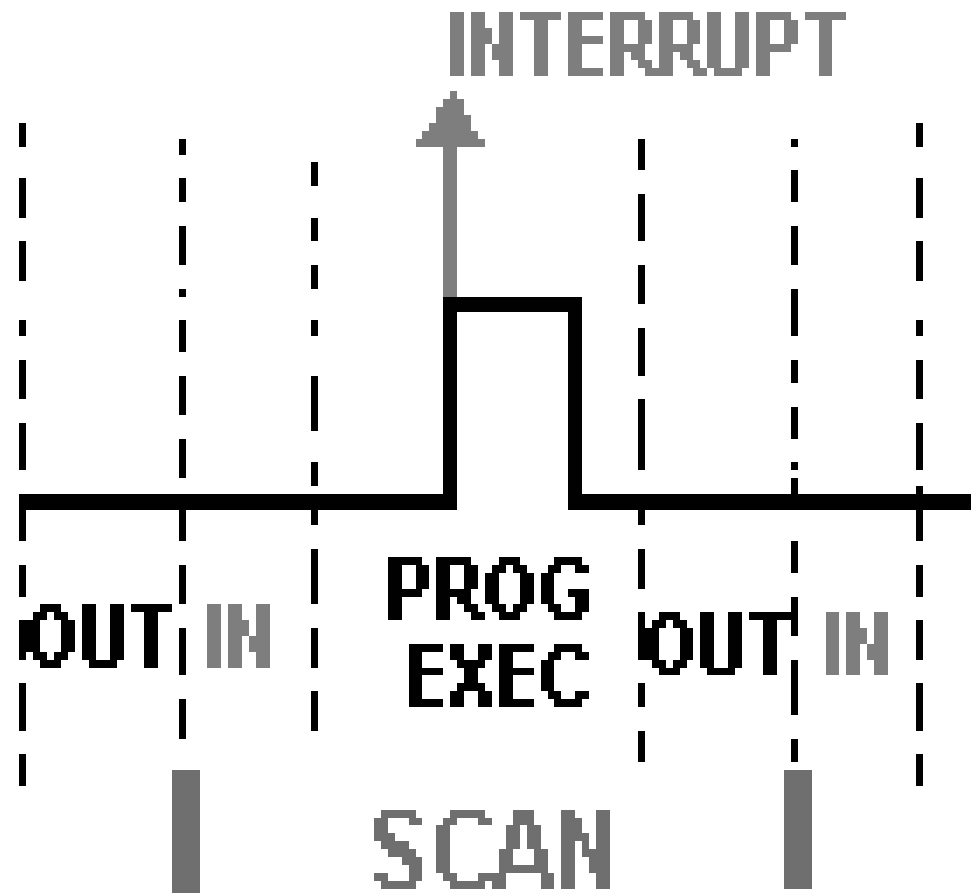
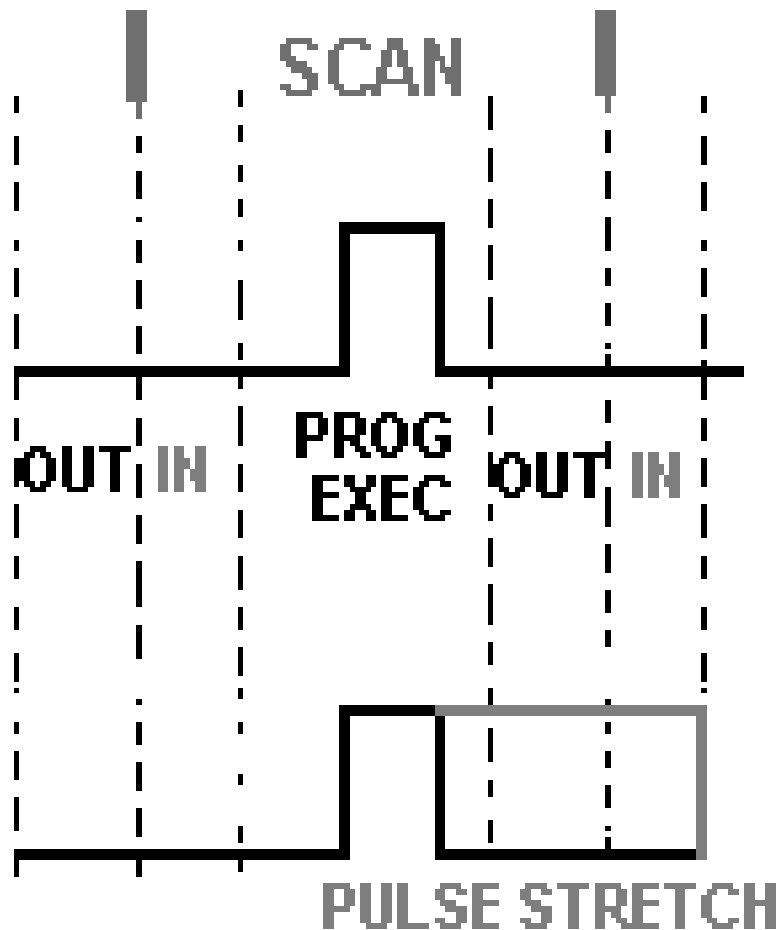


Check Input Status



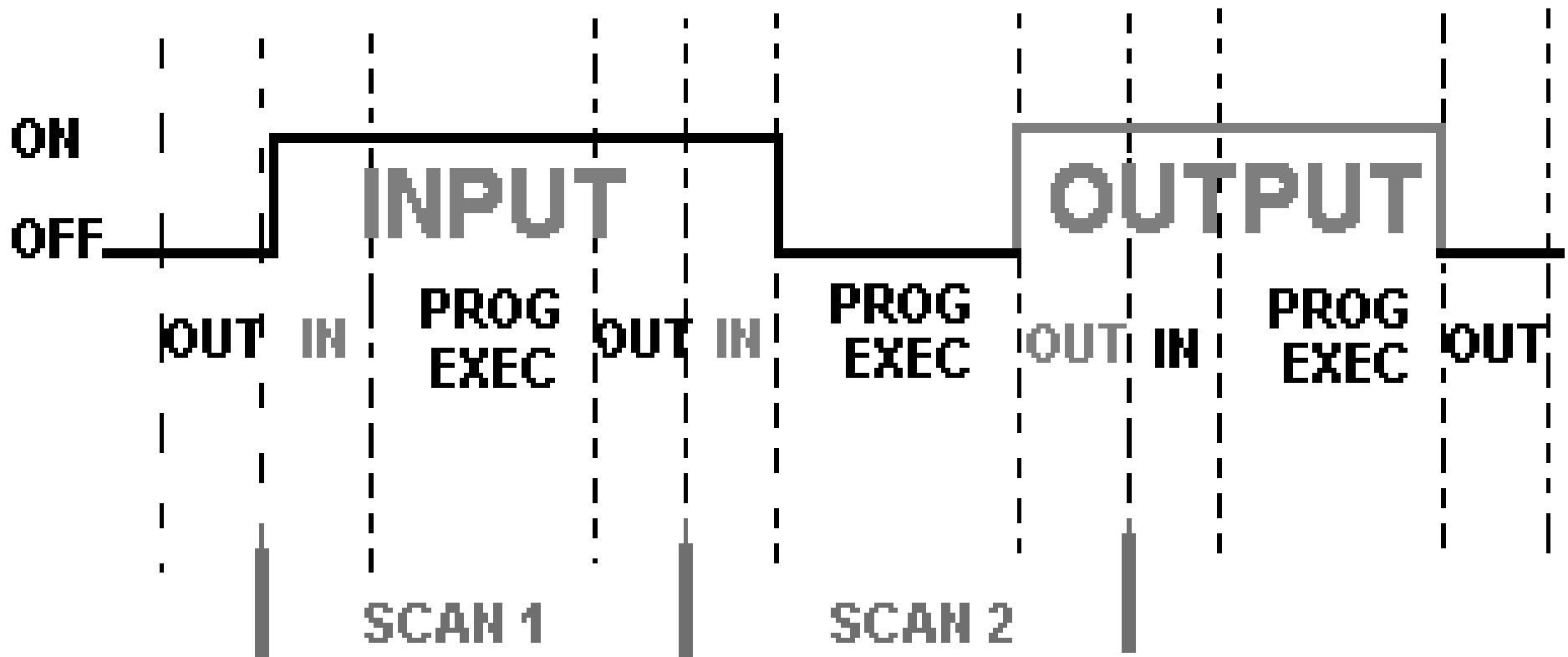
the input should be
on for at least
**1 input delay time
+ one scan time.**

Control Input length



longest Output Delay

The maximum delay is thus
2 scan cycles - 1 input delay time



PLC working modes

PLC controller has three modes:

- PROGRAM/STOP
- RUN and
- MONITOR mode.

I/O ADDRESSING

- **The PLC has to be able to identify each particular input and output.**
- **It does this by allocating addresses to each input and output.**
- **I/O addresses usually assigned to their related I/O terminals.**

- **With small PLC this is likely to be just a number, prefixed by a letter to indicate whether it is an input or an output.**
- **Larger PLCs have its input and output addresses in terms of the module number and the terminal number within that module.**

➤ When you write your program, you can use either of three modes of addressing instruction operands:

- Direct
- Symbolic
- Indirect

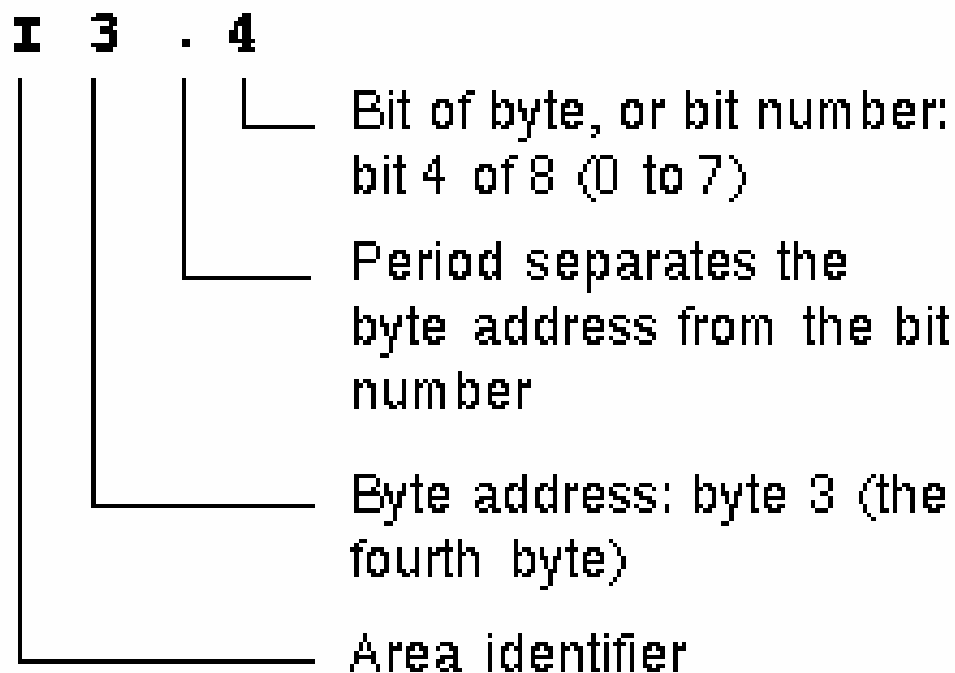
Direct Addressing

- The PLC stores information in different memory locations that have unique addresses.
- This allows program to have direct access to the information (explicitly identify the memory address).

- Direct addressing specifies the memory area, size, and location.
- To access a bit in a memory area, specify the address, which includes:
 - **the memory area identifier,**
 - **the byte address, and**
 - **the bit number preceded by a period.**

- This is an example of accessing a bit (which is "byte.bit" addressing).

In this example, the memory area and byte address (I = input, and 3 = byte 3) are followed by a period (".") to separate the bit



Process-image Input (I) Memory Area

	7	6	5	4	3	2	1	0
Byte 0								
Byte 1								
Byte 2								
Byte 3								
Byte 4								
Byte 5								

Symbolic Addressing

- Symbolic addressing uses a combination of alphanumeric characters to identify an address.
- For SIMATIC programs, you make global symbol assignments by using the symbol table.
- If you have assigned symbolic addresses in the SIMATIC Symbol Table, you can toggle to switch between viewing parameter addresses with Absolute (i.e., I0.0) or Symbolic (i.e., Pump1) representation.

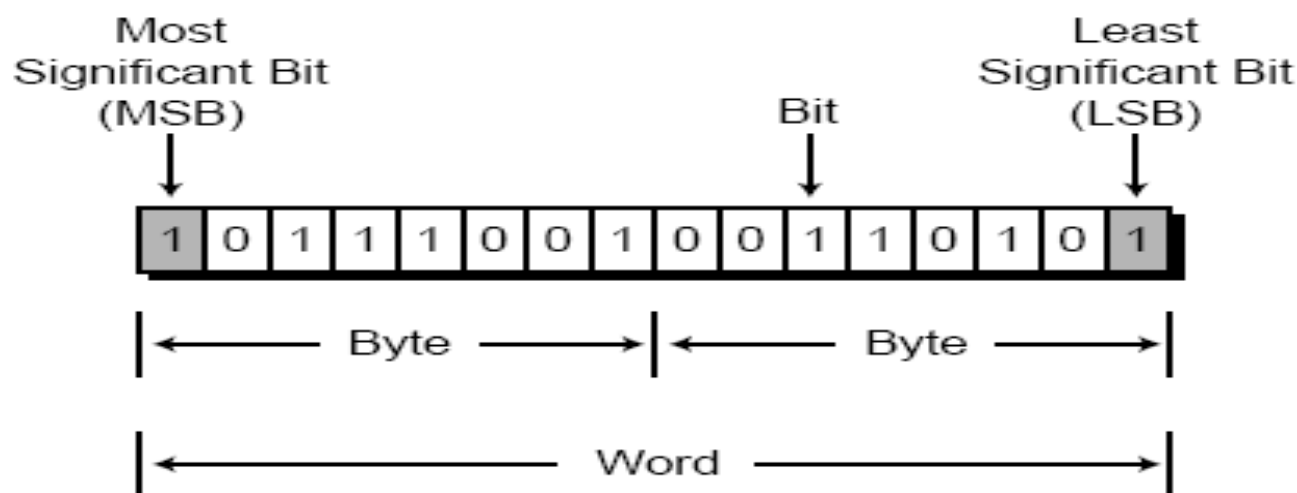
Indirect Addressing

- Indirect addressing uses a pointer to access the data in memory.
- Pointers are **double word memory** locations that contain the address of another memory location. You can only use V memory locations, L memory locations, or accumulator registers (AC1, AC2, AC3) as pointers.
- To create a pointer, you must use the Move Double Word instruction to move the address of the memory location to the pointer location.

Data Blocks

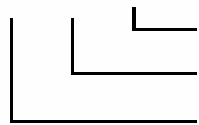
- Timers, counters and individual relays are all concerned with the handling of individual bits, i.e. single on-off signal.
- PLC operations involve blocks of data representing a value, such blocks being termed **words**.

- Each digit of a binary number is known as a **bit**;
- A group of 8 bits is a **byte**; and
- A group of one or more bytes is a **word**.
- Figure below presents a binary number composed of 16 bits, with the **least significant bit (LSB)**, the lowest valued bit in the word, and the **most significant bit (MSB)**, the largest valued bit in the word, identified.



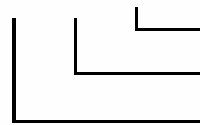
Bit/Byte/Word/Double Word

V B 100



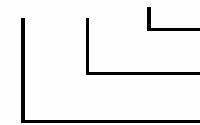
Byte address
Access to a byte size
Area identifier

V W 100

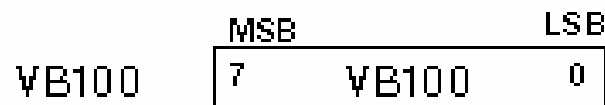


Byte address
Access to a word size
Area identifier

V D 100

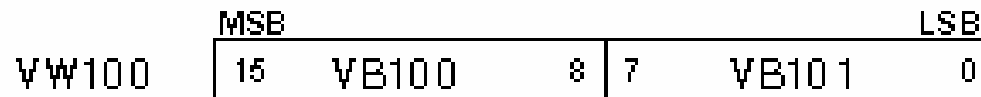


Byte address
Access to a double word size
Area identifier



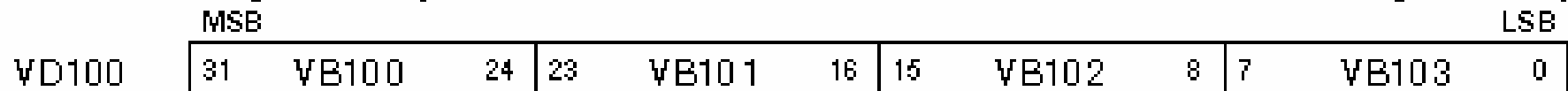
Most significant byte

Least significant byte



Most significant byte

Least significant byte



MSB = most significant bit
LSB = least significant bit

PLC Coding Systems

- In general, there are **two** categories of codes:
 - those that represent **numbers** only,
 - those that represent **letters, symbols, and decimal numbers**

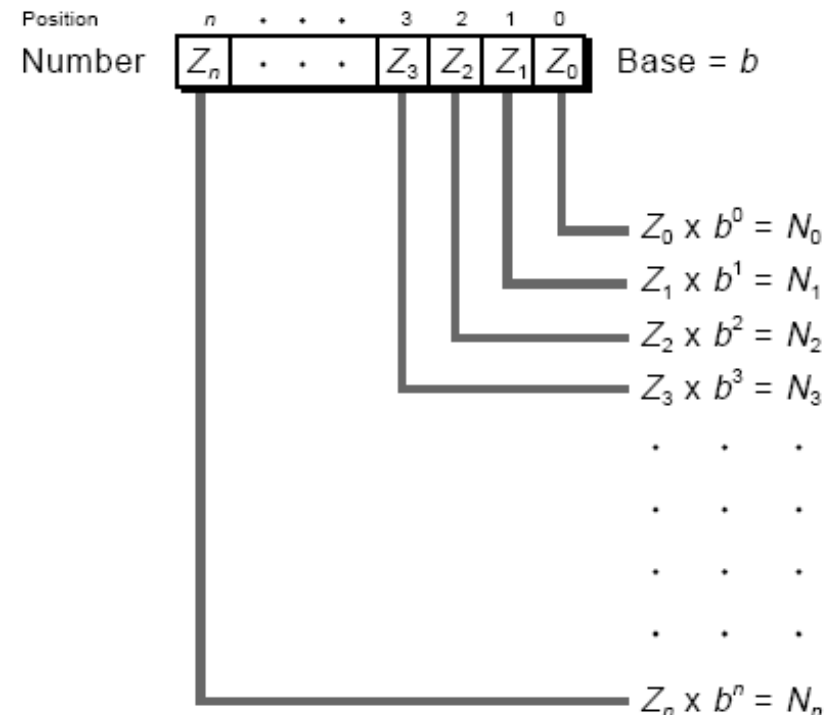
➤ **Four number systems** are the most frequently used during input/output address assignment and programming:

binary (base 2),

octal (base 8),

decimal (base 10), and

hexadecimal (base 16)



- Several codes for representing numbers, symbols, and letters are standard throughout the industry.
- Among the most common are the following:
 - **BCD** (binary coded decimal) format is often used with PLCs when they are connected to devices such as digital displays.
 - **Gray code**, and
 - **ASCII** (the American Standard Code for Information Interchange) is the most common code for alphanumeric representation.

REGISTER WORD FORMATS

- A PLC performs all of its internal operations in binary format using 1s and 0s.
- The status of I/O field devices is also read and written, in binary form, to and from the PLC's CPU.
- Generally, these operations are performed using a group of 16 bits that represent numbers and codes.

- The grouping of bits with which a PLC operates is called a word (a **register** or *location*).
- Although the data stored in a register is represented by binary 1s and 0s, the format in which this binary data is stored may differ from one PLC to another.
- Generally, data is represented in either straight binary or binary coded decimal (BCD) format.

BINARY FORMATS

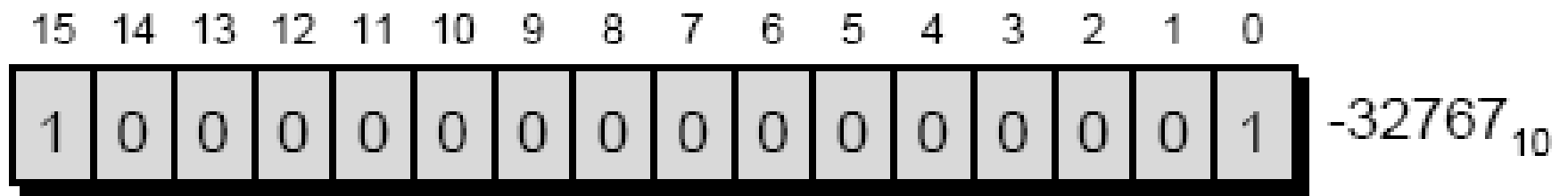
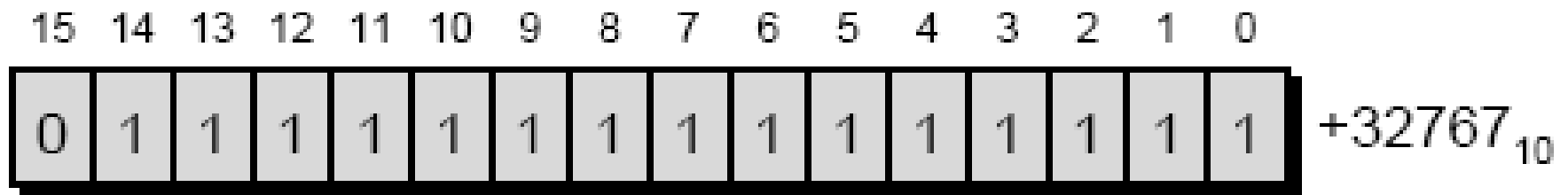
- Data stored in binary format can be directly converted to its decimal equivalent without any special restrictions.
- A **register** is where data can be stored.
- Each data register can store a binary word of usually 8 or 16 bits.

- The number of bits determines the size of the number that can be stored ($2^n - 1$).
 - 4-bit register can store a positive number between 0 and +15.
 - 8-bit: 0 and +255.
 - 16-bit: 0 and +65535;

- If **negative numbers** are required, the MSB is used to represent the sign:
 - 0 representing a positive number
 - 1 representing a **negative** number
 - Format used for the negative numbers is two's complement (*each bit, from right to left, is inverted only after the first 1 is detected*).

- In this format, a 16-bit register can represent a maximum value of 65535.
- If the MSB of the register is used as a sign bit, then the maximum decimal value that the 16-bit register can store is +32767 or -32767.

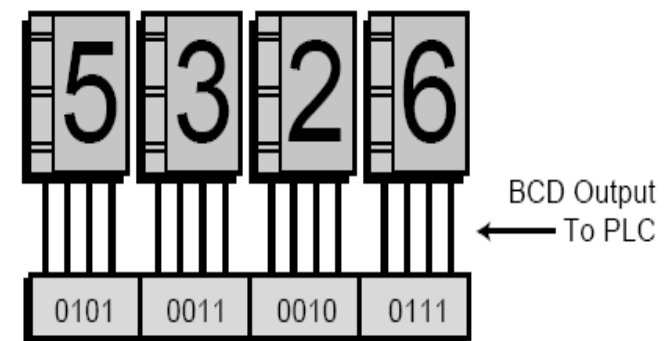
Sign Bit



The negative representation of $32767_{(10)}$ derived using the two's complement method

BCD FORMAT

- The BCD representation of a decimal number is obtained by replacing each decimal digit with its BCD equivalent (4-bit binary number).
- Some PLCs operate and store data in several of their software instructions, such as arithmetic and data manipulations, using the BCD format.
- In BCD format, a 16-bit register can hold up to a 4-digit decimal value, with the decimal values that can be represented ranging from 0000–9999.



- Typical PLC applications of BCD codes include data entry (time, volume, weight, etc.) via thumbwheel switches, data display via seven segment displays, input from absolute encoders, and analog input/output instructions.
- This BCD data is converted internally by the PLC into the binary equivalent of the input data.
- Input and output of BCD data requires four lines of an input/output interface for each decimal digit.

GRAY CODE

- The **Gray code** is one of a series of cyclic codes known as reflected codes and is suited primarily for position transducers.
- It is basically a binary code that has been modified in such a way that only one bit changes as the counting number increases.
- Most encoders use the Gray code to determine angular position.

Decimal – Binary - Gray

Gray Code	Binary	Decimal
0000	0	0
0001	1	1
0011	10	2
0010	11	3
0110	100	4
0111	101	5
0101	110	6
0100	111	7
1100	1000	8
1101	1001	9
1111	1010	10
1110	1011	11
1010	1100	12
1011	1101	13
1001	1110	14
1000	1111	15

ASCII

- Standard ASCII character sets use a 7-bit code ($2^7 = 128$ possible characters).
- This 7-bit code provides all possible combinations of characters used when communicating with peripherals and interfaces.

➤ Next Class

PLC COMMUNICATIONS

PLC PROGRAMMING FUNDAMENTALS

Dr. Mazouz Salahat

Objectives

- Be familiar with different programming languages.

Topics

- Programming languages standardization.
- Introduction to
 - Ladder program
 - Statement list
 - Functional blocks.

Steps of PLC Program Planning

- Define the process to be controlled.
- Define the algorithm to be used.
- Translate the algorithm into PLC instructions.
- Test and debug the algorithm.
- Document the program.

Program Options

- Almost every program for programming a PLC possesses various useful options:
 - Forced switching on and off of the system inputs/outputs (I/O lines),
 - Program follow up in real time,
 - Documenting a diagram.

Programming languages

- A program loaded into PLC systems in **machine code**, a sequence of binary code numbers to represent the program instructions.
- Assembly language based on the use of mnemonics can be used, and a computer program called an assembler is used to translate the mnemonics into machine code.
- High level Languages (C, BASIC, etc.) can be used.
- Graphical Languages.

IEC Standards

- Standard IEC 1311 part 3 classifies programming methods into two categories:
 - **Textual** language, involves text and has two methods:
 - Instruction list
 - Structured text
 - **Graphical** language, involves graphical images and has two methods:
 - Ladder diagrams
 - Function block diagrams

Introduction to Ladder Logic

- Ladder logic uses graphic symbols similar to relay schematic circuit diagrams.
- Ladder diagram consists of two vertical lines representing the power rails. Circuits are connected as horizontal lines between these two verticals.

ladder diagram features

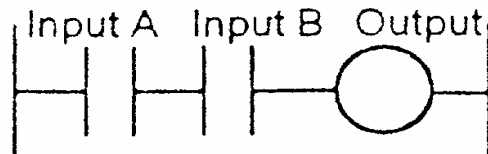
- Power flows from left to right.
- Output on right side can not be connected directly with left side.
- Contact can not be placed on the right of output.
- Each rung contains one output at least.
- Each output can be used only once in the program.
- A particular input a/o output can appear in more than one rung of a ladder.
- The inputs a/o outputs are all identified by their addresses, the notation used depending on the PLC manufacturer.

ladder logic basic elements

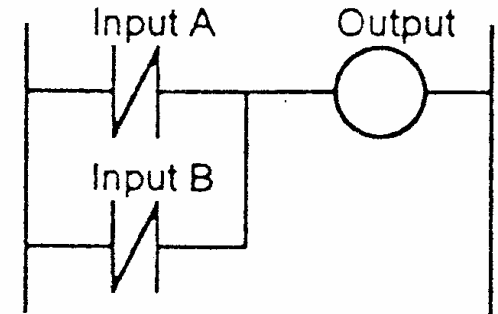
- **Contacts (NO or NC).**
- **Coils.**
- **Boxes.**

Basic ladder logic functions

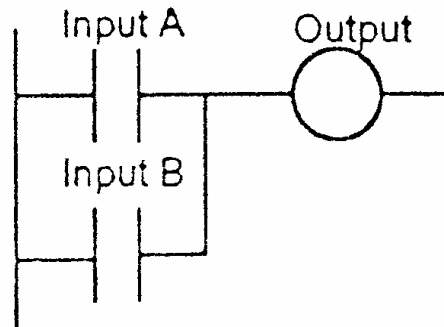
➤ AND



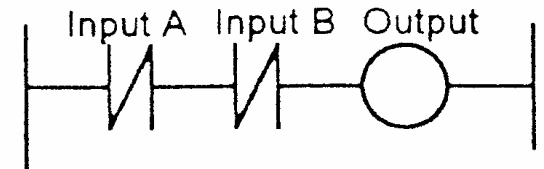
➤ NAND



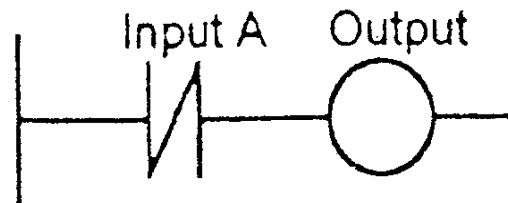
➤ OR



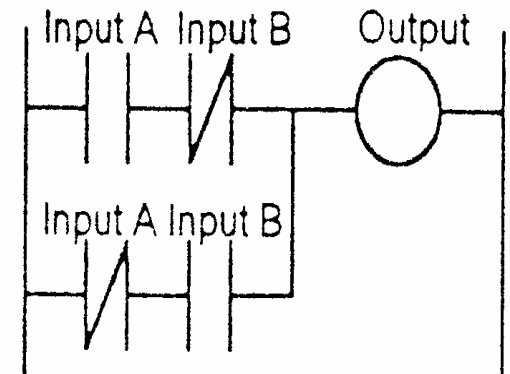
➤ NOR



➤ NOT



➤ XOR



Introduction to Statement list

- Statement list is a programming language using mnemonic abbreviations of Boolean logic operations.
- Boolean operations work on combination of variables that are true or false.

Statement Parts

- **A statement** is an instruction or directive for the PLC.
 - **Operation:** what operation is to be done,
 - **Operand:** what item is to be operated on by the operation.

Statement list Operations

- Load (LD) instruction.
- And (A) instruction.
- Or (O) instruction.
- Output (=) instruction.

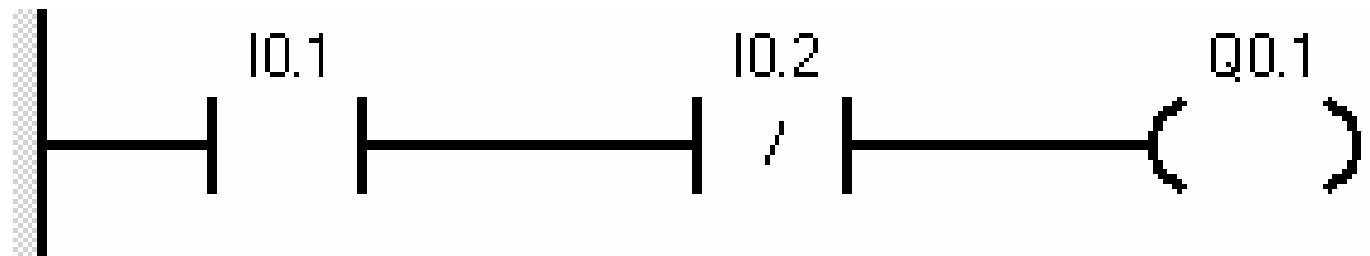
Introduction to Function Block Diagrams

- Function block is represented as a box with the function name written in.

Examples

AND

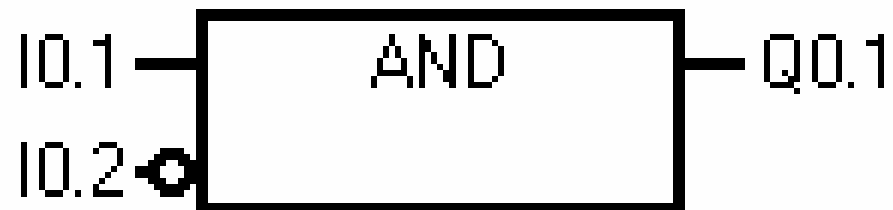
➤ Ladder



➤ Statement List

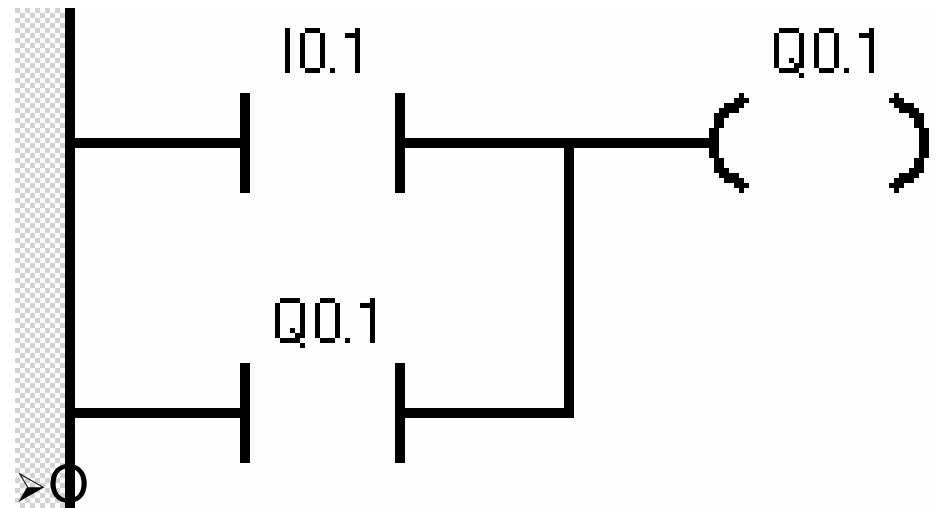
```
LD    I0.1
AN    I0.2
=     Q0.1
```

➤ Functional Block



OR (Feedback)

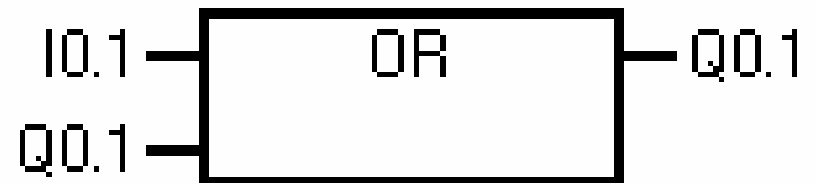
➤ Ladder



➤ Statement List (STL)

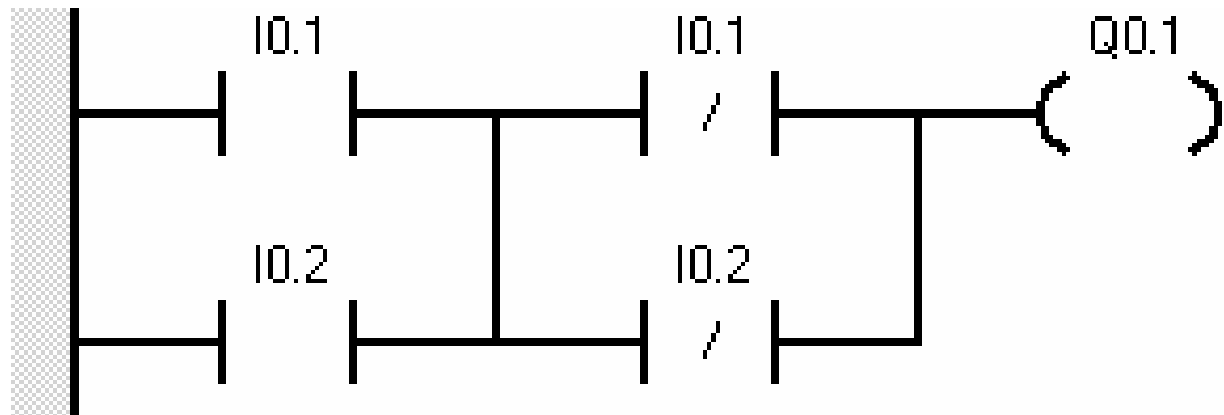
```
LD      I0.1
O        Q0.1
=        Q0.1
```

➤ Functional Block (FBD)



XOR

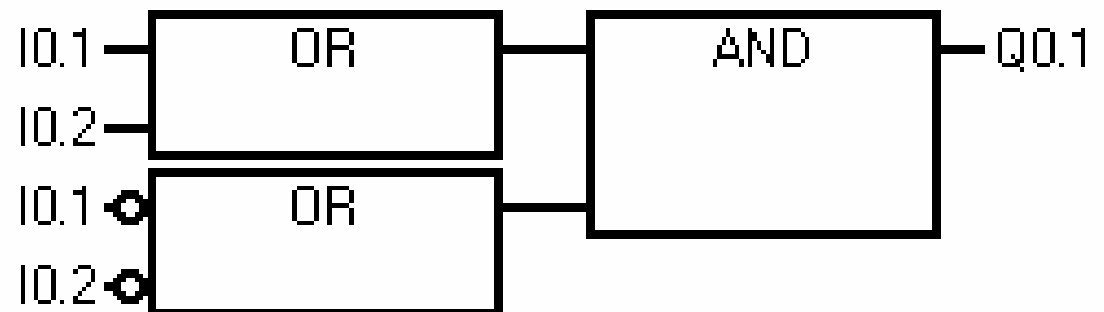
➤ Ladder



➤ Statement List (STL)

```
LD      I0.1
O       I0.2
LDN     I0.1
ON      I0.2
ALD
=       Q0.1
```

➤ Functional Block (FBD)



➤ Next Class

BASIC (STANDARD) PROGRAMMING INSTRUCTIONS

BASIC PROGRAMMING INSTRUCTIONS

Dr. Mazouz Salahat

Objectives

- Understand and implement basic programming instructions.

Topics

- Relay Operation.
- Bit instructions (I, O, M, OSR, S, R)
- Timer instructions.
- Counter instructions.

PLC Programming Instructions

- Relay-type (Basic) instructions
- Data Handling Instructions.
 - Move Instructions
 - Comparison instructions
 - Mathematical instructions
 - Continuous Control Instructions (PID).
- Program flow control instructions
- High speed counter instructions
- Communication instructions
- ASCII instructions.

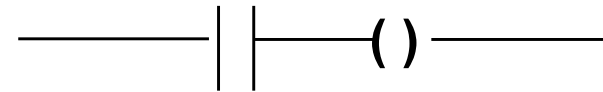
Relay-type (Basic) instructions

- Input relay I
- Output relay O
- One shot relay (pulse)
- SET and RESET
- Timer T
- Counter C

RELAY

A Relay consists of two parts, the coil and the contact(s)

Contacts:



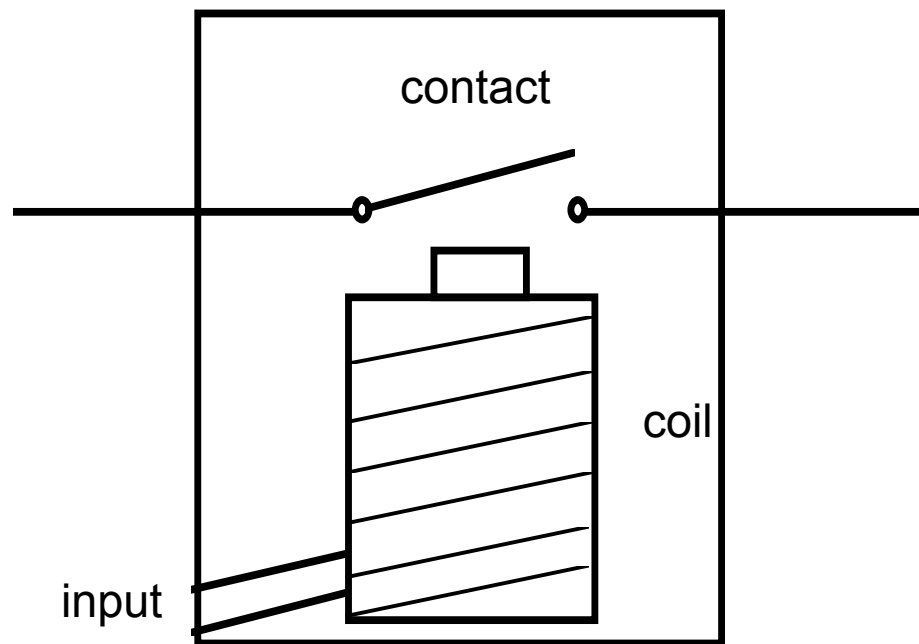
- a. Normally open - | | -
- b. Normally closed - | / | -
- c. Off-on transitional - | ↑ | -
- d. On-off transitional - | ↓ | -

Coil:

- a. Energize Coil - () -
- b. Latch (SET) - (S) -
- c. Unlatch (RESET) - (R) -

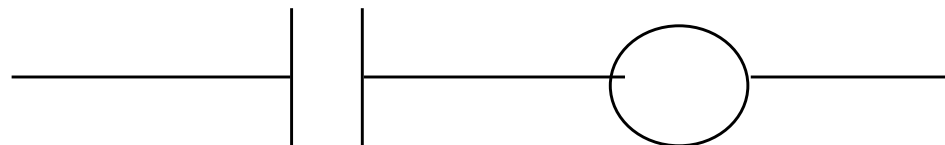
CONTROL RELAYS

A switch whose operation is activated by an electromagnet is called a "relay"

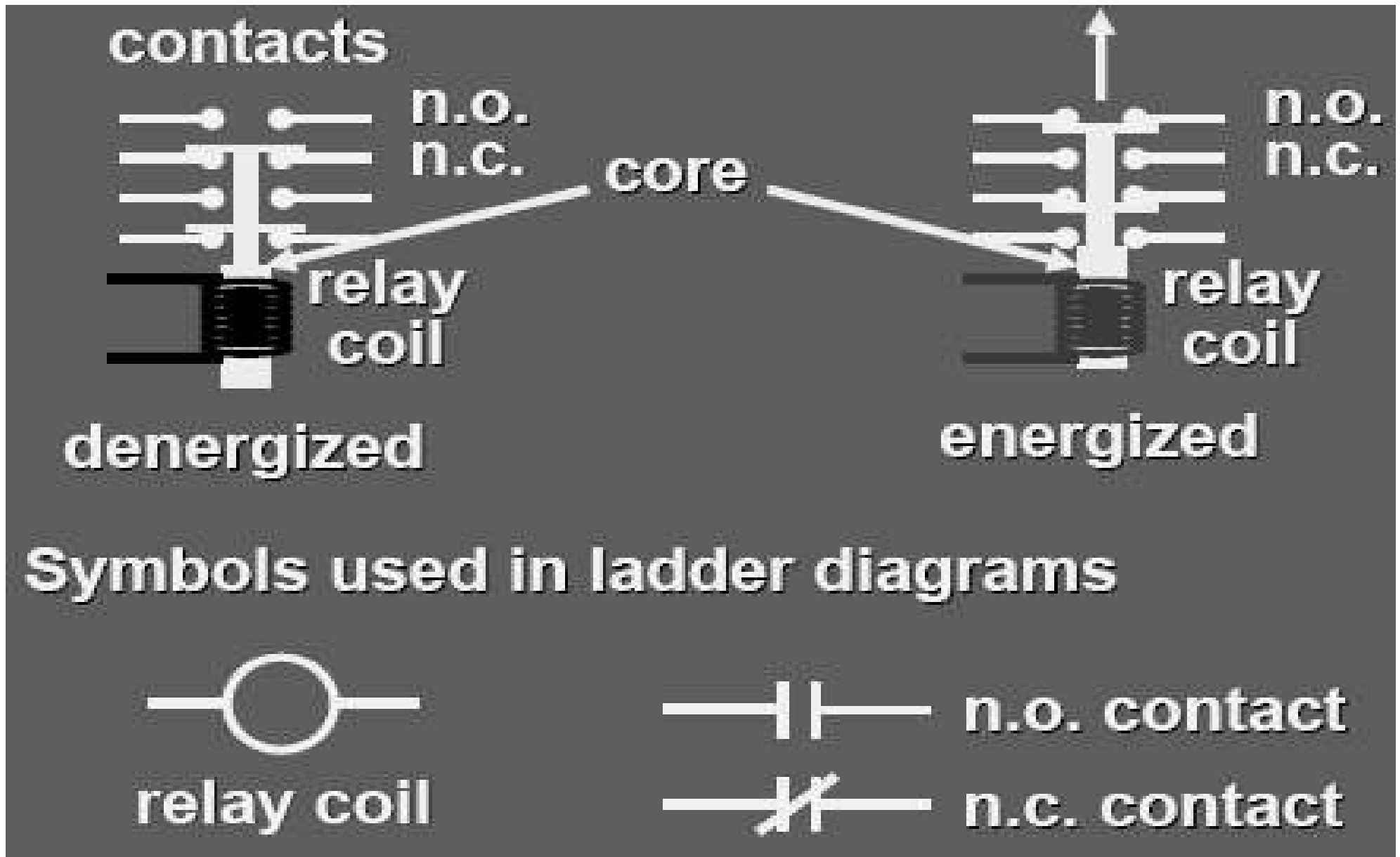


Relay coil

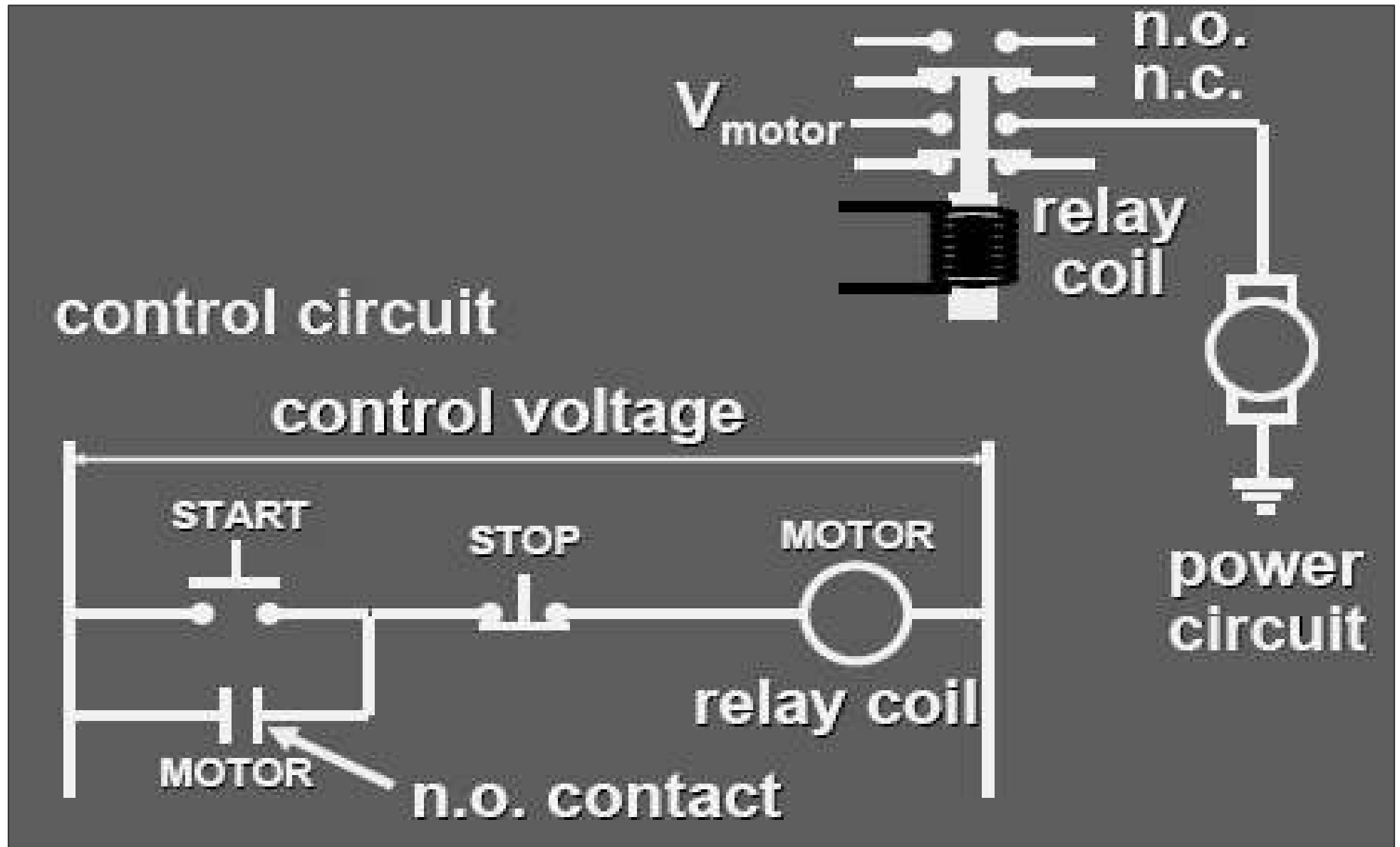
Output contact



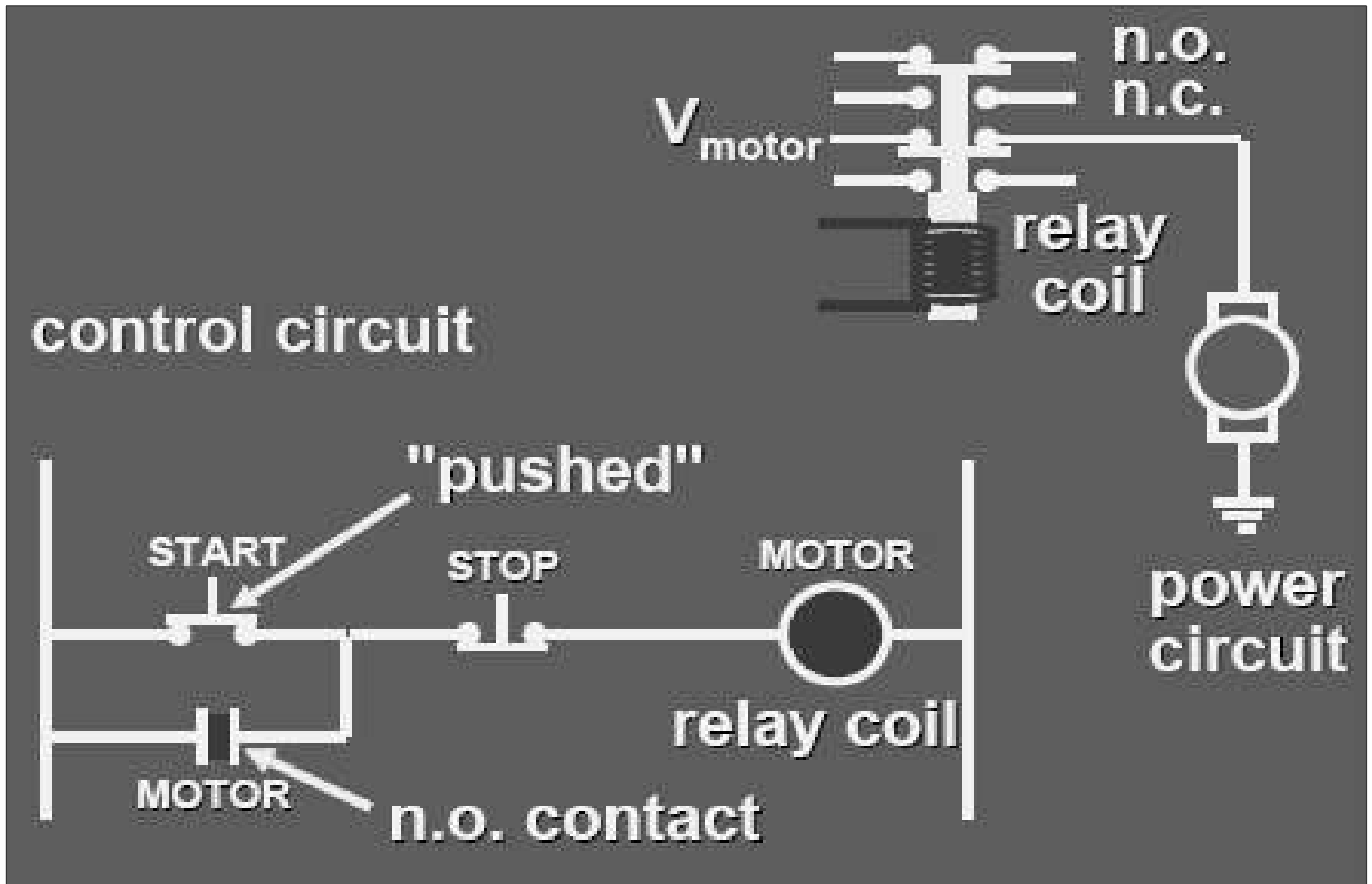
Control Relays



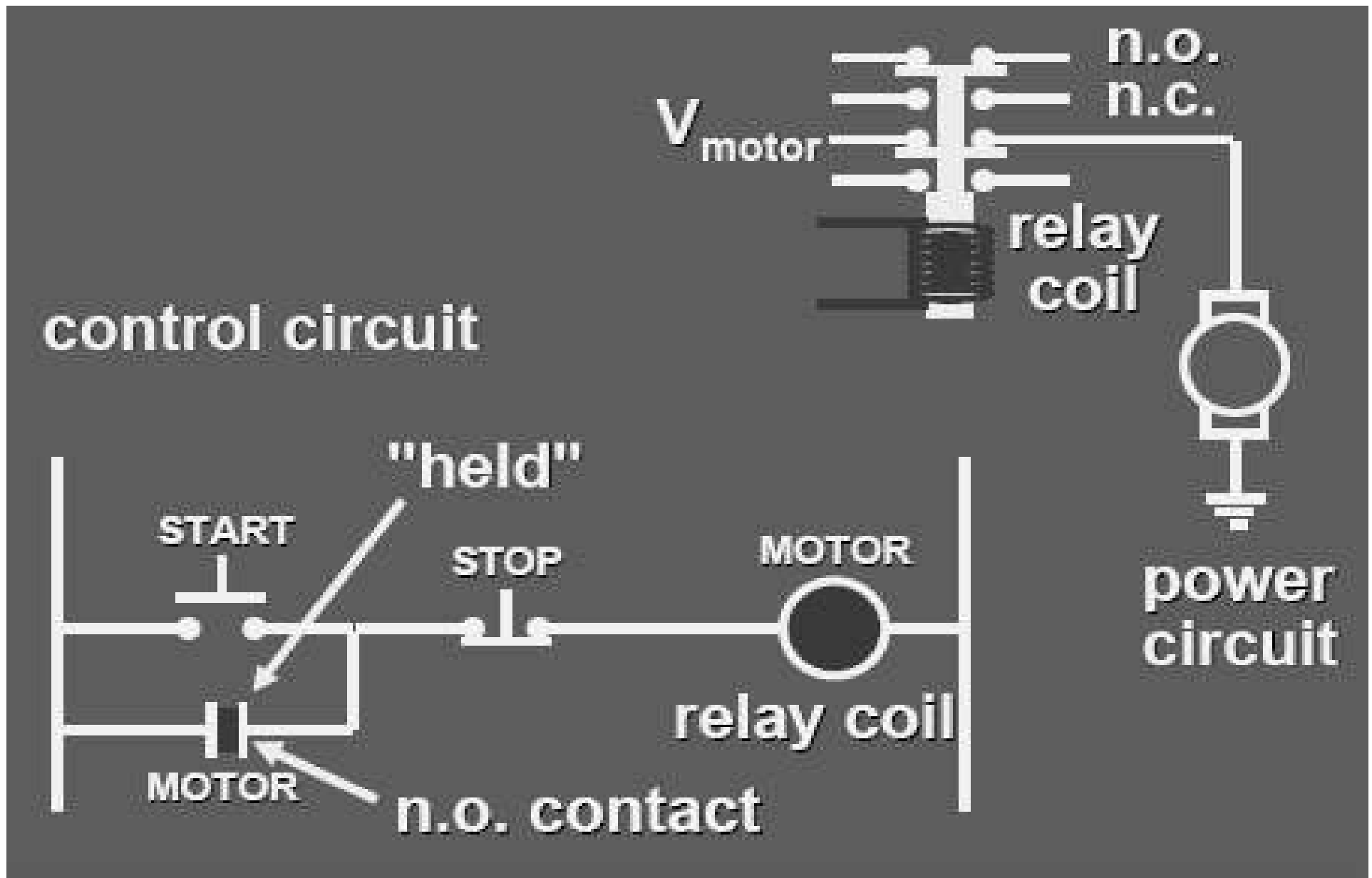
Motor Control Ladder Diagram



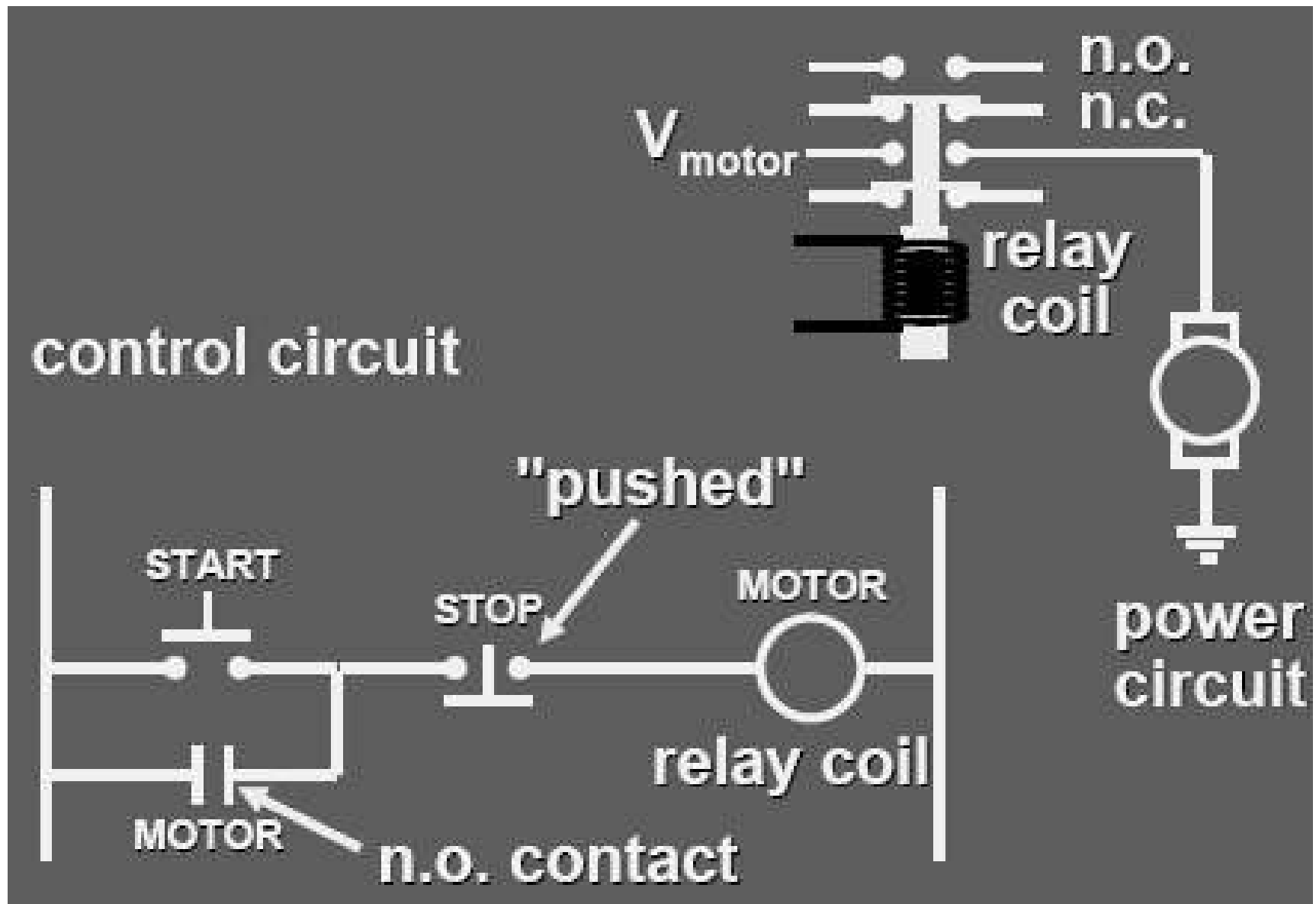
START Pushed - Motor On



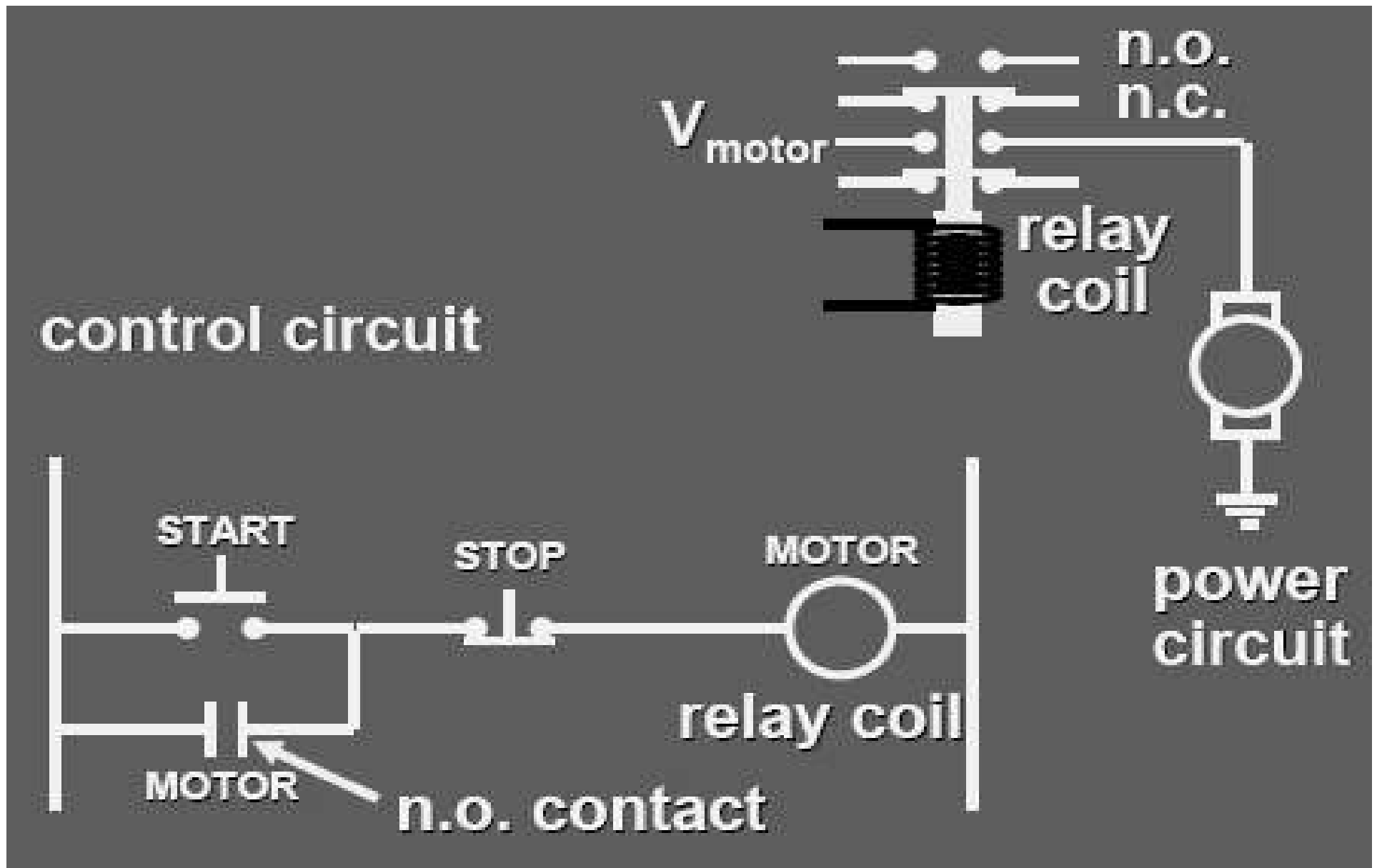
START Released – Motor Remains On



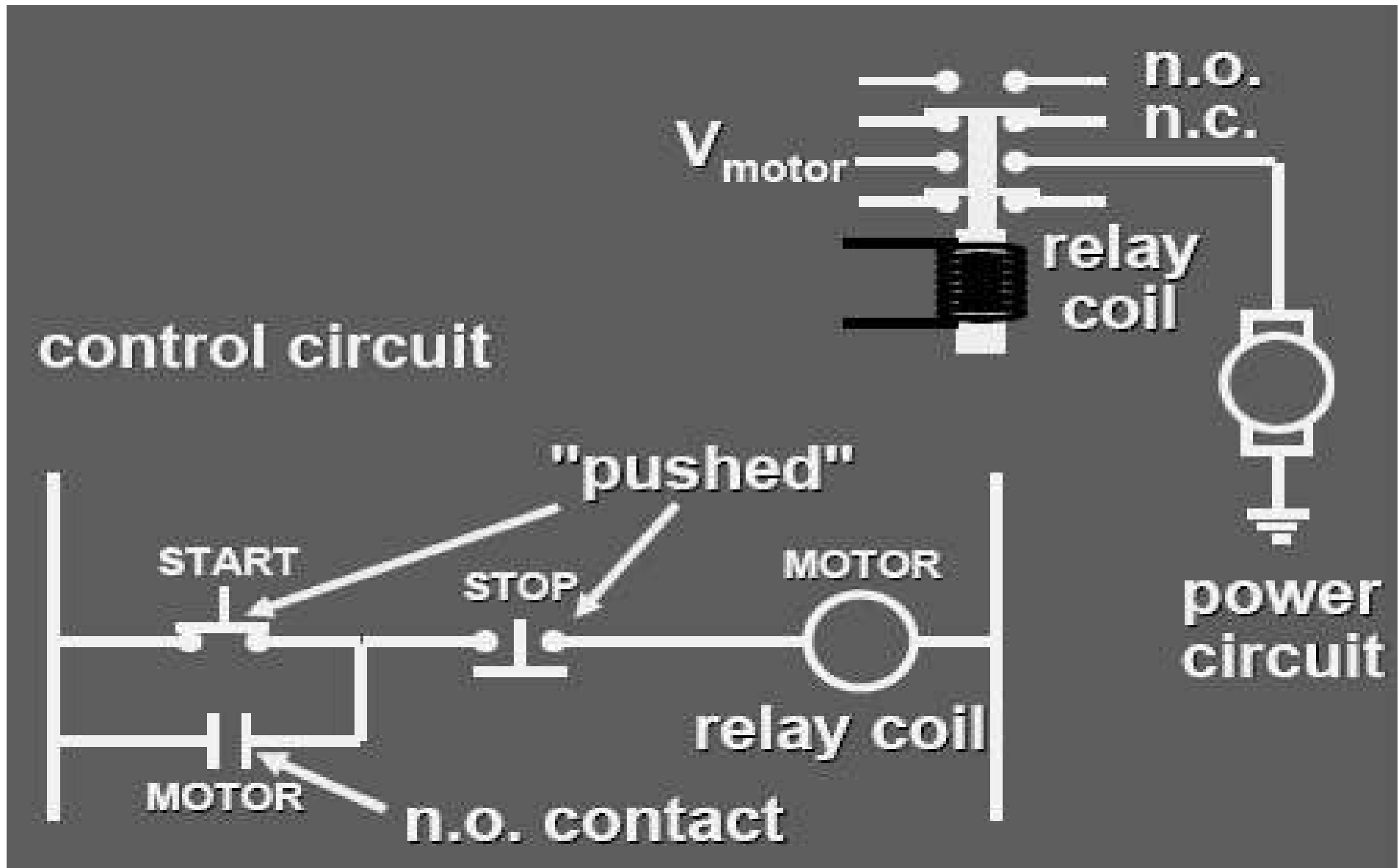
STOP Pushed – Motor OFF



STOP Released – Motor Remains Off



Off Dominant



Basic Instructions

Relay-type (Basic) Instructions

■ Examine

instructions:

- Examine On.
- Examine Off.

■ Branch

instructions:

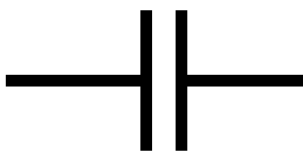
- Branch Start.
- Branch End.

■ Output

instructions:

- Output Energize.
- Output SET.
- Output RESET.

PLC Input Terms

■ **Examine ON** 

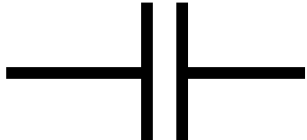
- Functions like normally open.
- PLC will examine the reference address (coil). If it is on (energized) continuity will continue on the rung.

PLC Input Terms

■ Examine OFF

- Functions like normally closed.
- PLC will examine the reference address (coil). If it is off (de-energized), logic continuity is continued through the rung.

Example of Examine ON

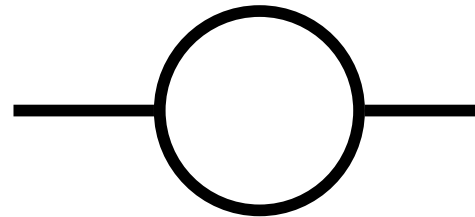
- External contact is NC stop button.
- Examine ON() will show that contact is closed (1 stored).
 - The rung is true (continuity).
- If the button is pushed, a zero is stored.
 - The rung is false.
- *Choose the physical input switch with the desired action (N.O. or N.C.)*

PLC Outputs

- **Two types**

- Coils used internal to the program.
- Coils used to control external devices.

- Coils can have unlimited N.O. or N.C. contacts associated with them.



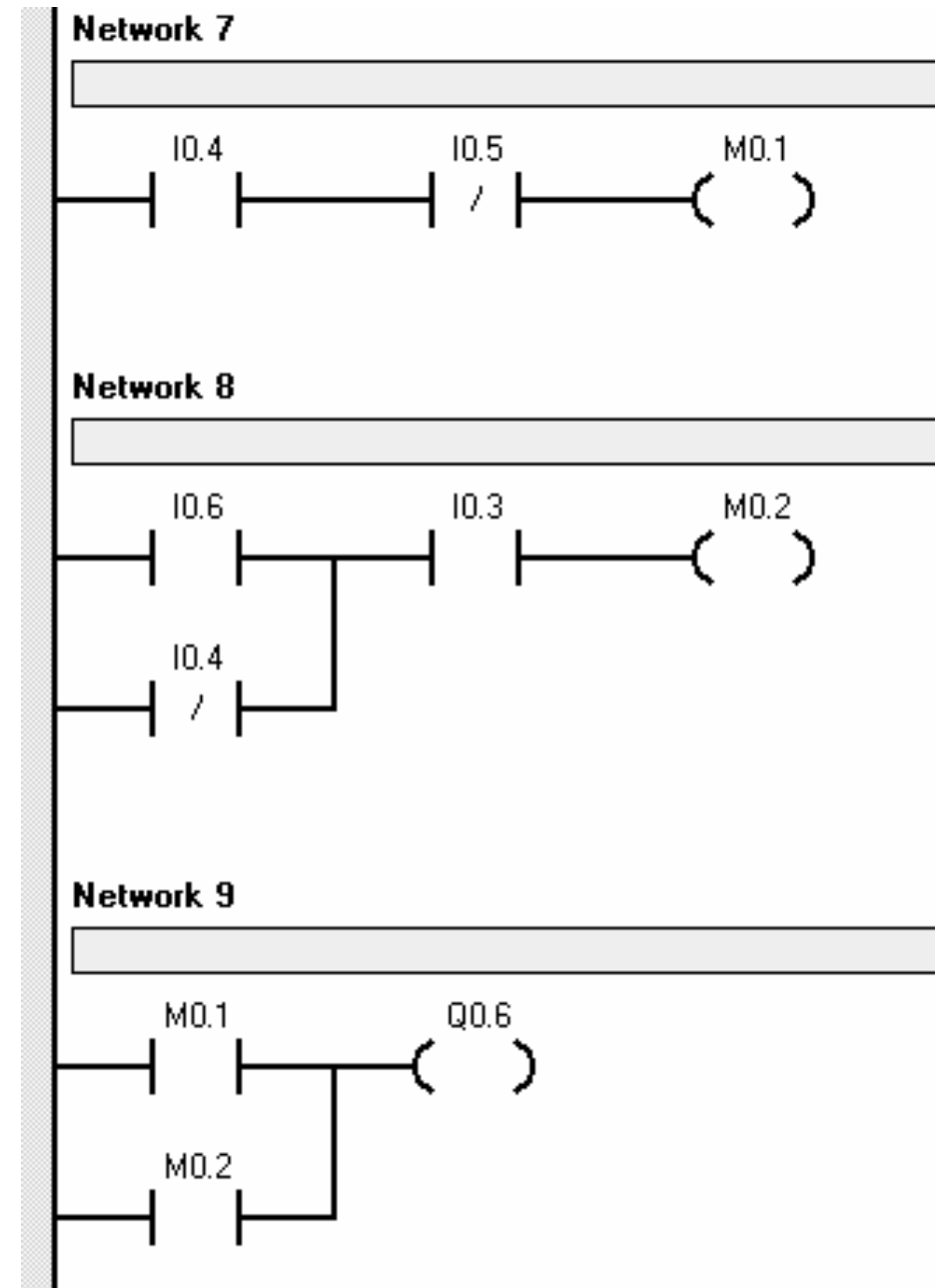
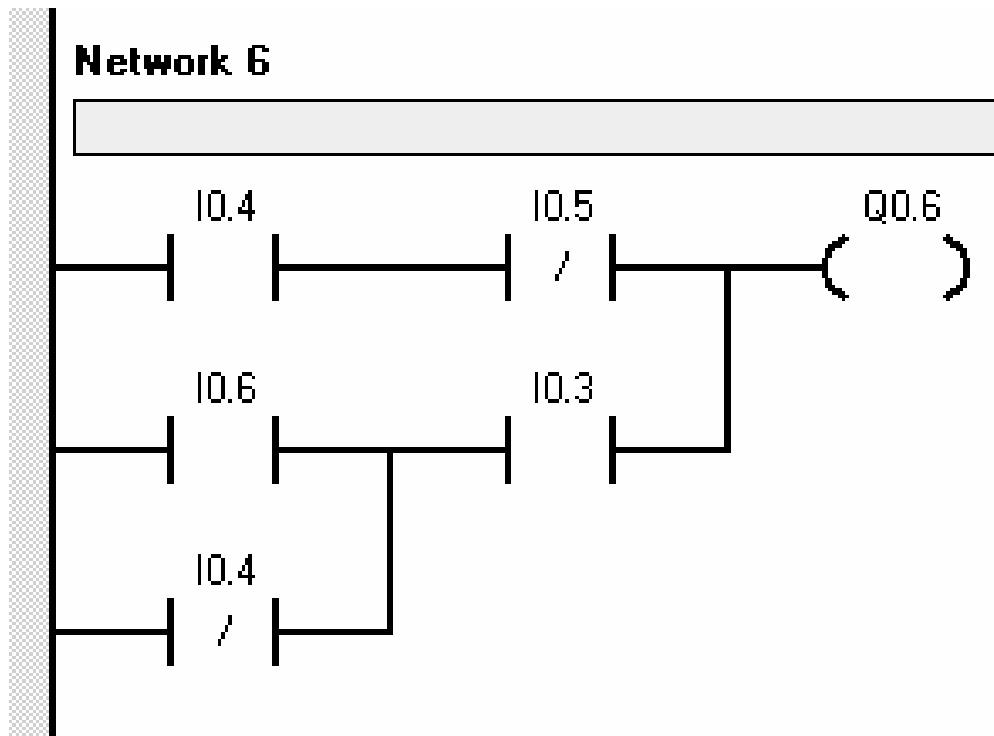
Internal Relays

- Auxiliary relays, markers, flags, coils, bit storage.
- Used to hold data, and behave like relays, being able to be switched on or off and switch other devices on or off.
- They do not exist as real-world switching devices but are merely bits in the storage memory.

Internal Relays Use

- In programs with multiple input conditions or arrangements.
- For latching a circuit and for resetting a latch circuit.
- Giving special built-in functions with PLCs.

Example



Special Memory

SMB0 System Status Bits

SMB1 Instruction Execution Status Bits

SMB2 Freeport Receive Character

SMB3 Freeport Parity Error

SMB4 Interrupt Queue Overflow, Run-time Program Error, Interrupts Enabled, Freeport Transmitter Idle, Value Forced

SMB5 IO Error Status Bits

SMB6 CPU ID Register

SMB8–SMB21 IO Module ID and Error Registers

SMV22–SMV26 Scan Times

SMB28–SMB29 Analog Adjustment

S7-200	SM	
Symbol Name	Addr.	User programs read SMB0 status data
Always_On	SM0.0	This bit is always ON.
First_Scan_On	SM0.1	This bit is ON for the first scan cycle only. One use is to call an initialization subroutine.
Retentive_Lost	SM0.2	This bit is turned ON for one <u>scan cycle if retentive data was lost</u> . This bit can be used as either an error memory bit or as a mechanism to invoke a special startup sequence.
RUN_Power_Up	SM0.3	This bit is turned ON for one <u>scan cycle when RUN mode is entered from a power-up condition</u> . This bit can be used to provide machine warm-up time before starting an operation.
Clock_60s	SM0.4	This bit provides a clock pulse that is OFF for 30 seconds and ON for 30 seconds, for a cycle time of 1-minute. It provides an easy-to-use delay or a 1-minute clock pulse.
Clock_1s	SM0.5	This bit provides a clock pulse that is OFF for 0.5 seconds and then ON for 0.5 seconds for a cycle time of 1 second. It provides an easy-to-use delay or a 1-second clock pulse.
Clock_Scan	SM0.6	This bit is a scan cycle clock that is ON for one scan and then OFF for the next scan. This bit can be used as a scan counter input.
Mode_Switch	SM0.7	<u>This bit indicates the current position of the Mode switch</u> (OFF=TERM position and ON=RUN position). You can use this bit to enable Freeport mode only when the switch is in the RUN position. Normal communication with the PC/programming device can be re-enabled by switching to the TERM position.

One-shot operations

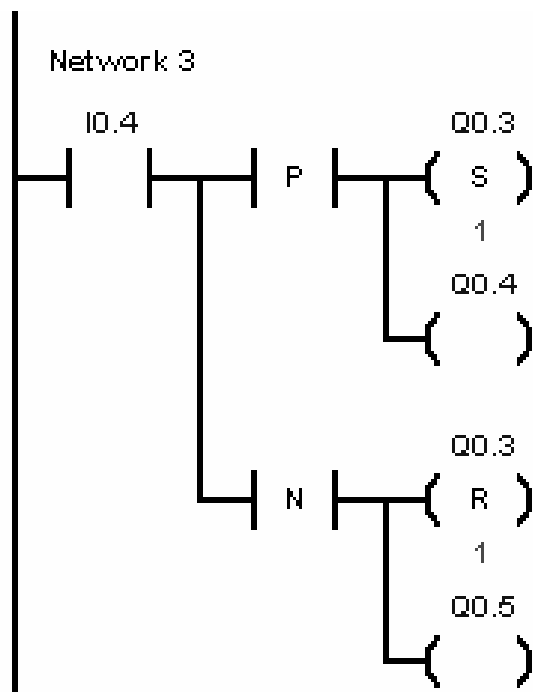
(Pulse operations)

- It provides a fixed duration pulse at its contacts when operated.
- Such function is used to create pulses for the resetting of counters and timers and making the start of cycles.
- One-shots type reacts to an input signal *off to on transition* (positive-going) and a different type reacts to an on to off transition (negative-going).

Latch Instructions

(Set and Reset)

- The **SET** instruction causes the relay to self-hold, (latch). It then remains in that condition until the **RESET** instruction is received.
- The **LATCH** instruction is often called a **SET** or **OTL** (output latch).
- The **UNLATCH** instruction is often called a **RES** (reset), **OTU** (output unlatch) or **RST** (reset).

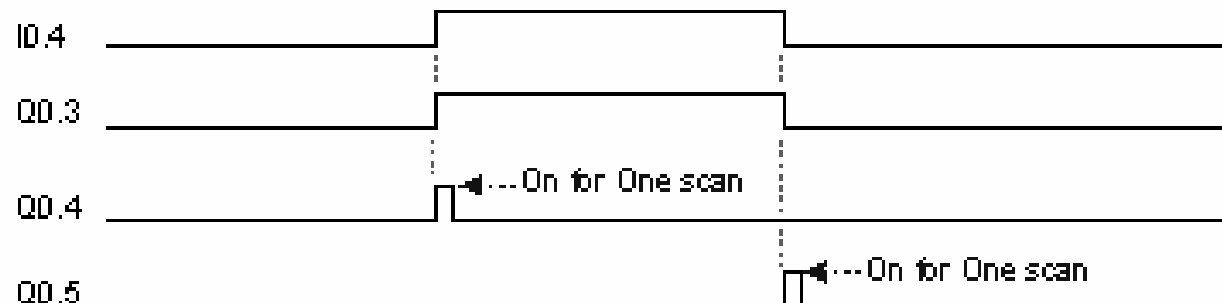


```

LD      IO.4
LPS
EU
S       Q0.3, 1
=       Q0.4
LPP
ED
R       Q0.3, 1
=       Q0.5

```

Network 3



```

NETWORK 3      // A positive Edge Up input on a P contact or a negative Edge Down input
// on a N contact outputs a pulse with a 1 scan cycle duration. In Run mode,
// the pulsed state changes of Q0.4 and Q0.5 are too fast to be visible in
// program status view. The Set (S) and Reset (R) outputs latch the pulse in
// Q0.3 and make the state change visible in program status view.

```


Timers

- Timer is an instruction that waits a set amount of time before doing something (control time).
- Timers count fractions of seconds or seconds using the internal CPU clock.
- The time duration for which a timer has been set is termed the preset and is set in multiples of the time base used.

Timer Operation

- When the instructions before the timer symbol are true the timer starts "ticking".
- When the time elapses the timer will automatically close its contacts.
- When the program is running on the plc the program typically displays the current value.
- Typically timers can tick from 0 to **9999** (16-bit BCD) or 0 to **65535** times (16-bit binary).

Timers Types

- On-Delay timer- simply "delays turning on". It is called *TON*, *TIM* or *TMR*.
- *Off-Delay timer*- simply "delays turning off". It is called *TOF* and is less common than the on-delay type.
 - The on/off delay timers above would be reset if the input sensor wasn't on/off for the complete timer duration.
- *Retentive or Accumulating timer*- holds or retains the current elapsed time when the sensor turns off in mid-stream. It is called *RTO* or *TMRA*.
 - This type of timer needs 2 inputs.

Timers

We need to know 2 things when using timers:

1. What will enable the timer?

- Typically this is one of the inputs (a sensor connected to one input).

2. How long we want to delay before we react?

- Wait x seconds before we turn on a load.

Timers

- Most manufacturers consider timers to behave like **relays** with coils which when energized result in the closing or opening of contacts after some preset time.
- The timer is thus treated as an output for a rung with control being exercised over pairs of contacts elsewhere.
- Others treat a timer as a **delay block** which when inserted in a rung delays signals in that rung reaching the output.

Timer Accuracy

There are software and Hardware Errors when using a timer.

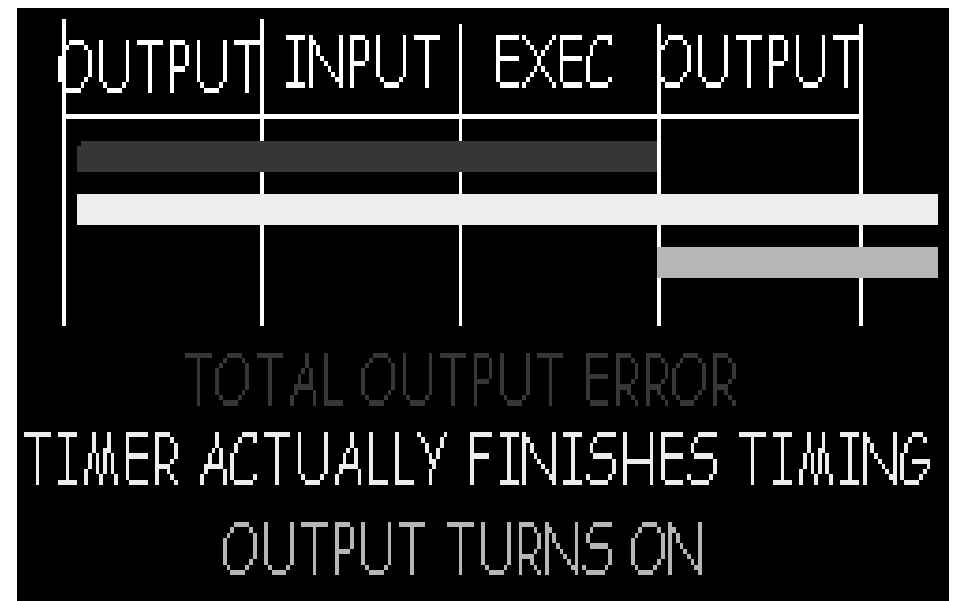
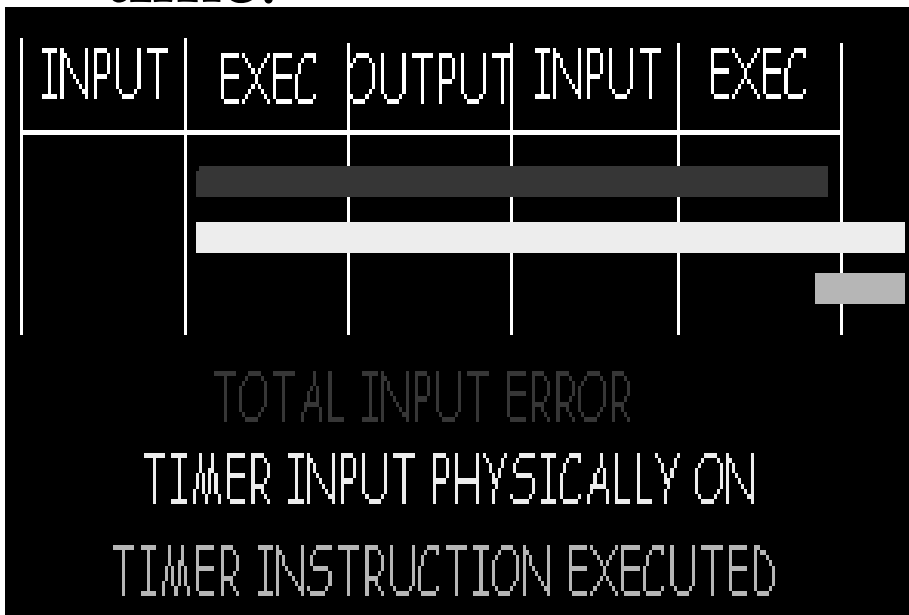
Two types of Software Errors

- **Input error** depending upon when the timer input turns on during the scan cycle.
- **Output error** depending upon when in the ladder the timer actually "times out" and when the plc finishes executing the program to get to the part of the scan when it updates the outputs.
- **Total software error** is the sum of both the input and output errors.

The total worst possible timer error would be equal to:
 1 scan time + 1 program execution time + 1 scan time =
 2 scan times + 1 program execution time.

- The worst possible input error would be
 1 complete scan time +
 1 program execution
 time.

- The worst possible output error would be
 1 complete scan time.



Hardware Error

- There is a hardware input error as well as a hardware output error.
- The **hardware input error** is caused by the time it takes for the plc to actually realize that the input is on when it scans its inputs. Typically this duration is about **10 ms** (to eliminate noise or "bouncing" inputs).

- The **hardware output error** is caused by the time it takes from when the plc tells its output to physically turn on until the moment it actually does. Typically a transistor takes about **0.5 ms** whereas a mechanical relay takes about **10 ms**.
- If the error becomes too big for the application, consider using an external **"hardware" timer**.

Counters

- A counter is set to some preset value and, when this value of input pulses has been received, it will operate its contacts.
- **The counter accumulated value ONLY changes at the off to on transition of the pulse input.**
- Typically counters can count from 0 to 9999, 0 to 65535.

Counter Types

- **Up-counters** counts from zero up to the preset value. These are called CTU, CNT, C, or CTR.
- **Down-counters** count down from the preset value to zero. These are called CTD.
 - For **CTU or CTD counter** we need 2 inputs.
- **Up-down counters** count up and/or down. These are called CTUD.
 - For **up-down counter** we need to assign 3 inputs.

Counter Instruction

To use counters we must know **3 things**:

- **Where** the pulses that we want to count are coming from. Typically this is from one of the inputs.
- **How** many pulses we want to count before we react.
- **When/how** we will reset the counter so it can count again

Counter Formats

- Some manufacturers consider the counter as a **relay** and consist of two basic elements:
 - One relay coil to count input pulses and one to reset the counter, and
 - the associated contacts of the counter being used in other rungs.
- Others treat the counter as an **intermediate block** in a rung from which signals emanate when the count is attained.

High Speed Counter

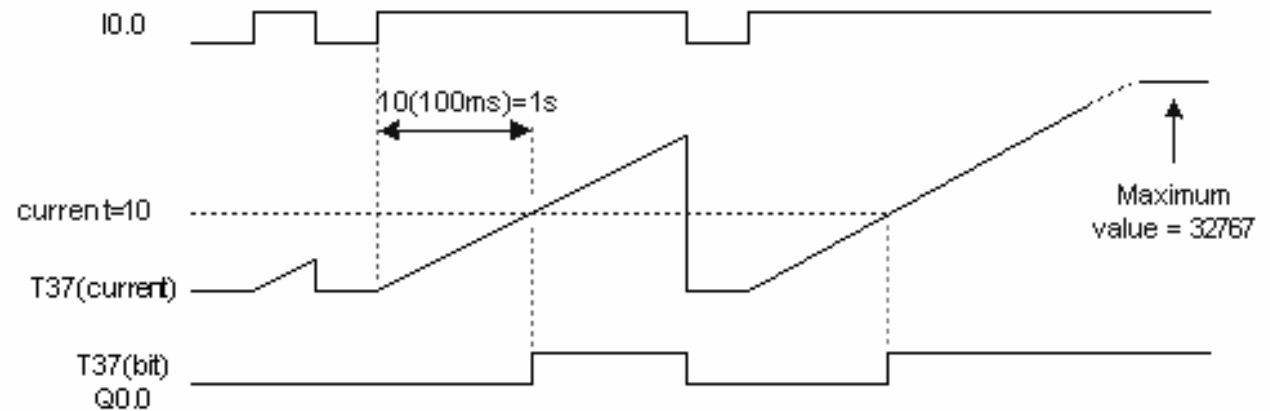
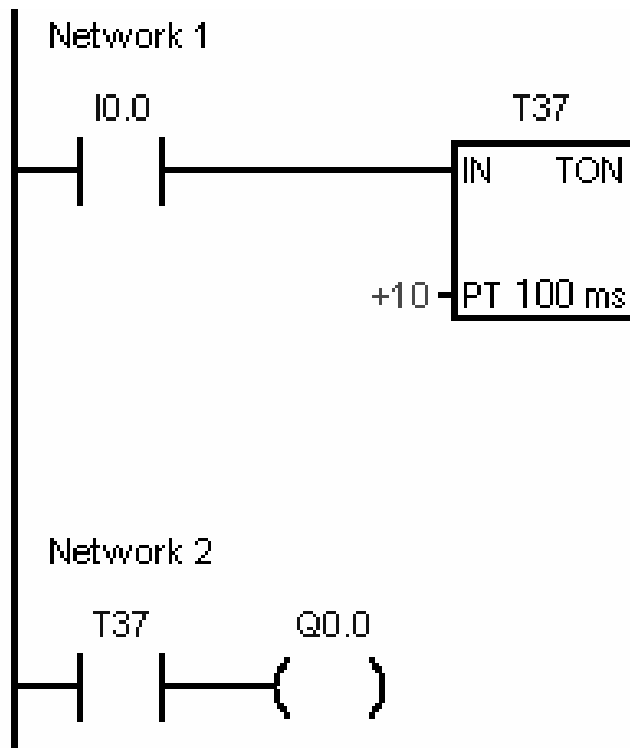
- Most manufacturers also include a limited number of **high-speed counters (HSC)**. .
- Typically a high-speed counter is a "hardware" device.
- Hardware counters **not dependent on scan time.**

Counter Accuracy

- The normal counters are typically "software" counters - they don't physically exist in the plc but rather they are simulated in software.
- A good rule of thumb is simply to always use the normal (software) counters unless the pulses you are counting will arrive faster than 2X the scan time.

Sequencers

- The sequencer is a form of counter that is used for sequential control.
- The PLC sequencer consists of a master counter that has a range of presets counts corresponding to the different steps and so, as it progresses through the count, when each preset count is reached can be used to control outputs.
- The outputs are internal relays, these in turn being used to control the external output devices.

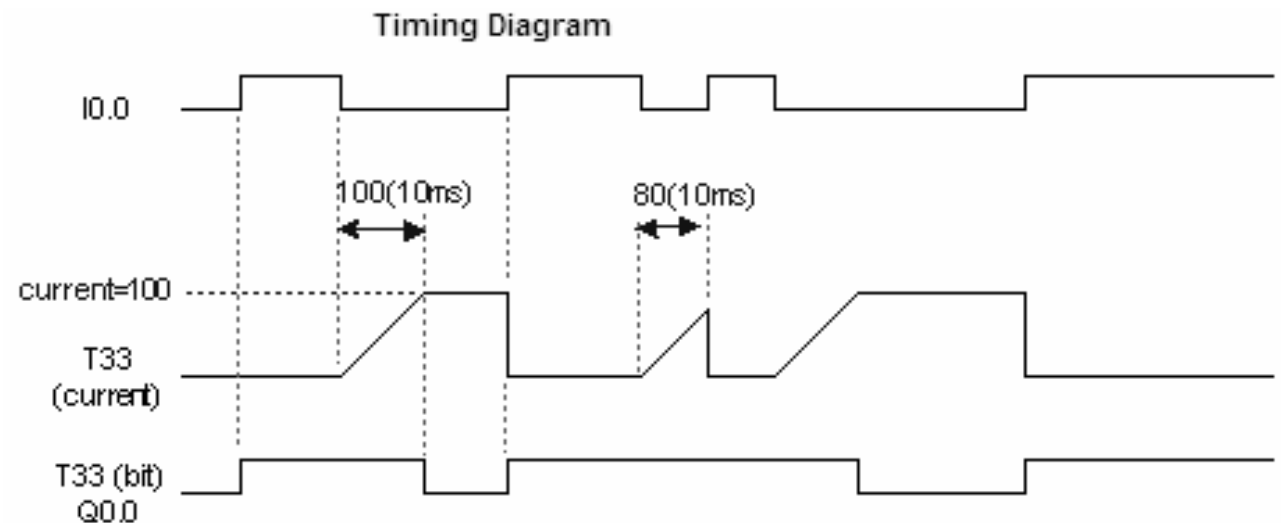
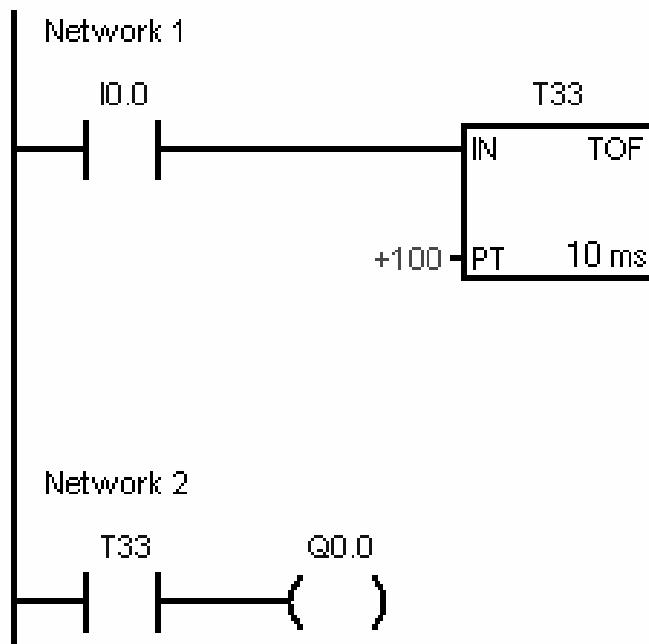


NETWORK 1 // 100ms timer T37 times out after (10)100 ms or 1s
 // I0.0 ON=T37 enabled, I0.0 OFF=disable and reset T37

```
LD I0.0
TON T37 +10
```

NETWORK 2 // T37 bit is controlled by timer T37

```
LD T37
= Q0.0
```

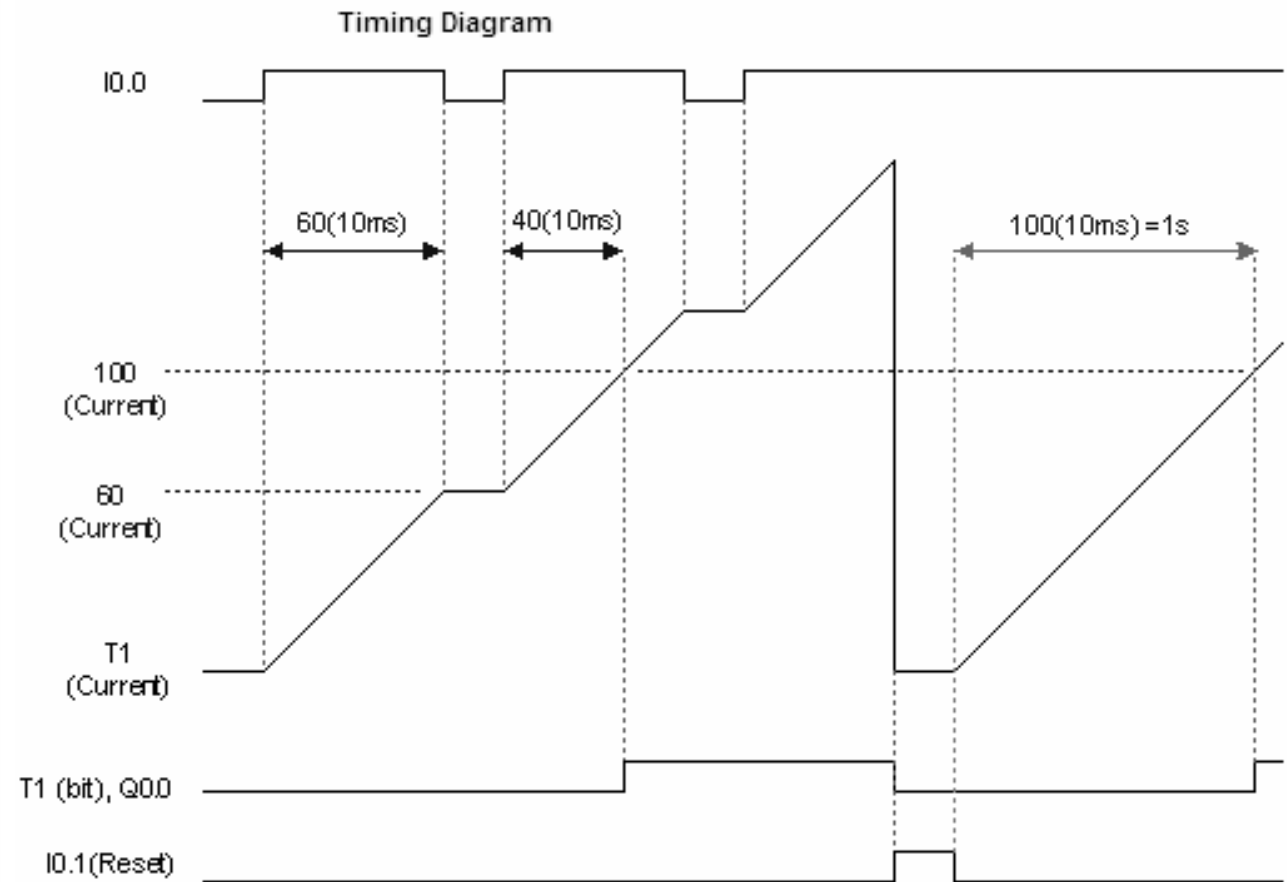
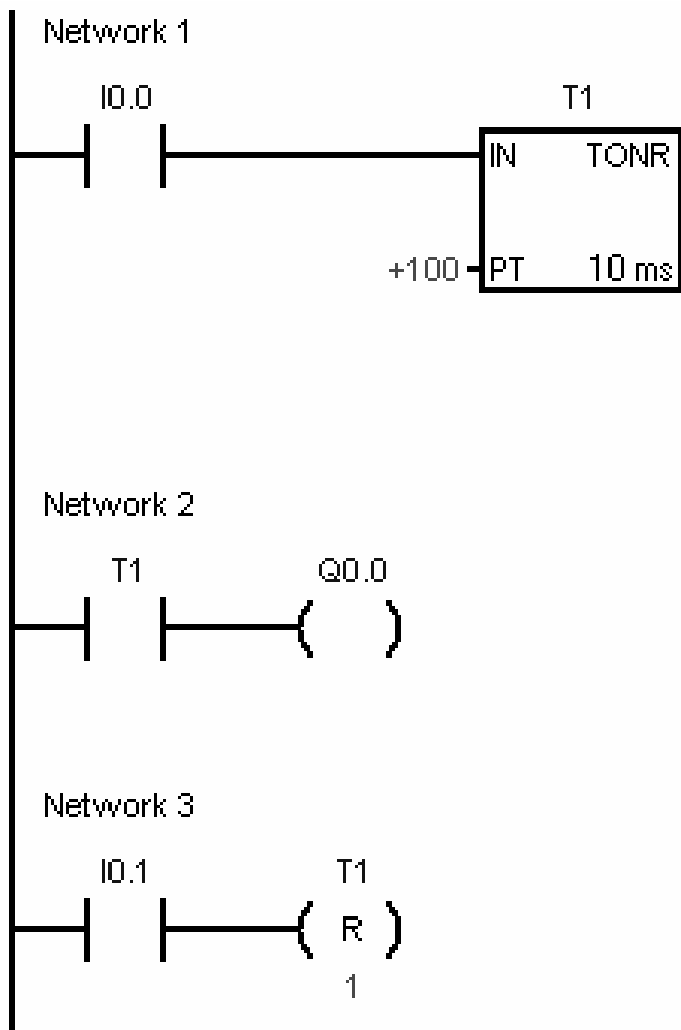


```

NETWORK 1 // 10ms timer T33 times out after (100 x 10 ms) or 1s
// I0.0 ON-to-OFF=T33 enabled, I0.0 OFF-to-ON=disable and reset T33
LD I0.0
TOF T33 +100

NETWORK 2 //Timer T33 controls Q0.0 through timer contact T33
//
LD T33
= Q0.0

```



```

NETWORK 1 // 10 ms TONR timer T1 times out at PT=(100 x 10 ms) or 1 second
//
LD IO.0
TONR T1 +100

```

```

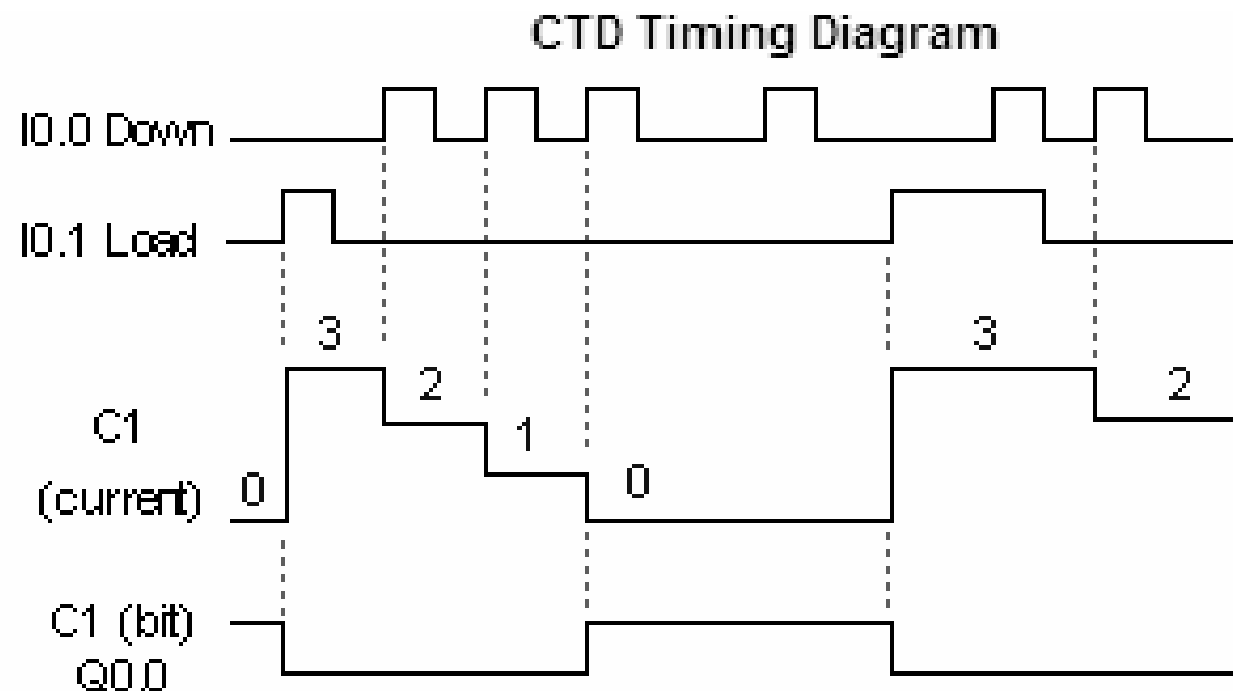
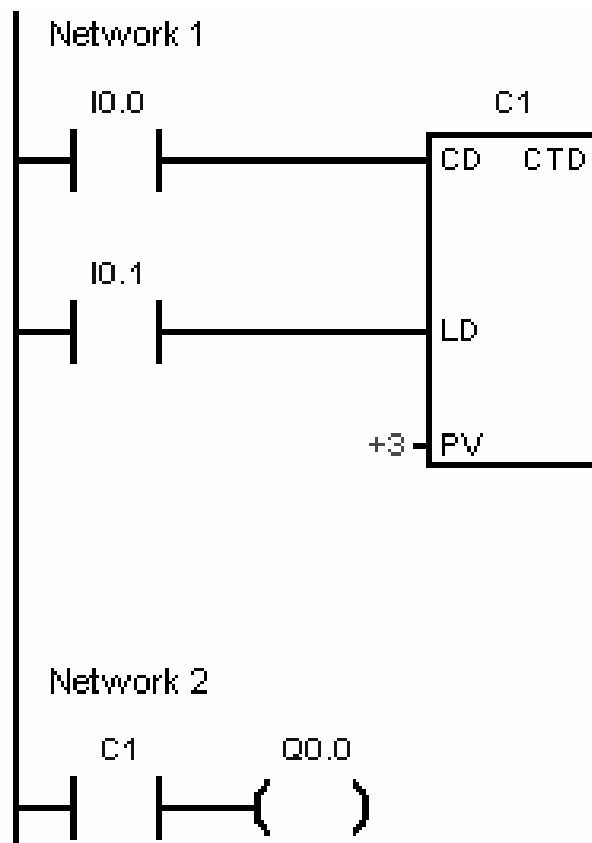
NETWORK 2 // T1 bit is controlled by timer T1
// Turns Q0.0 on after the timer accumulates a total of 1 second
LD T1
= Q0.0

```

```

NETWORK 3 // TONR timers must be reset by a reset instruction with a T address
// Resets timer T1 (current and bit) when IO.1 is on
LD IO.1
R T1 1

```



```

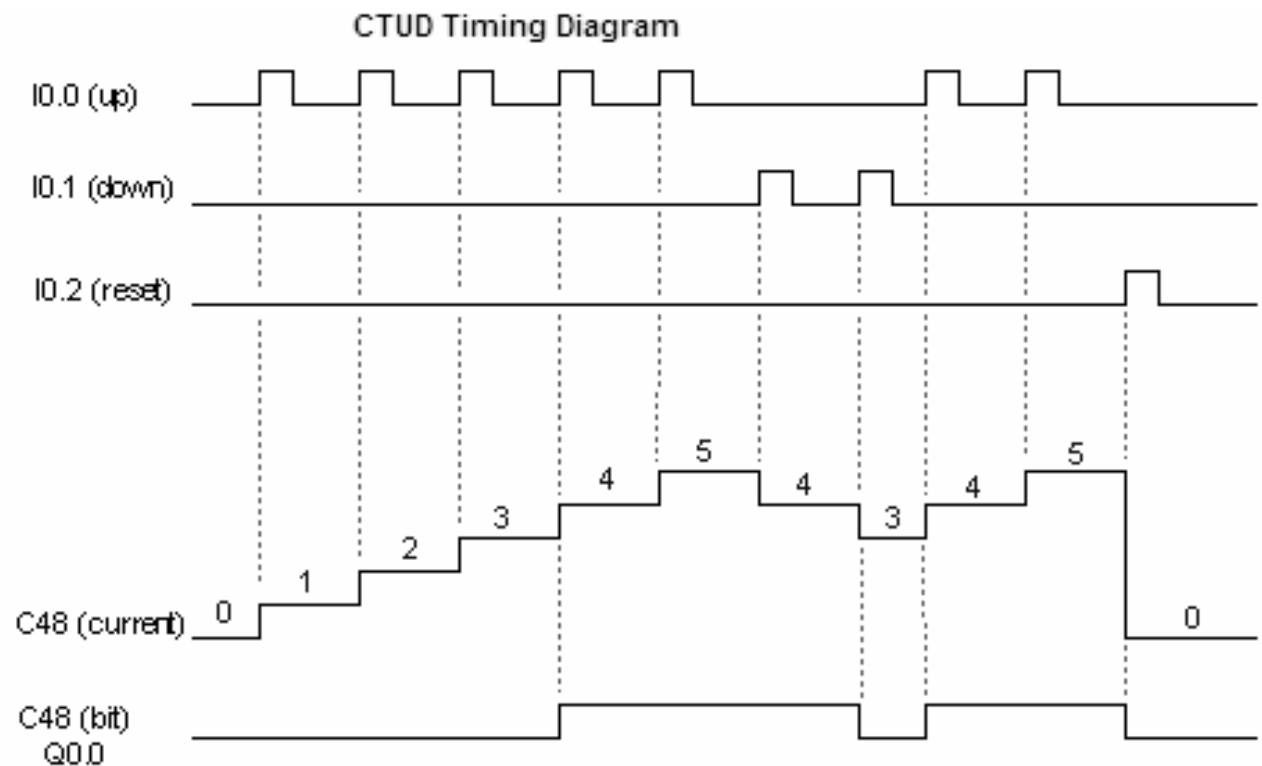
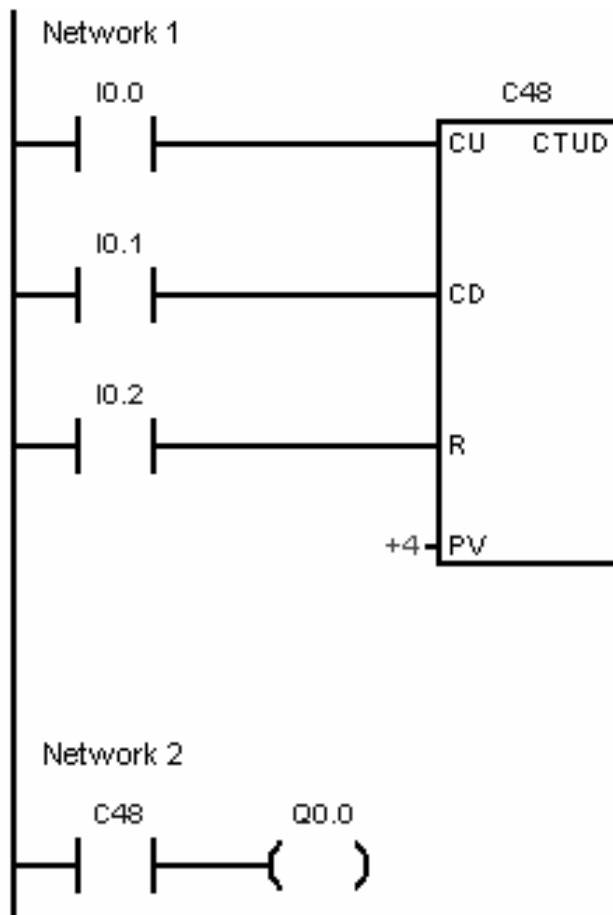
NETWORK 1 // Count down counter C1 current value counts from 3 to 0
// with IO.1 OFF, IO.0 OFF-ON decrements C1 current value
// IO.1 ON loads countdown preset value 3
LD IO.0
LD IO.1
CTD C1 +3

```

```

NETWORK 2 // C1 bit is ON when counter C1 current value = 0
LD C1
= Q0.0

```



```

NETWORK 1 // IO.0 counts up - IO.1 counts down - IO.2 resets current value to 0
LD I0.0
LD I0.1
LD I0.2
CTUD C48 +4

```

```

NETWORK 2 // Count Up/Down counter C48 turns on C48 bit when current value >= 4
LD C48
= Q0.0

```

■ Next Class

ADVANCED (SPECIAL) PROGRAMMING INSTRUCTIONS

ADVANCED PROGRAMMING INSTRUCTIONS

Dr. Mazouz Salahat

Objectives

- Understand and implement special programming functions.

Topics

- Special Instructions:
 - Data Handling Instructions.
 - Program Flow Control Instructions.
 - Shift Registers.

PLC Advanced Instructions

■ Data Handling Instructions

- Data movement instructions
- Data comparison
- Arithmetic (mathematical) Instructions
- Continuous control (PID Instruction)

■ Program Flow Control Instructions

- Master Control/ Master Control Reset (SCR/SCRE)
- Jump Instructions

■ Shift Registers

Data Handling Instructions

- Data handling consists of operations involving **moving** or **transferring** numeric information stored in one memory word location to another word in a different location, **comparing** data values and carrying out simple **arithmetic operations**.

For data movement the typical instruction will contain:

- the data-handling **instruction**,
- the **source** (S) address from where the data is to be obtained and
- the **destination** (D) address to where it to be moved.

Data movement instructions

- Data transfer might be to move
 - a preset value to a timer or counter,
 - a time or counter value to some register for storage,
 - data from an input to a register
 - a register to output.

Data movement instructions

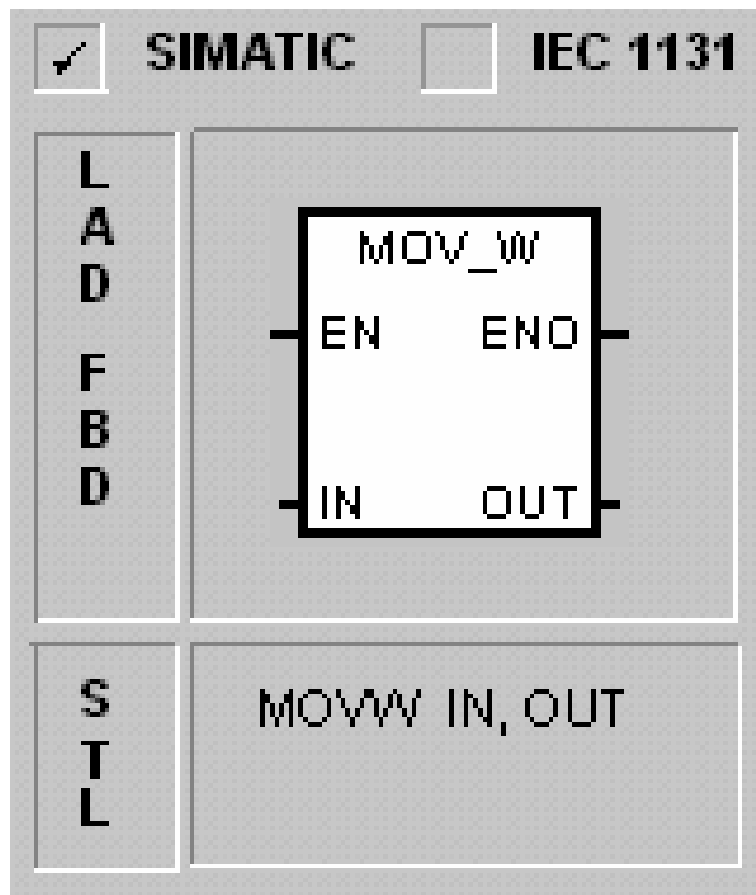
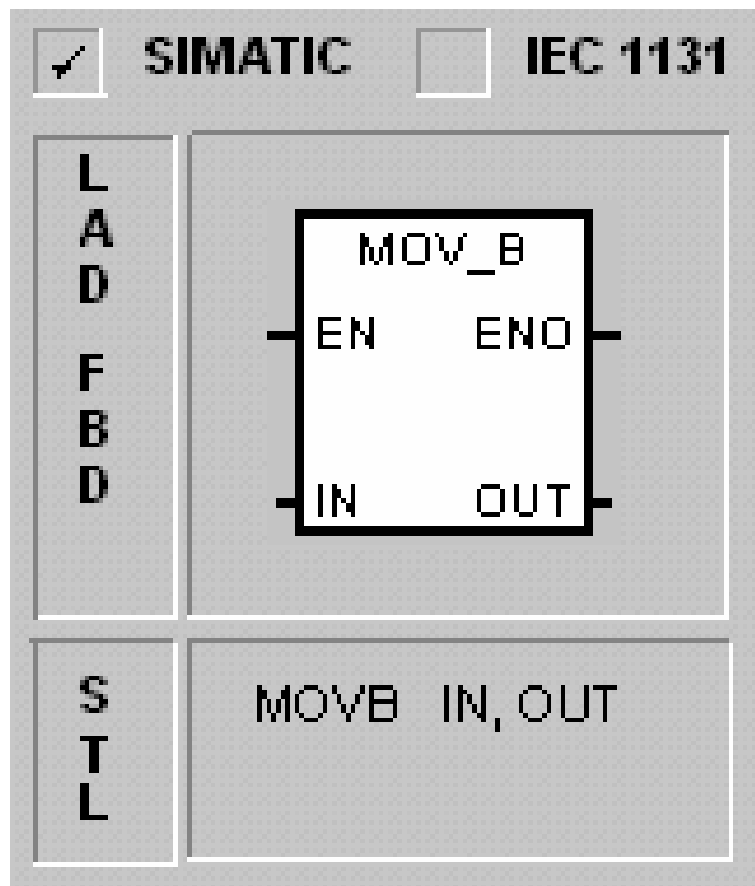
- There are typically **2 common instruction "sets"**:
 - The single instruction is commonly called **MOV** (move) copies a value from one address to another.
 - The paired instruction typically is called **LDA** (LoaD Accumulator) and **STA** (STore Accumulator).
 - **LDA** is where the data we want to move is located.
 - **STA** is similar to the destination of a MOV instruction.

MOVE Instruction

The MOVE instruction needs to know 2 things:

- **Source** - where the data we want to move is located.
 - We could write a constant here or a location or address.
- **Destination** - the location where the data will be moved to.
 - We write an address here. Also, the data can be moved to the physical outputs.

- The Move and Assign Values instruction moves the value IN to the address OUT. The input parameter is not modified during execution.



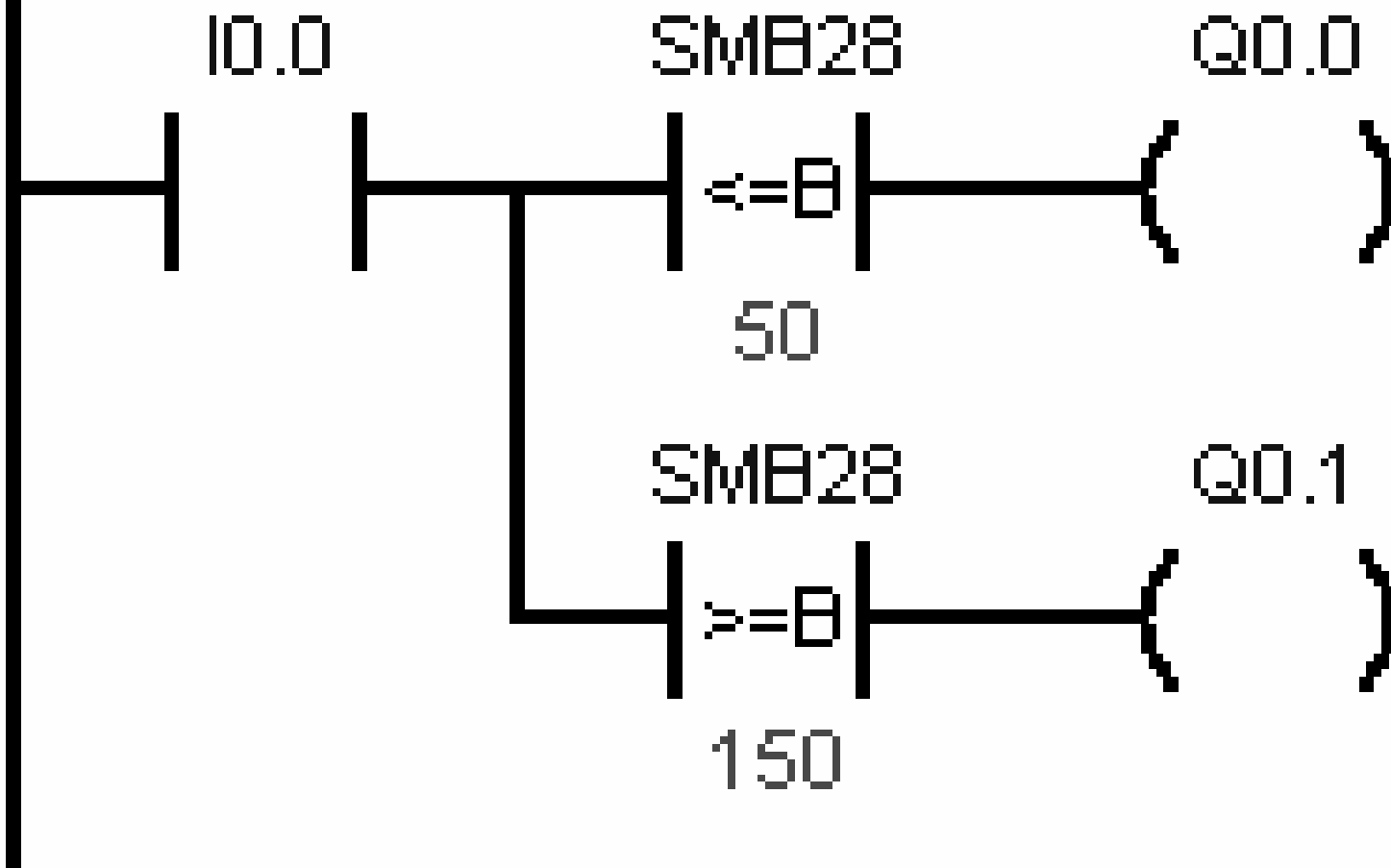
Data comparison

- The data comparison instruction gets the PLC to compare two data values.
 - Thus it might be to compare a digital value read from some input device with a second value contained in a register.
- PLCs generally can make comparisons for:
 - less than ($<$ or LESS),
 - equal to ($=$ or EQU),
 - less than or equal to ($\leq$ or LEQ),
 - greater than ($>$ or GRT),
 - greater than or equal to ($\geq$ or GEQ), and
 - not equal to ($\neq$ NEQ).

Data Comparison Instruction

- Data comparison instruction contains:
 - the instruction to compare data,
 - the source (S) address from where the data is to be obtained for the comparison and
 - the destination (D) address of the data against which it is to be compared.
- When value 1 is compared with value 2 and the result agrees with the form of comparison then the outcome is a true output (1);
- if the comparison is not true then the outcome is false output (0).

Network 1



Typical Math Instructions

- Typical Mathematical Instructions:

ADD, SUB, MUL, DIV, CLR, SQR,

- Many PLCs also include other math capabilities.

Some of these functions could include:

Square roots

Scaling

Absolute value

Sine

Cosine

Tangent

Natural logarithm

Base 10 logarithm

X^Y

Arcsine (tan, cos)

....check with the manufacturer.

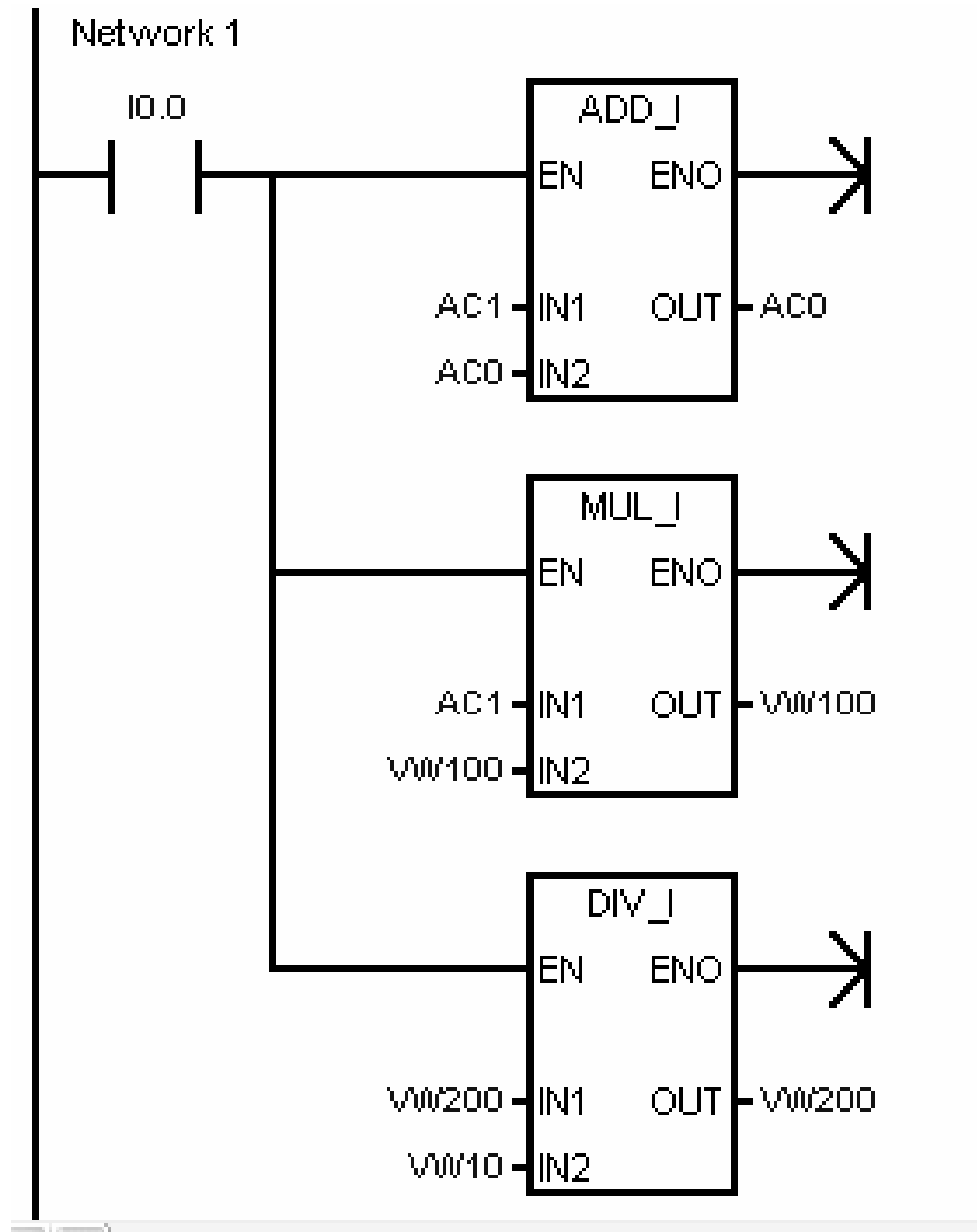
Arithmetic (mathematical) Instructions

- PLCs almost always include math functions to carry out some **arithmetic** operations :
 - **Addition (ADD)** - The capability to add one piece of data to another.
 - **Subtraction (SUB)** - The capability to subtract one piece of data from another.
 - **Multiplication (MUL)** - The capability to multiply one piece of data by another.
 - **Division (DIV)** - The capability to divide one piece of data from another.

Math Single Instruction

There are generally two common methods used by the majority of plc makers.

- **A single instruction** that asks for a few key pieces of information. This method typically requires:
 - **Source A**
 - **Source B**
 - **Destination**



Math Dual Instruction

- The **dual instruction** method.
 - In this method, we give the symbol only the Source B location.
 - The Source A location is given by the **LDA** instruction.
 - The Destination would be included in the **STA** instruction.

Overflow

- Typically the memory locations are 16-bit locations.
- If a result is **greater than** the value that could be stored in a memory location then we get an **overflow**. The plc turns on an internal relay that tells us an overflow has happened.
- We get an overflow if the number is greater than 65535 ($2^{16}=65536$).

Depending on the plc, we would have different data in the destination location. Some use 32-bit math which solves the problem.

- If we're doing division, and we **divide by zero** the overflow bit turns on.

Floating Point

- Floating point math is simply using decimal points.
 - We could say that 10 divided by 3 is 3.333333 (floating point), or we could say that 10 divided by 3 is 3 with a remainder of 1 (long division).
- Many micro/mini PLCs don't include floating point math, but most of the larger systems typically do.

Continuous control (PID Instruction)

- Continuous control of some variable can be achieved by comparing the actual value of the variable with the desired set value and then giving an output depending on the control law required.
- Many PLCs provide the PID calculation to determine the controller output as a standard routine. All that is then necessary is to pass the desired parameters, i.e. the values of K_p , K_i , and K_D , and input/output locations to the routine via the PLC program.

Program Flow Control Instructions

- Control instructions are used to enable or disable a block of logic program or to move execution of a program from one place to another place.
- The control instructions include:
 - Master Control instruction (SCR/SCRE)
 - Jump to label instruction (JMP)
 - Label instruction (LBL)
 - Jump to Subroutine instruction (JSR)
 - Subroutine instruction (SBR)
 - Return from Subroutine instruction (RET)

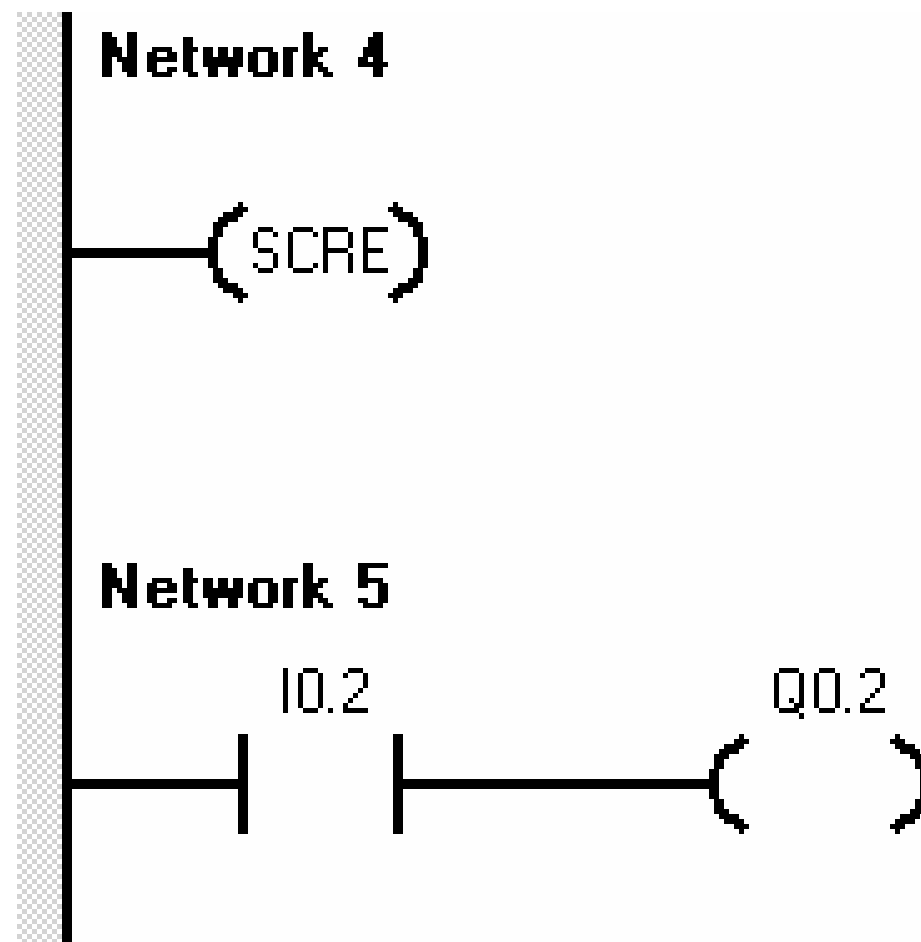
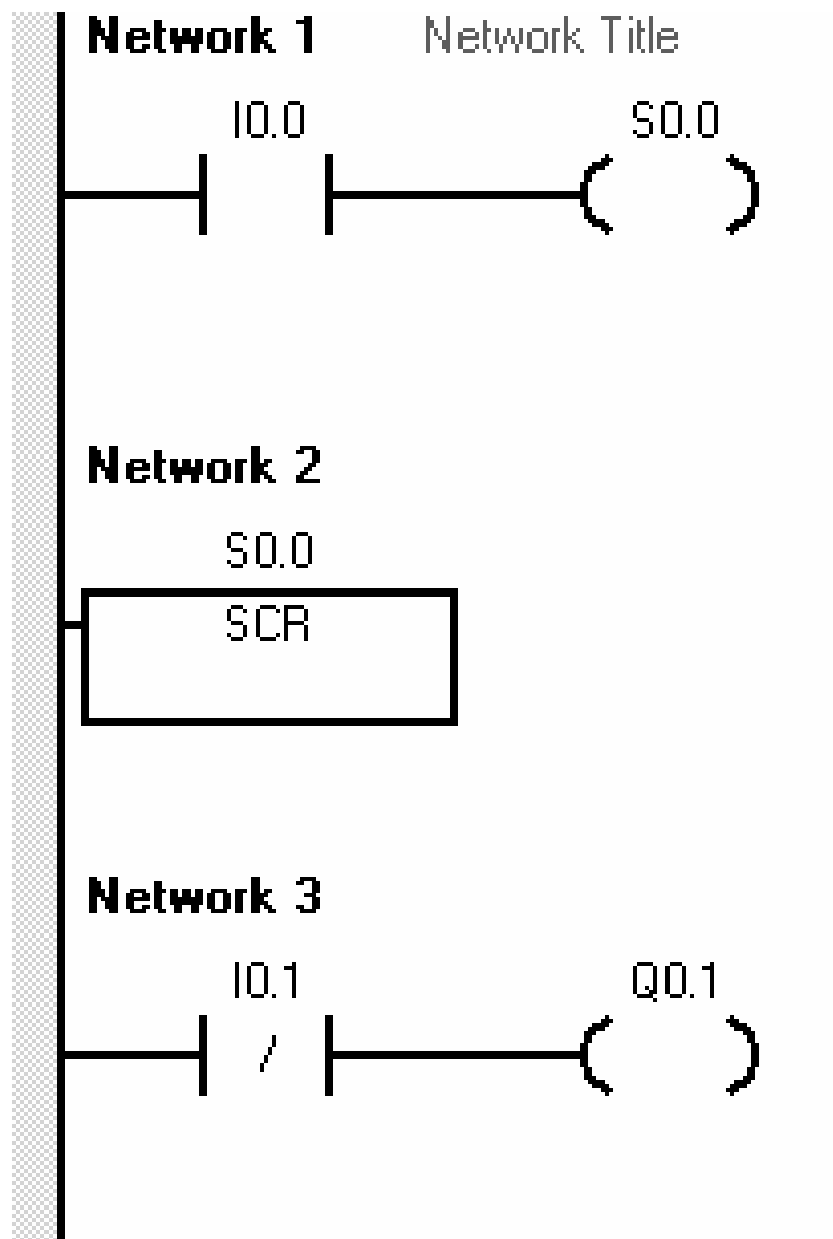
Master Control/ Master Control Reset (MC/MCR), **(SCR/SCRE)**

- When large numbers of outputs have to be controlled, it is sometimes necessary for whole sections of program to be turned on or off when certain criteria are realized. This could be achieved by including a MCR instruction.
- A MC/MCR instruction is an output instruction.
The master control instruction typically is used in pairs with a master control reset.
- Different formats are used by different manufacturers.
 - **MC/MCR** (master control/master control reset),
 - **MCS/MCR** (master control set/master control reset)
or
 - **MCR** (master control reset).

- The zone being controlled begins with a rung that has the first MC instruction, which status depends on its rung condition. This zone ends with a rung that has the second MCR instruction only.
- When the rung with the first MCR instruction is true, the first MCR instruction is high and the outputs of the rung in the controlled zone can be energized or deenergized according to their rung conditions.

When the this rung is false, all the outputs in the zone are deenergized, regardless their rung conditions.

- A program might use a number of master control relays, enabling various sections of program to be switched in or out.
- **Timers** should not be used inside the MC/MCR block because some manufacturers will reset them to zero when the block is false whereas other manufacturers will have them retain the current time state.
- **Counters** typically retain their current counted value.

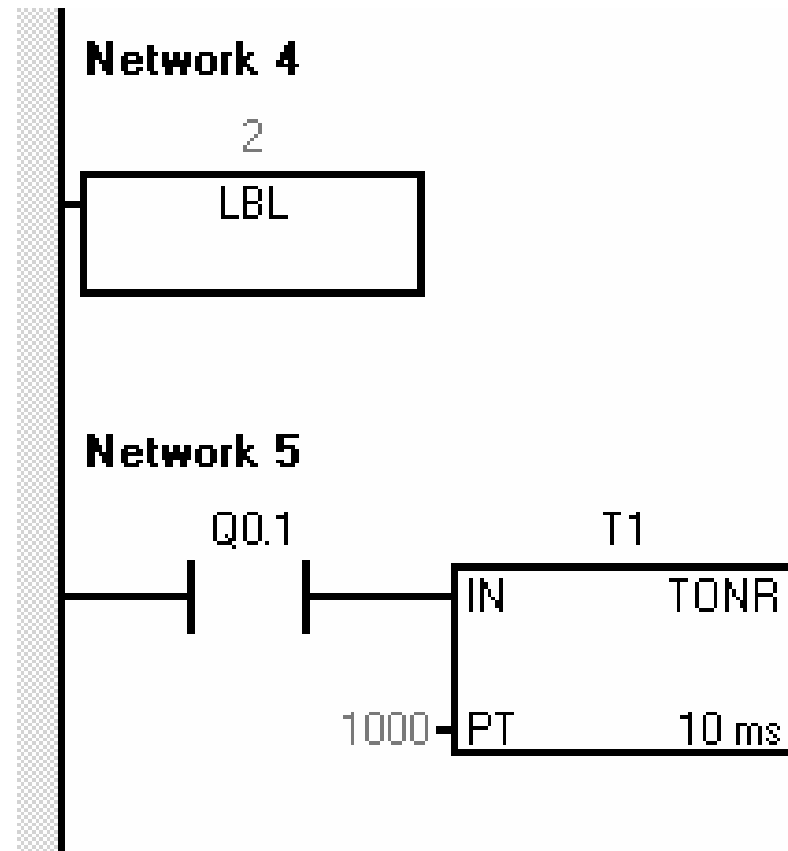
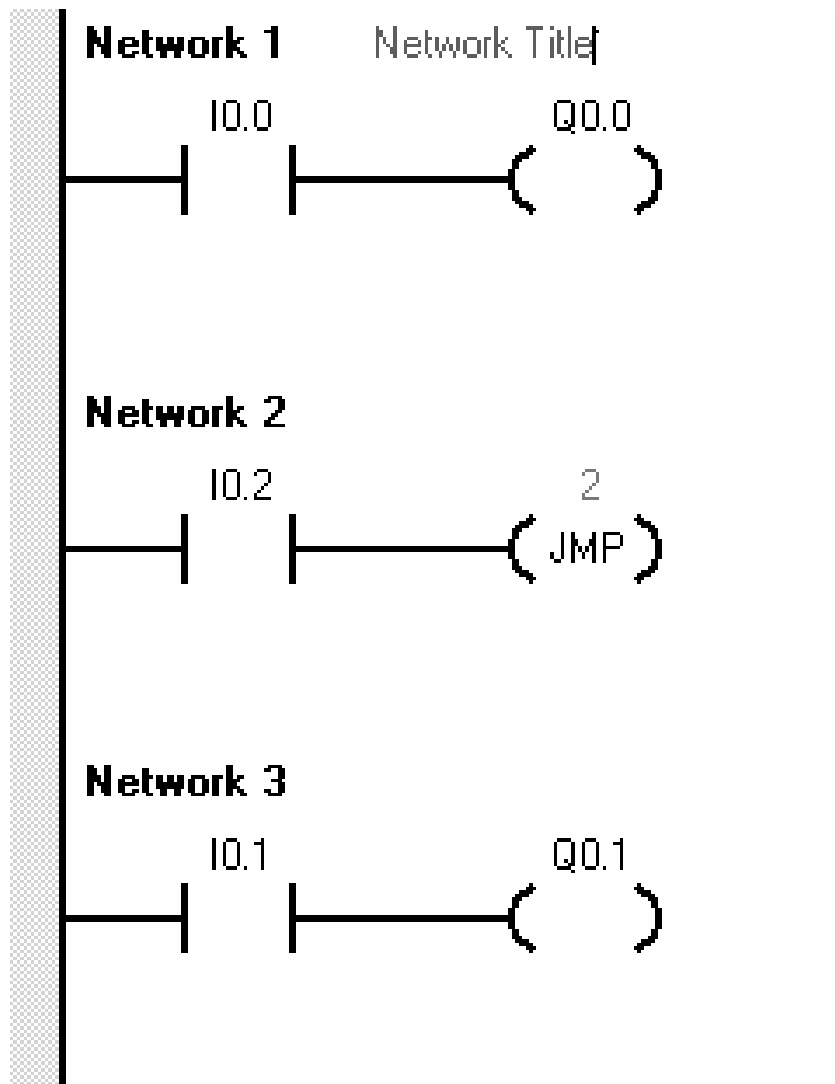


Jump Instructions

- The JUMP instructions allow to break the rung sequence and **move the program execution** from one rung **to another** or **to a subroutine**.
- The Jump is a controlled output instruction.
- You can jump forward or backward.
- You can use multiple jump to the same label.
- Jumps within jumps are possible
- There are:
 - Jump to Label
 - Jump to subroutine

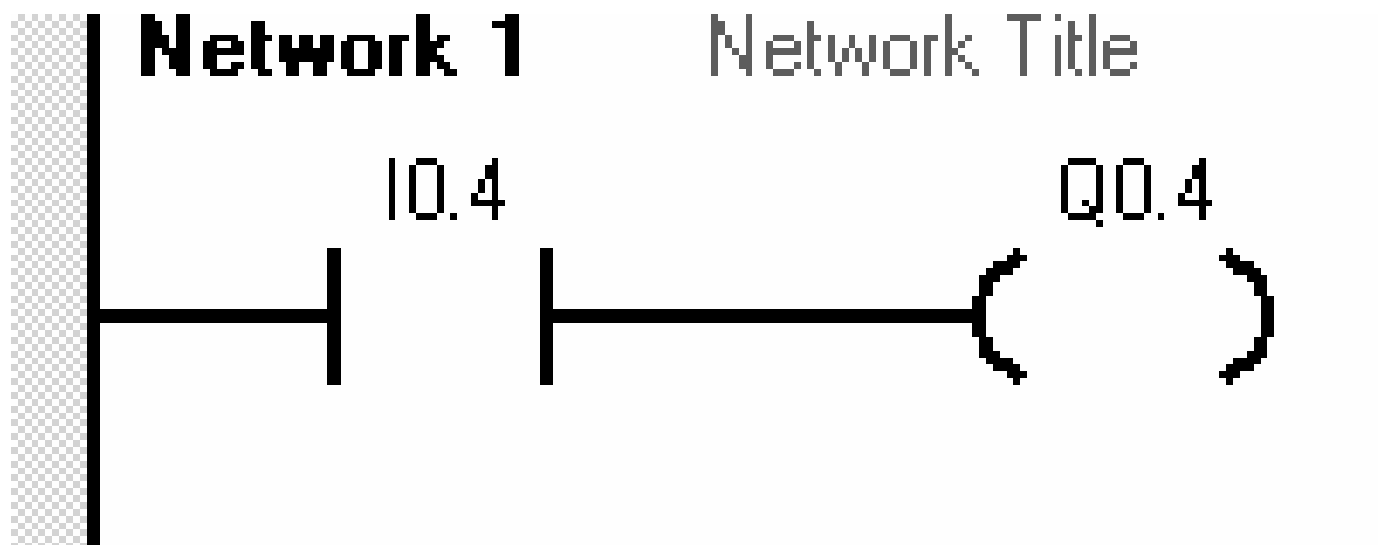
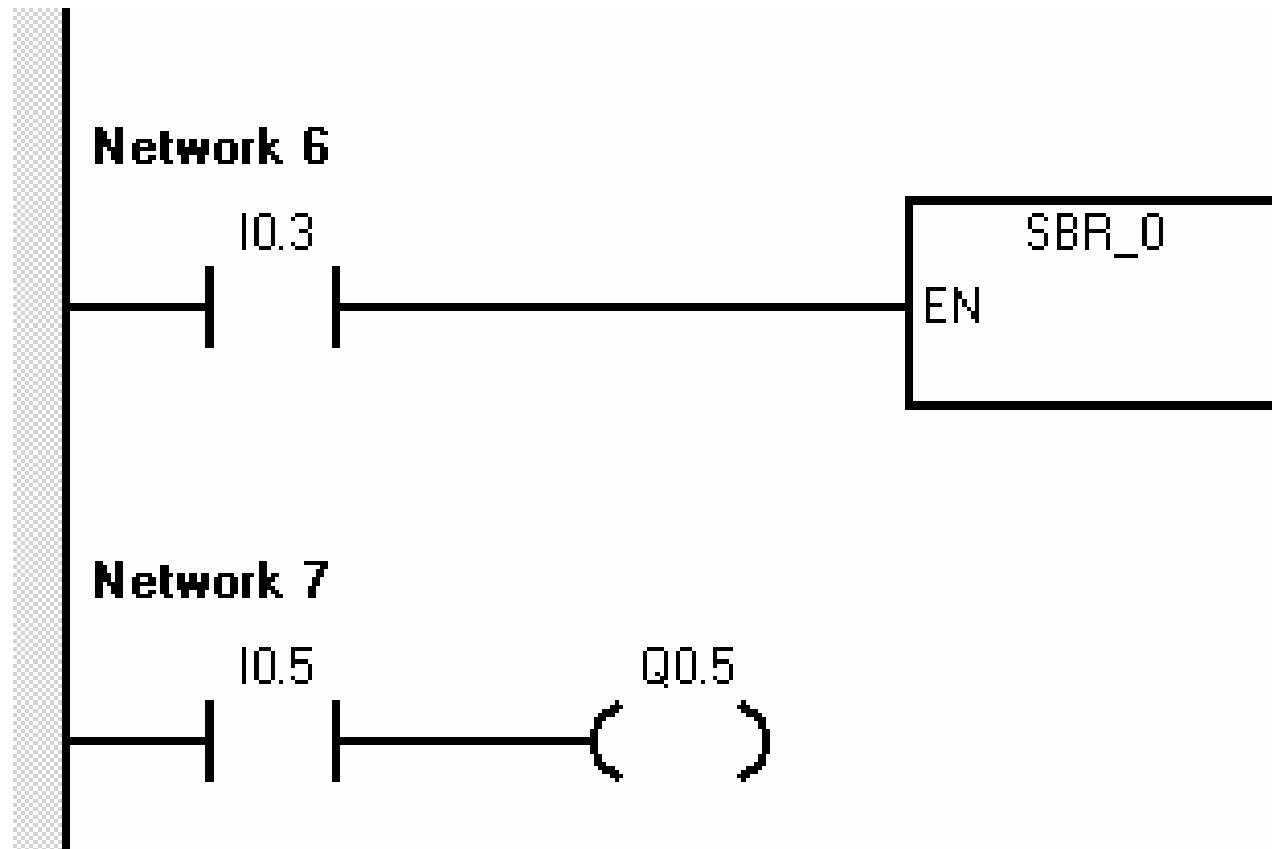
JUMP to LABEL

- When the rung condition of a Jump to Label instruction is true, PLC breaks its execution sequence and moves to the rung with a Label instruction that has the same label number as the Jump to Label instruction does.
 - The Label instruction is always the first instruction in a rung.



JUMP to SUBROUTINE

- When the rung condition of a Jump to subroutine instruction is true, the PLC jumps to the Subroutine instruction at the beginning of the target subroutine file and resumes execution at that point.
- A Jump to Subroutine instruction must be used with a subroutine.
 - In the first rung of a subroutine file, the first instruction must be a Subroutine instruction.



RETURN / END

- A Return from Subroutine instruction marks the end of Subroutine instruction.

When the rung condition of this instruction is true, it causes the PLC to resume execution in the calling program file at the rung following the Jump to Subroutine instruction in the calling program.

- When a Return from Subroutine instruction is not programmed in a subroutine file, the END instruction automatically causes the PLC to move execution back to the rung following the Jump to Subroutine instruction.
- A Jump to Subroutine instruction can be used either in a main application program or a subroutine program to call another subroutine program.

Shift Registers

- The **shift register** is a number of internal relays grouped together (normally 8, 16, or 32) which allow stored bits to be shifted from one relay to another.
- The grouping together of internal relays to form a shift register is done automatically by a PLC when the shift register function is selected. This is done by using the programming code against the internal relay number that is to be the first in the register array.

- Shift registers can be used where a **sequence** of operations is required or to **keep track** of particular items in a production system.
- The shift register is most commonly used in conveyor systems, labeling or bottling applications, etc.

SHIFT OPERATION

- If we receive the shift signal, then the input signal enters the first location in the register and all bits shift along one location.
- Each new change in status gets stored in the first bit and the remaining bits get shifted down the train.
- The last bit overflows and is lost.

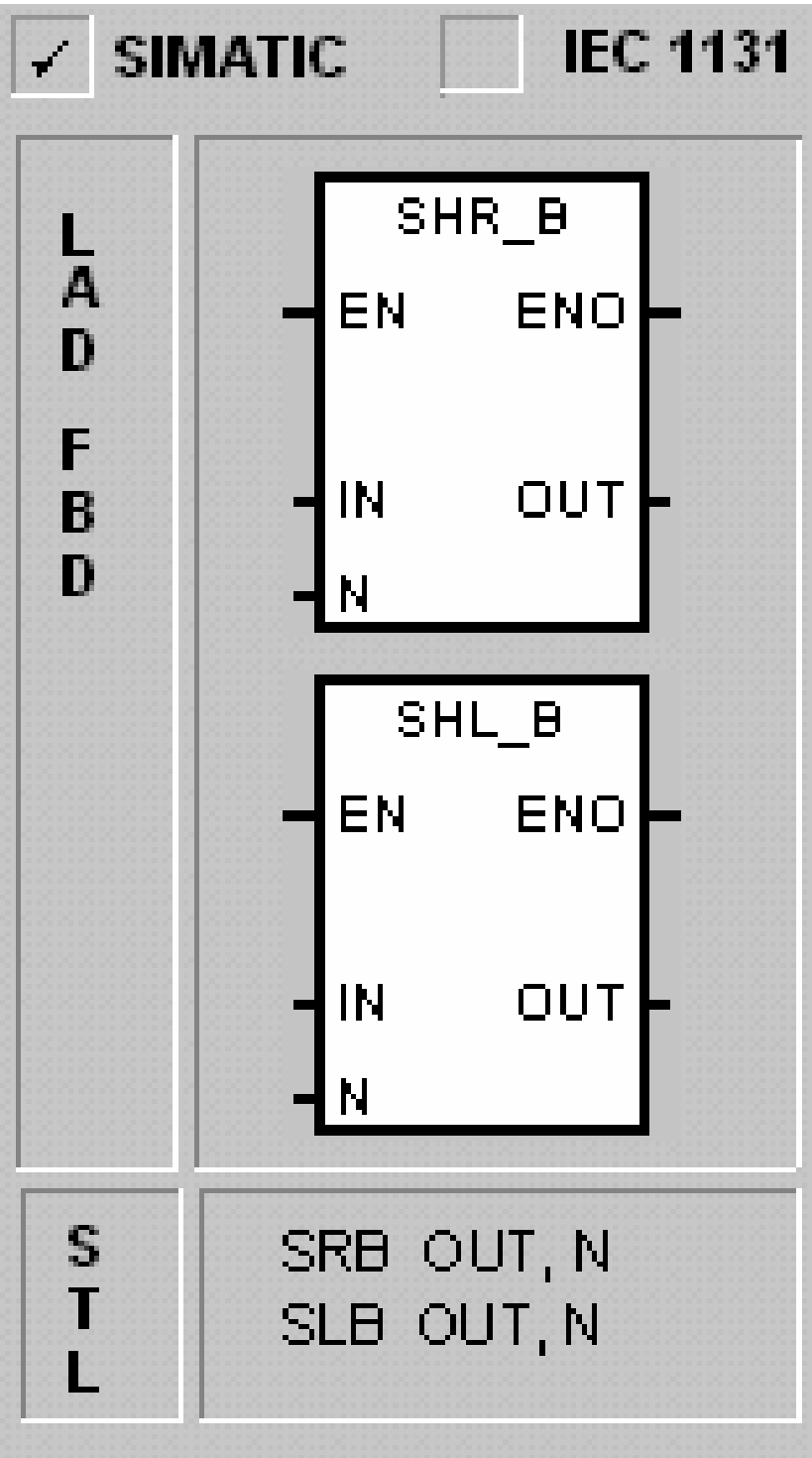
SHIFT INSTRUCTION

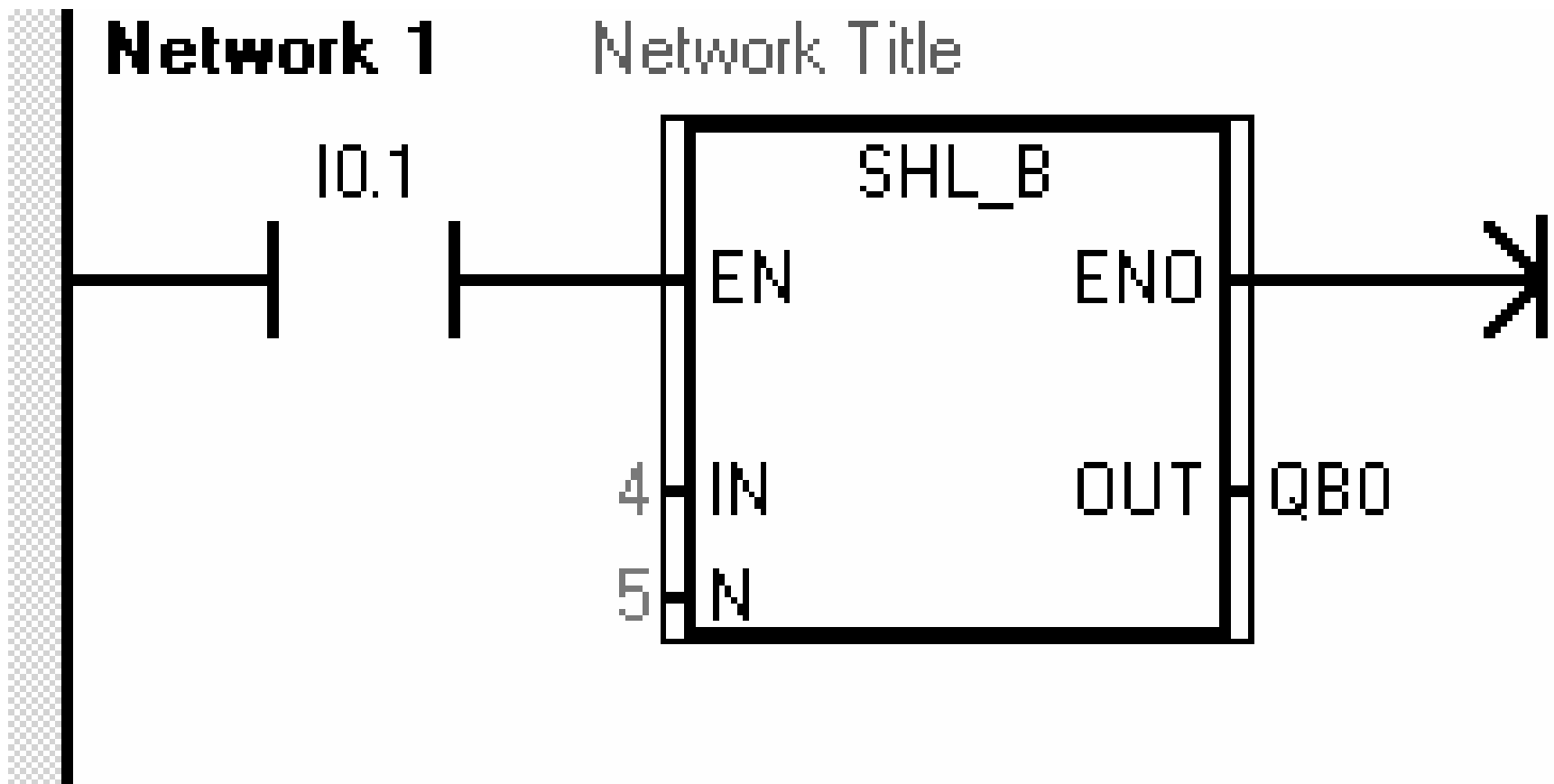
- The shift register goes by many names. SFT (ShiFT), BSL (Bit Shift Left), SFR (Shift Forward Register) are some of the common names. These registers shift the bits to the left.
- BSR (Bit Shift Right) and SFRN (Shift Forward Register Not) are some examples of instructions that shift bits to the right.
- We should note that not all manufacturers have shift registers that shift data to the right but most all do have left shifting registers.

SHIFT REGISTER INPUTS

- Notice that the symbol has some data inside and needs 3 inputs:
 - **Data-** The data input gathers the true/false statuses that will be shifted down the train. When the data input is true the first bit in the register will be a 1. This data is only entered into the register on the rising edge of the clock input.
 - **Clock-** On the rising edge of the clock input, the shift register shifts the data one location over inside the register and enters the status of the data input into the first bit. On each rising edge of this input the process will repeat.
 - **Reset-** The reset input clears all the bits inside the register we're using to 0.

- The Shift Right Byte (SRB) and Shift Left Byte (SLB) instructions shift the input value (IN) right or left by the shift count (N), and load the result in the output byte (OUT).
- If the shift count (N) is greater than or equal to 8, the value is shifted a maximum of 8 times.
- Shift right and shift left byte operations are unsigned.





■ Next Class

PLC SYSTEM ENGINEERING

PLC NETWORKS

Dr. Mazouz Salahat

OBJECTIVES

- Be familiar with different PLC configurations.
- Be familiar with different PLC communication standards and protocols.
- Understand and Implement PLC Networks.

TOPICS

- PLCs Configurations.
- Communication Standards and Protocols.
- PLC Networks.

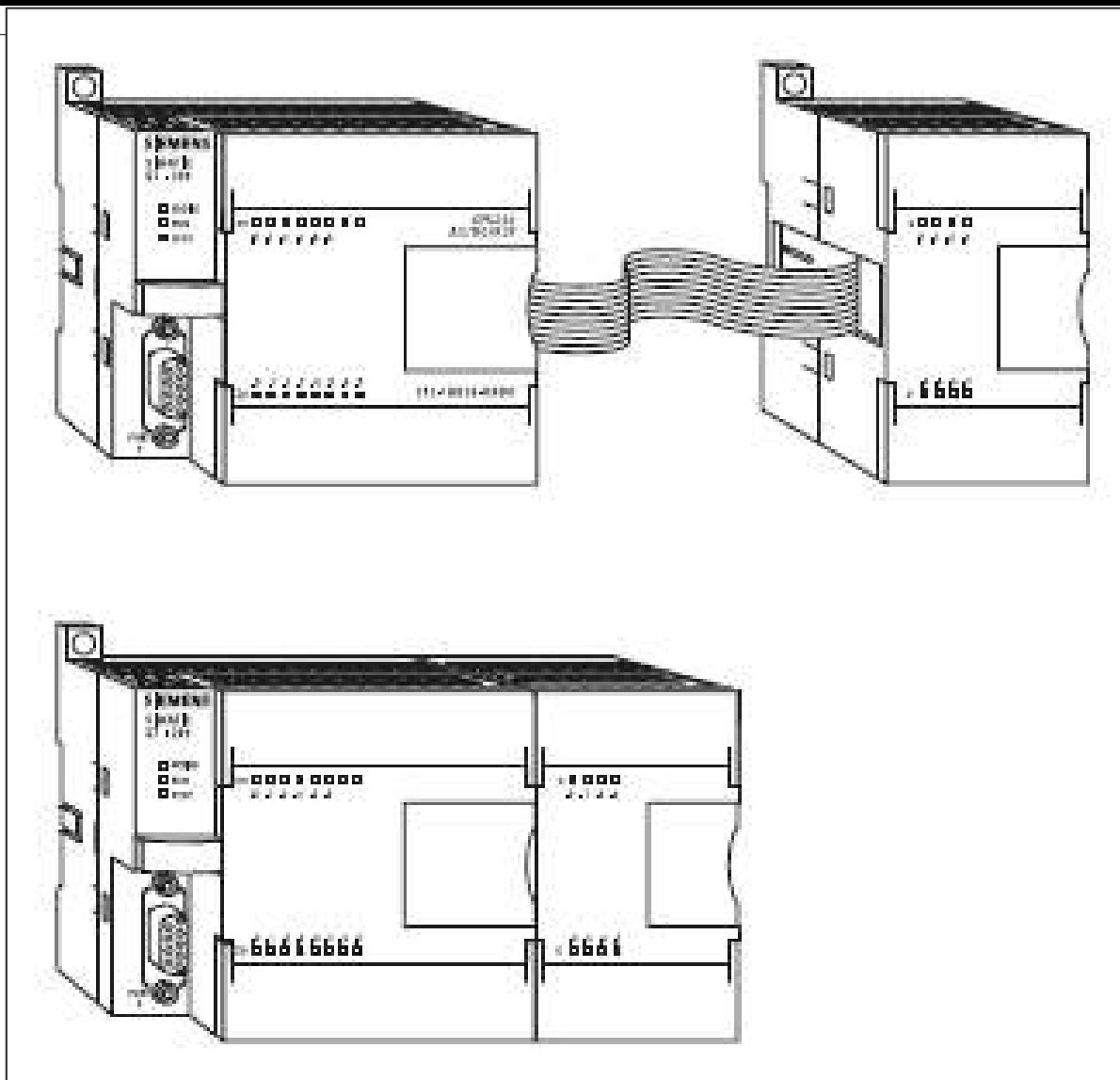
PLC CONFIGURATIONS

- Extension units.
- Remote I/O units.
- Multiple PLCs.

Extension modules

- **I/O number can be increased** through certain additional modules by system extension through extension lines.
- Each module can contain extension both of **input** and **output** lines.
- Extension modules can have inputs and outputs of a different nature from those on the PLC controller.

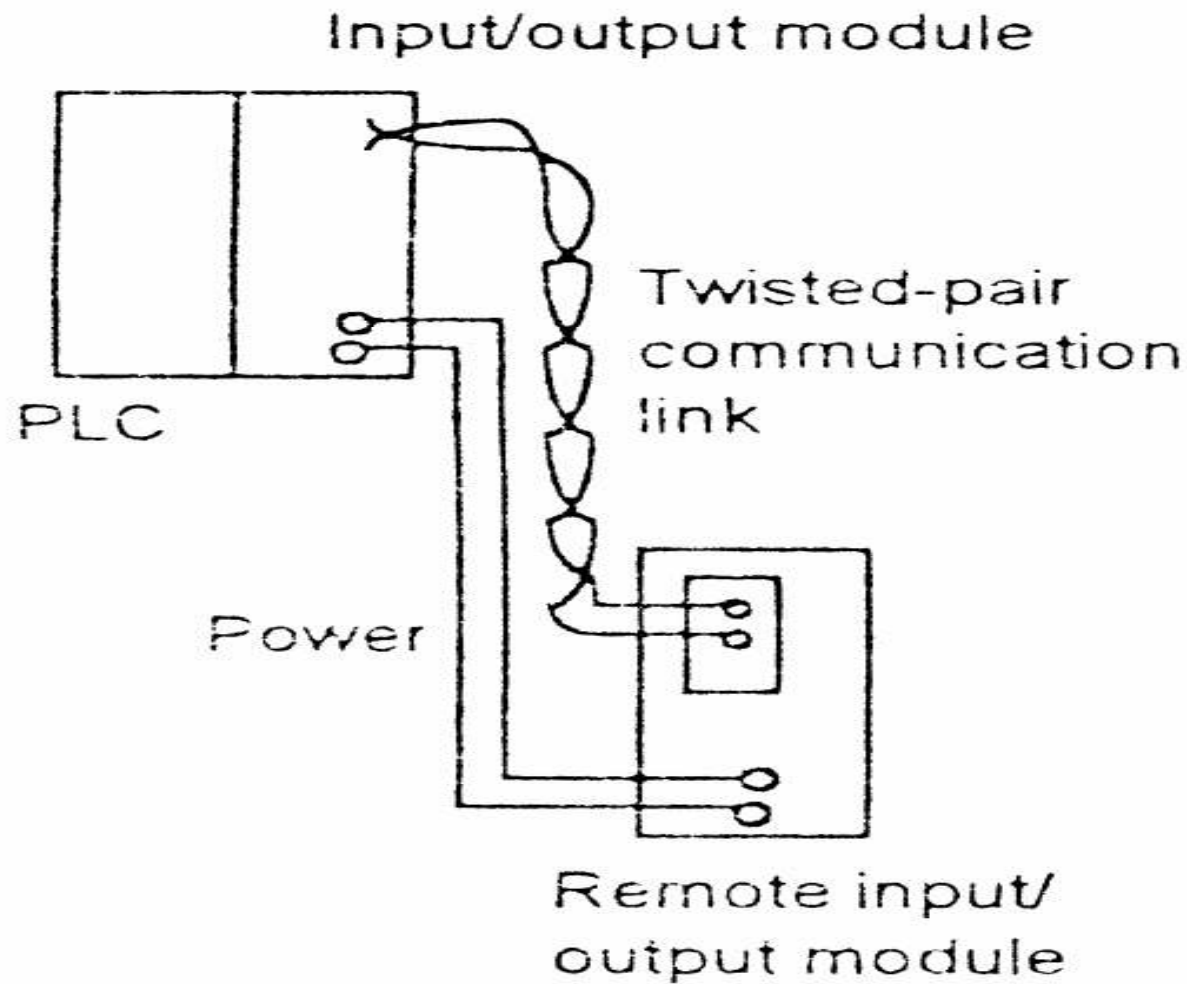
Extension Unit



Remote I/O Modules

- When there are **many I/O located considerable distances away** from the PLC an economic solution is to use remote I/O modules and use cables to connect these, over the long distances, to the PLC.

I/O Remote Modules



Remote PLCs

- In some situations a **number of PLCs may be linked** together with a master PLC unit sending and receiving I/O data from the other units.

PLC Connections

- **Twisted-pair** cabling, often routed through steel conduit.
- **Coaxial** cable enables higher data rates to be transmitted and does not require the shielding of steel conduit.
- **Fiber-optic** cabling has the advantage of high transmission rate, resistance to noise, small size and flexibility.

PLC COMMUNICATIONS

- Parallel
- Serial

Parallel communication

- **Parallel** communication is when all the constituent bits of a word are simultaneously transmitted along parallel cables.
- This allows data to be transmitted over short distances at high speeds.
- Might be used when connecting laboratory instruments to the system.

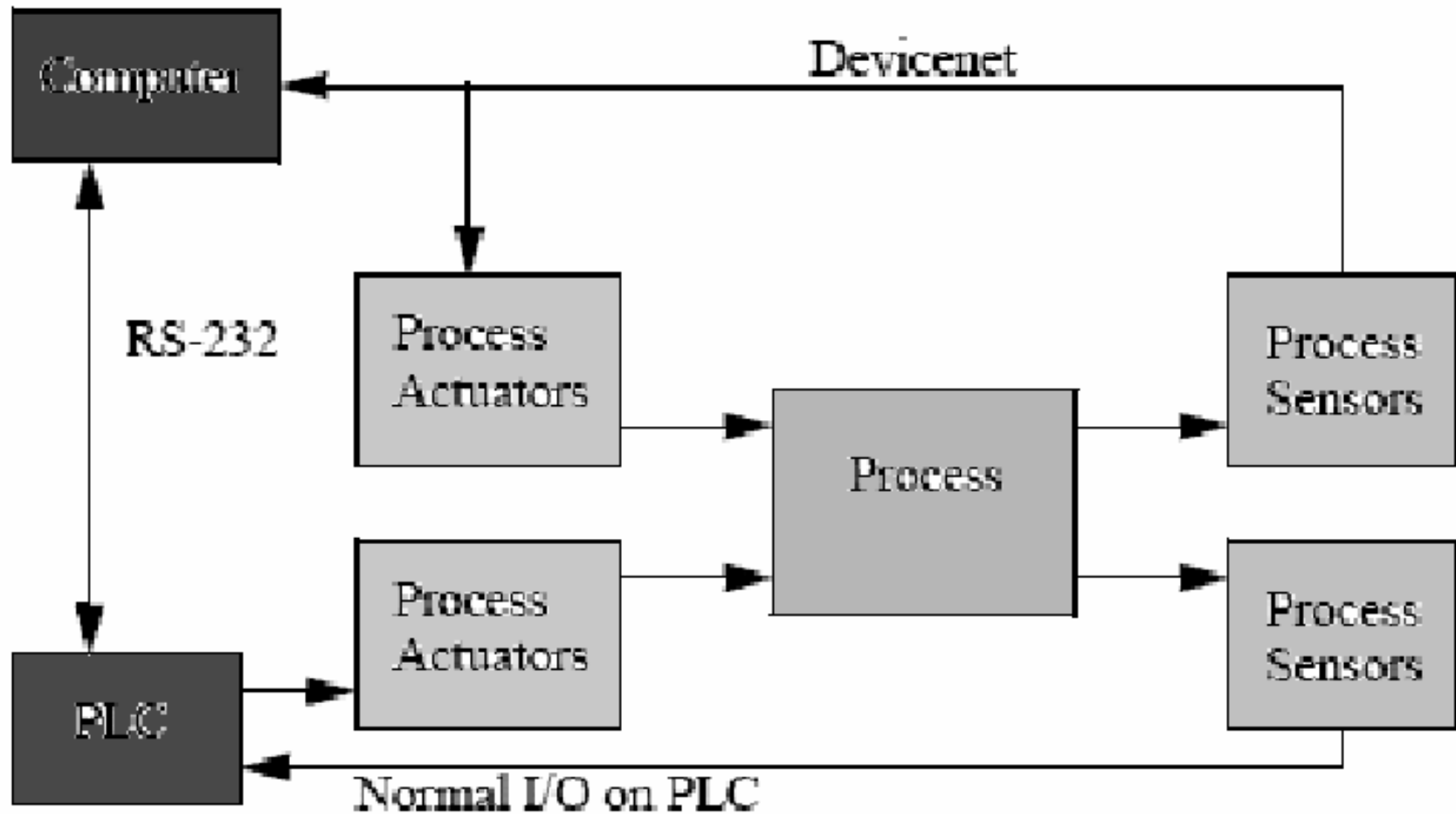
Parallel standards

- The standard interface most commonly used for parallel communication is IEEE-488, and now termed as General Purpose Instrument Bus (GPIB).
- Parallel data communications can take place between **listeners** , **talkers** , and **controllers**.
- There are **24 lines**: 8 data (bidirectional), 5 status & control, 3 handshaking, and 8 ground lines.

Serial communication

- **Serial** communication is when data is transmitted **one bit at a time**.
- A data word has to be **separated** into its constituent bits for transmission and then **reassembled** into the word when received.
- Serial communication is used for transmitting data over long distances.
- Might be used for the connection between a computer and a PLC.

A Communication Example



Serial standards (RS-232)

- RS (Recommended Standard) has been replaced with the EIA / TIA (Electronic Industries Association / Telecommunications Industry Association).
- The standard is officially EIA/TIA-232c. But, most of the world still call it RS-232c.
- **RS-232c** communications is the most popular method of PLC to external device communications. RS-232 is a communication interface included under **SCADA** applications.

Other Standards

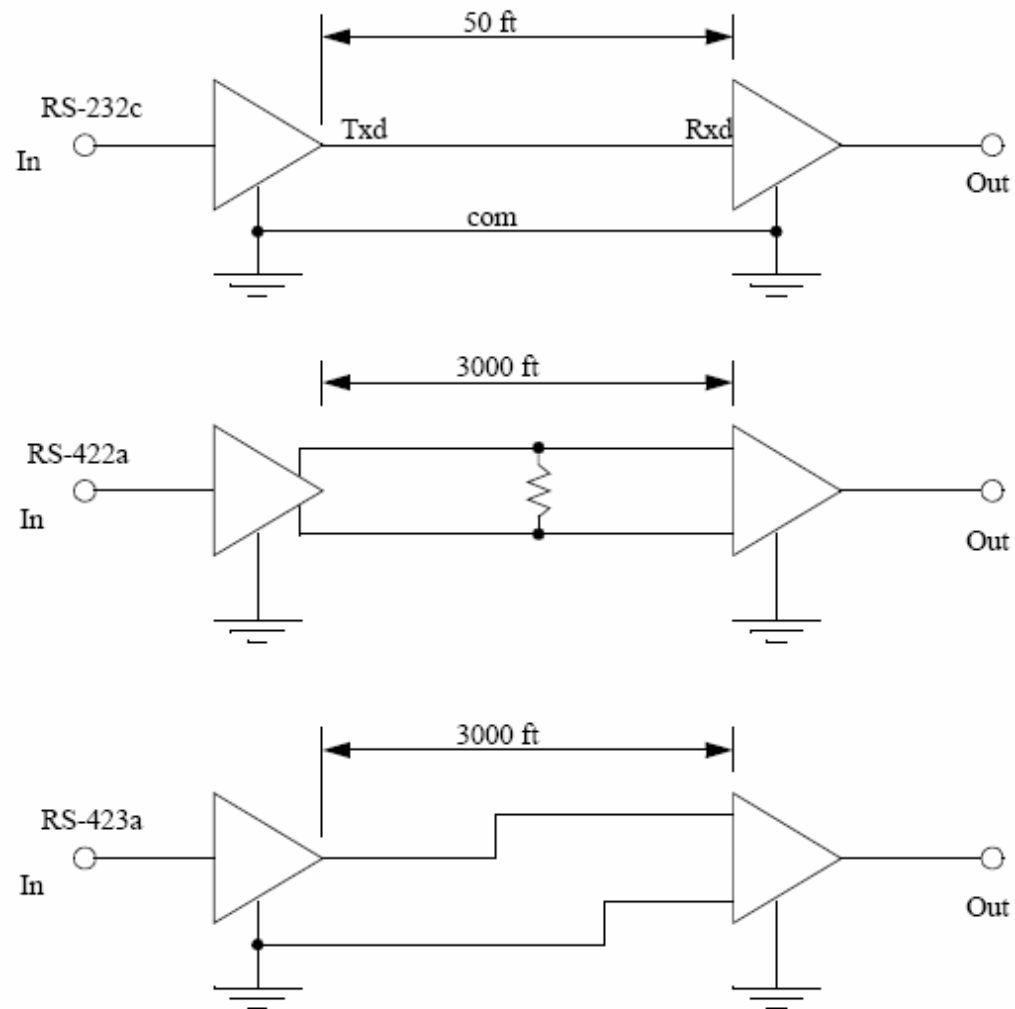
- Other standards permit higher transmission rates and longer cable distances.
 - **RS - 422**
 - **RS - 423**
 - RS - 485.
 - DH⁺ - 485.
 - EtherNet.

Communication Categories

- Electronic data communications between elements will generally fall into two broad categories:
 - Single-ended (RS-232, RS-423),
 - Differential (RS-485, RS-422).

Serial Data Standards

- **RS-232c** uses a voltage levels ± 12 V;
- **RS-422a** uses a 20 mA current loop;
- **RS-423a** uses a differential voltage level across two lines.



ASCII

- **ASCII** is a human-readable to computer-readable translation code (each letter/number is translated to 1's and 0's).
- It's a 7-bit code, so we can translate 128 characters (2^7 is 128).

START/STOP Bits

- **Start bit** is a synchronizing bit added just before each character we are sending.
(This is considered a **SPACE** or 0).
- **Stop bit** tells that the last character was just sent.
(This is considered a **MARK** or 1).

Parity bit

- Parity bit is added to check whether corruption has occurred. Common forms of parity are: **None, Even, and Odd.**
 - During transmission, the sender calculates the parity bit and sends it.
 - The receiver calculates parity for the character and compares the result to the parity bit received.
 - If the calculated and real parity bits don't match, an error occurred and an appropriate act to be taken.

Baud rate

- **Baud rate** - the number of bits per second that are being transmitted or received.
- Common values (speeds) are 1200, 2400, 4800, 9600, 19200, and 38400.

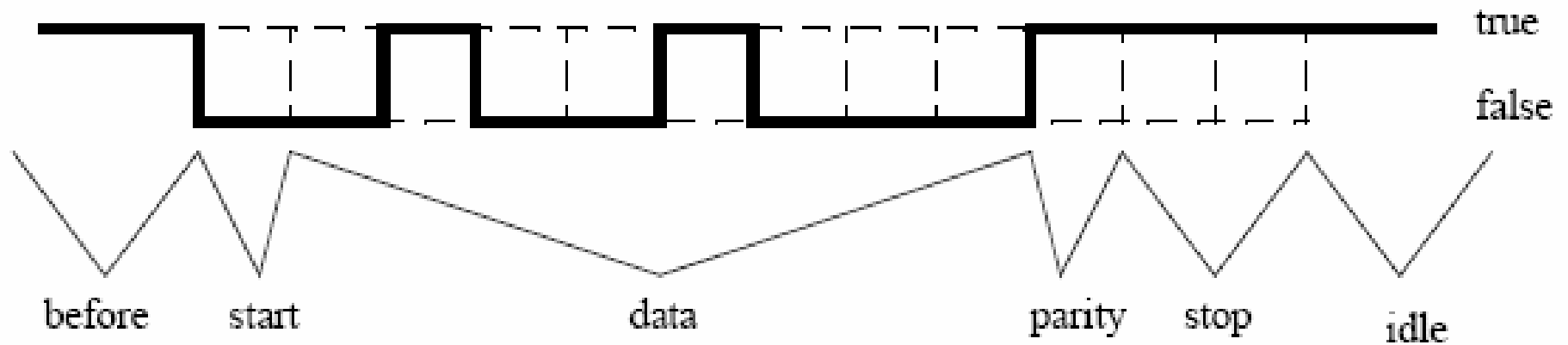
RS232 data format

- **RS232 data format**

(baud rate-data bits-parity-stop bits).

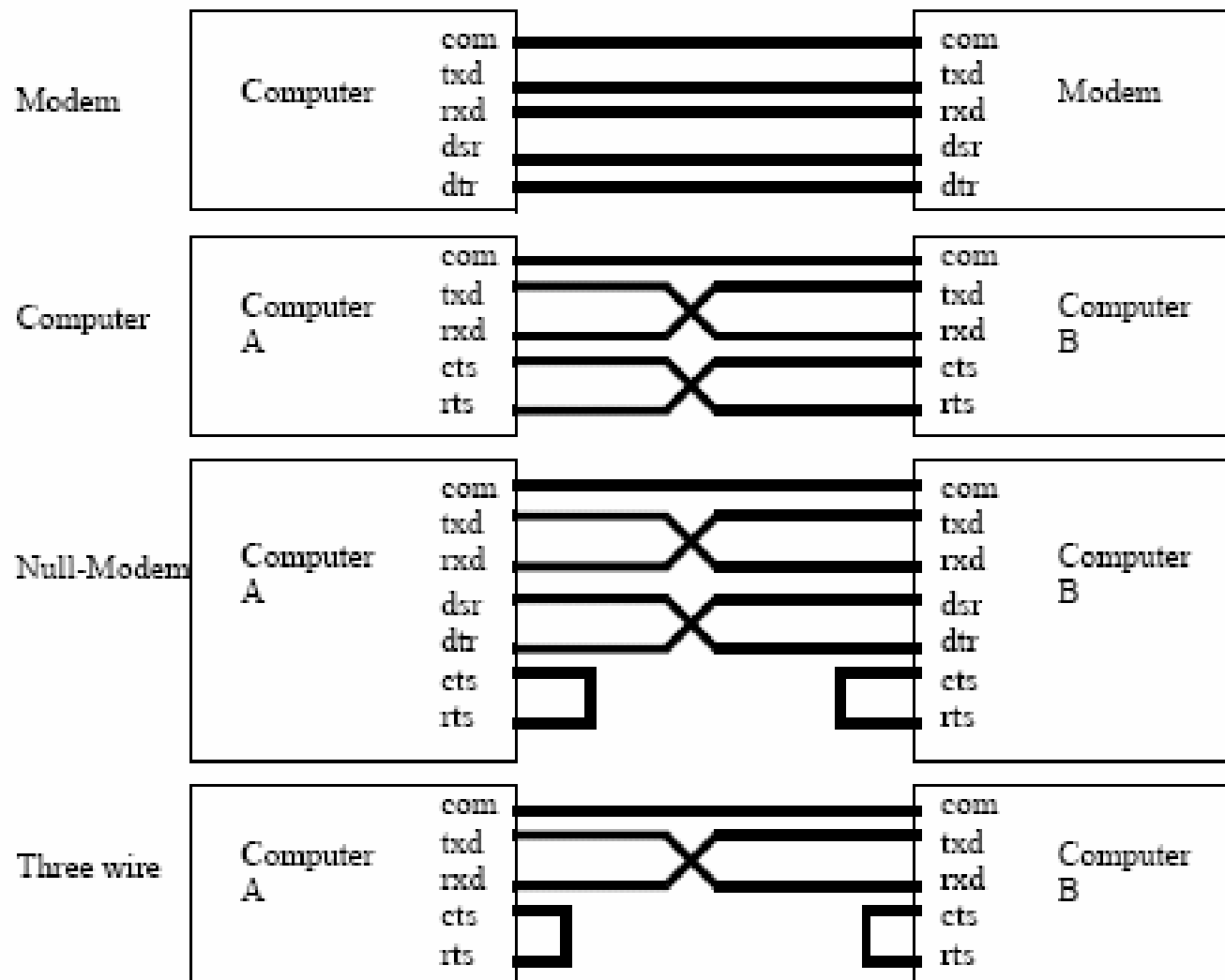
9600-8-N-1 means a baud rate of 9600, 8 data bits, parity of None, and 1 stop bit.

A Serial Data Byte



- a byte with the value 18 - (00010010)
- odd parity is being used
- two stop bits
- a 9600 baud would transfer up to
 $9600 / (1 + 8 + 1 + 2) = 800$ bytes each second

Common RS-232 Connection Schemes



DB9 Pins Description

- **TXD/RXD** (transmit data, receive data) - data lines
- **DCD** (data carrier detect) - indicates when a remote device is present
- **RI** (ring indicator) - used by modems to indicate when a connection is about to be made.
- **CTS/RTS** (clear to send, ready to send)
- **DSR/DTR** (data set ready, data terminal ready) these handshaking lines indicate when the remote machine is ready to receive data.
- **COM** - a common ground to provide a common reference voltage for the TXD and RXD.

Software Handshaking

- **Software handshaking (flow control)** is used to make sure both devices are ready to send/receive data.
- The most popular "character flow control" is called **XON/XOFF**.
- The receiver sends the XOFF character when it wants the transmitter to pause sending data. When it's ready to receive data again, it sends the transmitter the XON character.

STX & ETX

- Sometimes an **STX** and **ETX** pair is used for transmission/reception as well. STX is "start of text" and ETX is "end of text".
- The STX is sent before the data and tells the external device that data is coming. After all the data has been sent, an ETX character is sent.

ACK / NAK Pair

- The transmitter sends its data. If the receiver gets it without error, it sends back an ACK character.
- If there was an error, the receiver sends back a NAK character and the transmitter resends the data.

RS-232 Communications (hardware)

- RS-232 is an **asynchronous communications method**
(a marching band must be "in sync" with each other so that when one steps they all step. They are asynchronous in that they follow the band leader to keep their timing).

- We use a binary system to transmit our data in the ASCII format.
- PLCs serial port is used for transmission/reception of the data. It works by sending/receiving a voltage.
- With RS232, normally, a 1 bit is represented by a voltage -12 V, and a 0 by a voltage +12 V. (The voltage between +/- 3 volts is considered noise).

- *There are 2 types of RS-232 devices.*
 - **DTE - Data Terminal Equipment** and a common example is a computer.
 - **DCE - Data Communications Equipment** and a common example is a modem.
- PLC may be either a DTE or DCE device.
- When plc and external device are both DTE, (or both DCE) devices they can't talk to each other. The solution is to use a null-modem connection.

- Here's a typical communications session. Both devices are powered up.
- The plc is DTE and the external device is DCE.

- *The external device turns on **DSR (Data Set Ready** – pin 6) which tells the plc that's it's powered up and "there".*
- *The PLC turns on **RTS (Request to send** – pin 4) which is like asking the external device "are you ready to receive some data?"*
- *The external device responds by turning on its **CTS (Clear to Send** – pin 5) which says its ok to for the plc to send data.*

- *The plc sends the data on its TD (TRANSMIT DATA) terminal (Output Data – pin 2),*
- *The external device receives it on its RD (RECEIVE DATA) terminal (pin 3).*

Some data is sent and received.

- *After a while, the external device can't process the data quick enough. So, it **turns off** its CTS (CLEAR TO SEND) terminal (Pin 5) and the plc pauses sending data.*

- *The external device catches up and then turns its CTS terminal back on.*
- *The plc again starts sending data on its TD (transfer data – pin 2) terminal and the external device receives it on its RD (receive data – pin 3) terminal.*
- *The plc runs out of data to send and turns off its RTS (REQUEST TO SEND) terminal (pin 4).*
- *The external device sits and waits for more data.*

Using RS-232 with PLC

- Some manufacturers include RS-232 communication capability in the main processor.
- Some use the "programming port" for this.
- Others require a special module to "talk RS-232" with an external device.
- External device may be an operator interface, an external computer, a motor controller, a robot, a vision system, etc.

To communicate via RS-232 we have to setup:

- Where, in data memory, will we store the data to be sent?
- Where, in data memory, will we put the data we receive from the external device?
- How will we tell the plc when it's time to send our data out the serial port?

Connecting a PLC controller with a PC computer

- **PLC controller is linked with a PC computer through an RS-232 cable.**
- **One end of the cable is connected to a serial PC port (9-pin or 25-pin connector), while the other end is connected to a connector on the CPU or communication module of a controller.**

RS - 485

- RS-485 is one of multi-drop communication that allows us to 'talk' to multiple devices at the same time.
- According to the standard, up to 32 devices can be connected at the same time. Maximum distance from end to end can be up to 1200 meters.
- By using repeaters, however, both the total number of devices and maximum distance can be extended.

- RS-485 network can be used as a two-wire or four-wire network.
- The four wire network would be bi-directional (a simultaneous two way conversation can happen) whereas the two wire network works only in one direction.
 - It is either a 3 or 5 wire system. The third or fifth wire is actually a ground wire.

Setting up the DH-485 Network

- DH-485 (Data Highway) network designed to pass information between devices on the plant floor.
- DH-485 Network protocol supports two classes of devices: initiators and responders.
- **All initiators on the network get a chance to initiate message transfers.** To determine which initiator has the right to transmit, a **token passing algorithm** is used.

- A node holding the token can send any valid packet onto the network. Each node is allowed only one transmission (plus two retries) each time it receives the token.
- After a node sends one message packet, it attempts to give the token to its successor by sending a "token pass" packet to it.

- If no network activity occurs, the initiator sends the token pass packet again.
- After two retries the initiator will attempt to find a new successor.
- There must be at least one initiator on the network.

- When the time for a **link dead timeout** is exceeded, usually the initiator with the lowest address claims the token.

When an initiator has the token it will begin to build the network.

Protocols

- Protocol is defined to exercise control of the flow of data between two devices so what constitutes the message, and how the communication is to be initiated and terminated.
- A protocol is a set of rules that two or more devices must follow if they are to communicate with each other.
- Protocol includes everything from the meaning of data to the voltage levels on

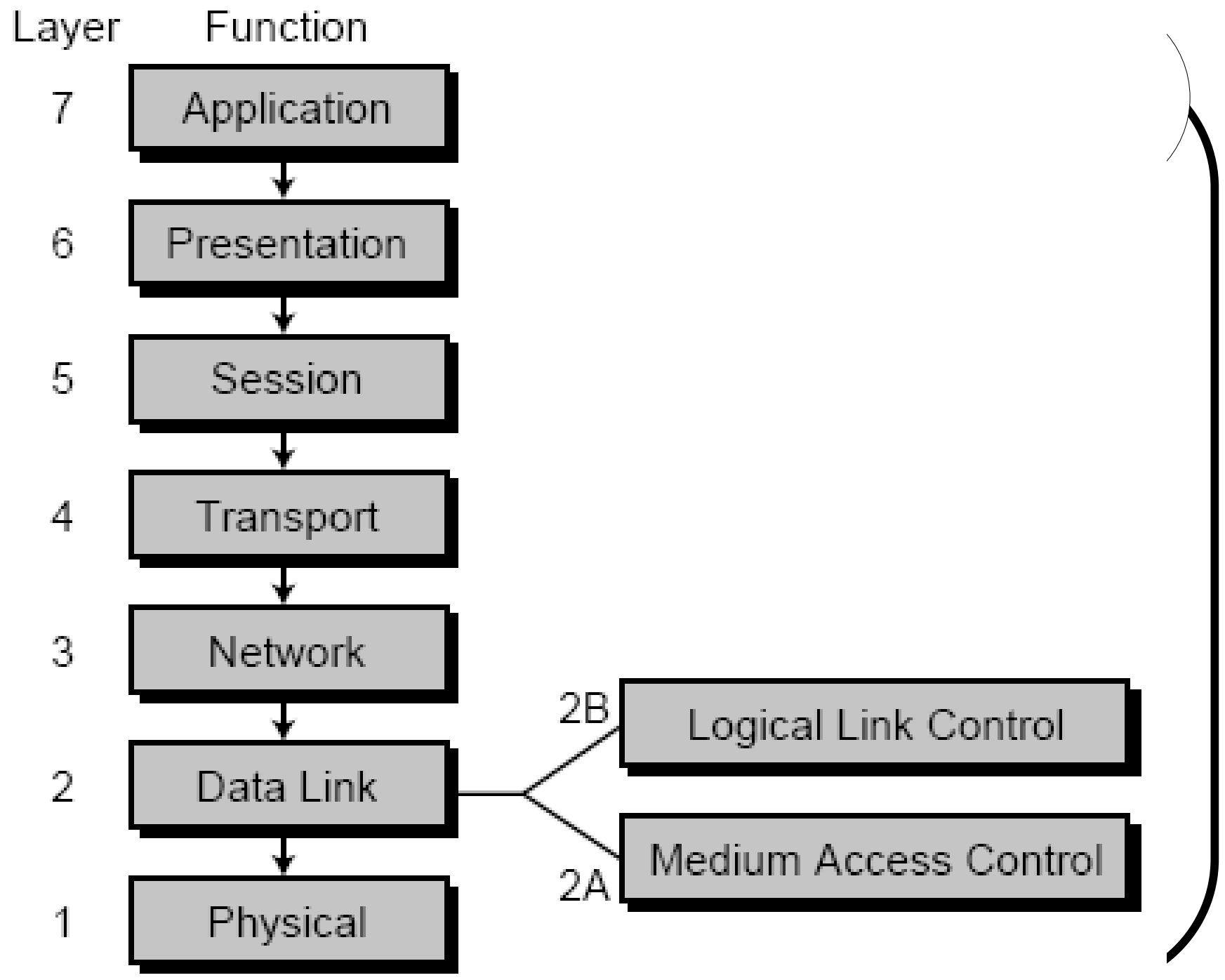
- A network protocol defines how a network will handle the following problems and tasks:
 - communication line errors
 - flow control (to keep buffers from overflowing)
 - access by multiple devices
 - failure detection
 - data translation
 - interpretation of messages

ISO / OSI model

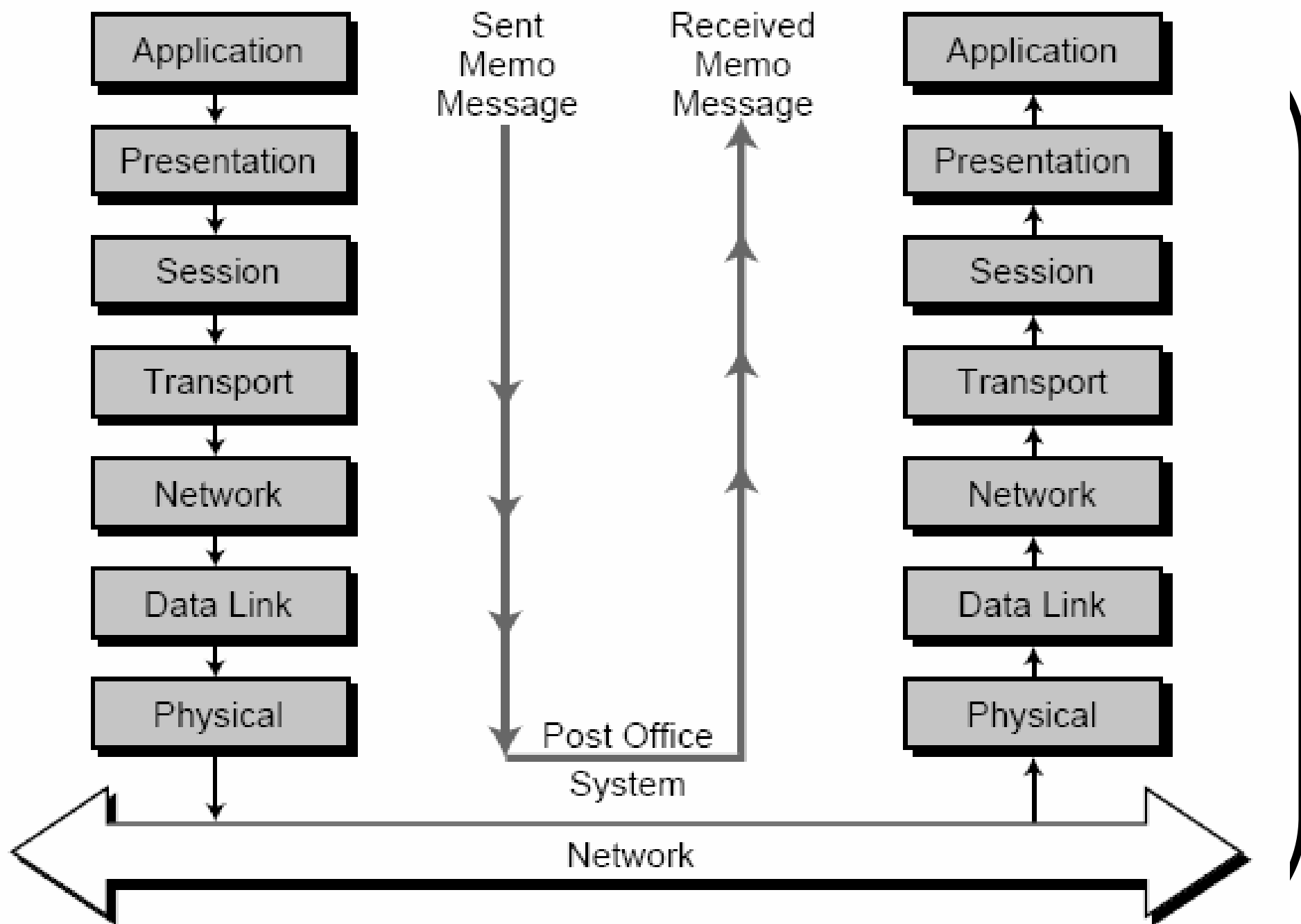
- Interconnecting several devices can present problems because of compatibility problems.
- In order to facilitate communications between different devices the International Standard Organization (ISO) devised a **ISO/OSI model** to be used for standardization for **Open System Interconnection (OSI)**.

ISO /OSI Model

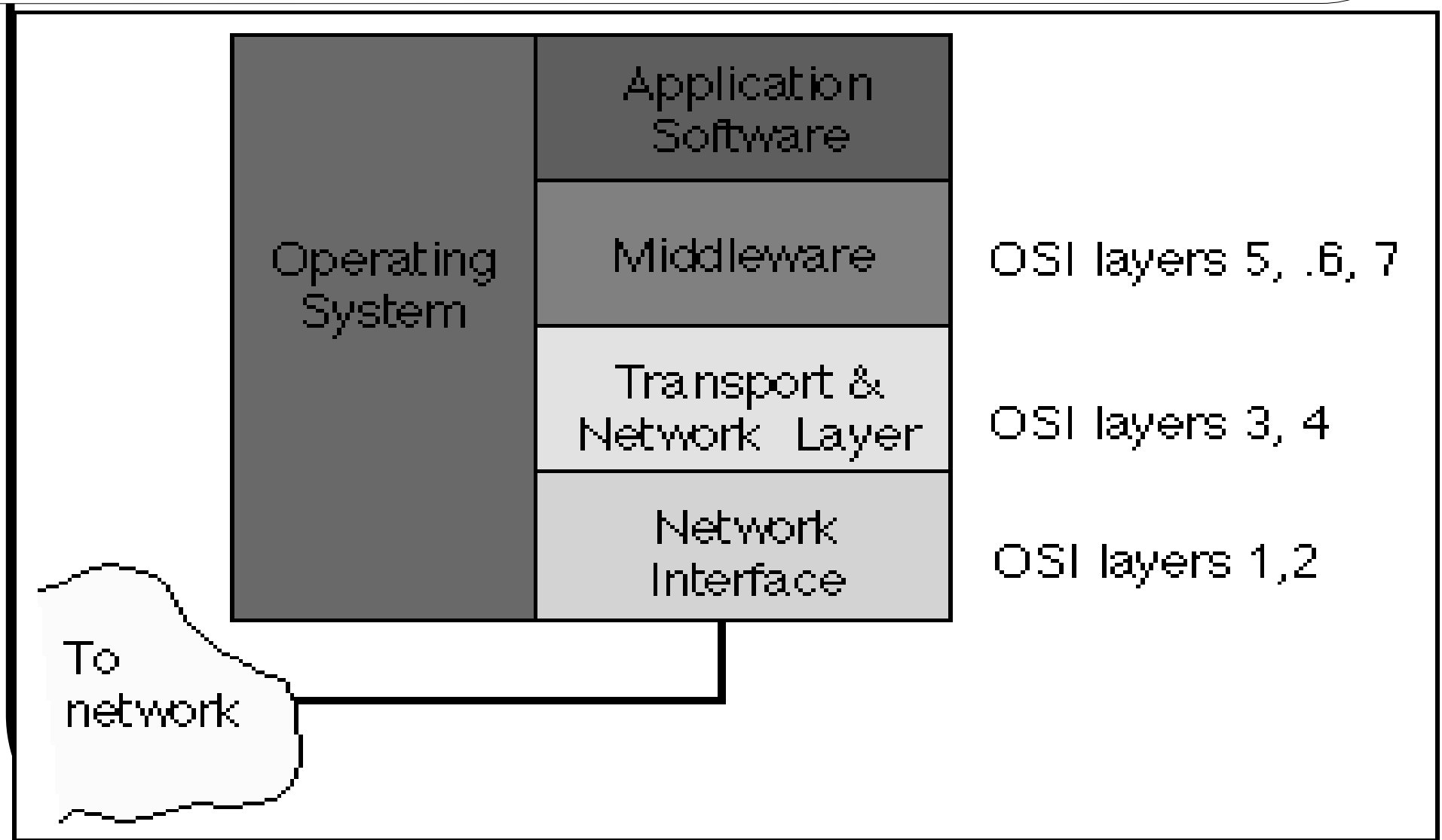
- A communication link between items of digital equipment is defined in terms of
 - **physical,**
 - **electrical,**
 - **protocol and**
 - **user standards.**



Layer	Layer Name	Function
Layer 7	Application	The level seen by users; the user interface
Layer 6	Presentation	Control functions requested by the user; data is restructured from other standard formats; code and data conversion
Layer 5	Session	System-to-system connection; log-in and log-off controlled here; establishes connections and disconnections
Layer 4	Transport	Provides reliable data transfer between end devices; network connections for a given transmission are established by protocol
Layer 3	Network	Outgoing messages are divided into packets; incoming packets are assembled into messages for higher levels, establishing connections between equipment on the network
Layer 2	Data link	Outgoing messages are assembled into frame and acknowledgements; error detection or error correction is performed
Layer 1	Physical	Parameters, such as signal voltage swing, bit duration, and electrical connections, are established in this layer



ISO/OSI Model (4/5)



ISO/OSI Model

- Each layer is self contained and only deals with the interfaces of the layer immediately above and below.

It performs its tasks and transfers its results to the layer above or the layer below.

- It enables manufacturers of products to design products operable in a particular layer that will interface with the hardware of other manufacturers.

Protocols

- ControlNet (AB)
- DeviceNet (AB)
- ModBus (Modicon)
- ProfiBus (Siemens)
- Ethernet

PROFIBUS

- PROFIBUS-DP purpose is for larger devices like PCs and PLCs to talk with multiple smaller devices like sensors, drives, valves, etc.
- It uses RS-485 for transmission of data.
- It uses a shielded twisted pair cable and enables data transmission speeds up to 12 Mbit/sec.

- A maximum of 9 segments (trunk line) are allowed on a network. The devices are the branches coming off the trunk line.
- Up to 32 individual devices can be connected to a single segment. That number can be expanded up to 126 if repeaters are used.
- Each PROFIBUS segment can be a maximum of 1200 meters in length. There are 10 defined communication speeds and each has a maximum defined cable length that's permitted.

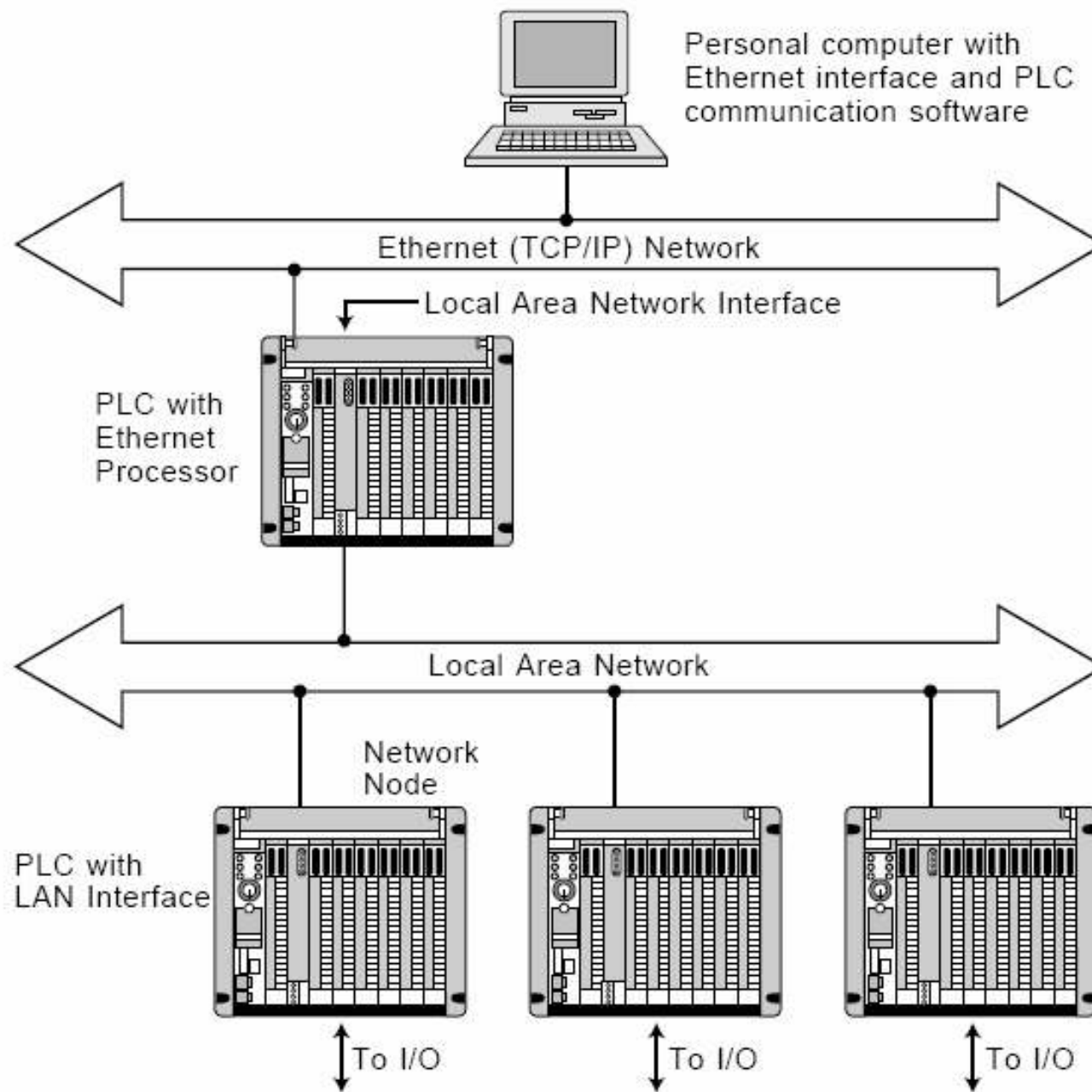
Master / Slave

- PROFIBUS uses a **master/slave configuration** for communication.
- It is usually a **single master device** (a PLC) that talks with **multiple slave devices** (sensors).
- The master devices **poll** the slaves when they have the token. Slave devices only answer when asked a question. They are passive and the master can be said to be active. The slave devices just collect data and pass it to the master device when asked to do so.

TCP/IP PROTOCOL

- Most manufacturers who offer Ethernet compatibility to implement supervisory functions over equipment controlling plant floor functions use a **transmission control protocol/internet protocol (TCP/IP)** for layers 3 and 4 of the OSI model.
- Some PLC manufacturers offer programmable controllers with TCP/IP over-Ethernet protocol built into the PLC processor. This allows the PLC to connect directly to a supervisory Ethernet network.

Note that the PLC can also have a control network with other PLCs



Networks

- As control systems become more complex, they require more effective communication schemes between the system components.
- Some machine and process control systems require that programmable controllers be interconnected, so that data can be passed among them easily to accomplish the control task.
- Other systems require a plantwide communication system that centralizes functions, such as data acquisition, system monitoring, maintenance diagnostics, and management production reporting, thus providing maximum efficiency and productivity.

LOCAL AREA NETWORKS

- The term **local area network** (LAN) is used to describe a communication network designed to link computers and their peripherals within the same building or site.
- A local area network is a high-speed, medium-distance communication system.
- For most LANs,
 - the maximum distance between two nodes in the network is at least one mile, and
 - the transmission speed ranges from 1 to 20 megabaud.

● Also, most local networks support at least 100 stations or nodes.

Industrial Network

- A special type of local area network, the industrial network, is one which meets the following criteria:
 - capable of supporting real-time control
 - high data integrity (error detection)
 - high noise immunity
 - high reliability in harsh environments
 - suitable for large installations.

LAN APPLICATIONS OF THE PLC

- **Centralized data acquisition and distributed control** are the most common applications of local area networks.
- Data collection and processing, when performed by an individual controller, can burden the processor's scan time, consume large amounts of memory, and complicate the control logic program.

- A data highway configuration, in which all data is passed to a host computer that performs all data processing, eliminates these problems.
- Also, distributed control applications allocate control functions, once performed by a single controller, among several controllers.
- This eliminates dependence on a single controller and improves performance and

- To use the distributed processing approach, a local area network and the PLCs attached to it must provide the following functions:
 - communication between programmable controllers
 - upload capability to a host computer from any PLC
 - download capability from a host computer to any PLC
 - reading/writing of I/O values and registers to any PLC

I/O BUS NETWORKS

- This network lets controllers better communicate with I/O field devices, to take advantage of their growing intelligence.
- This configuration decentralizes control in the PLC system, yielding larger and faster control systems.
- Two types of I/O bus networks
 - **device-level bus and**
 - **process bus.**

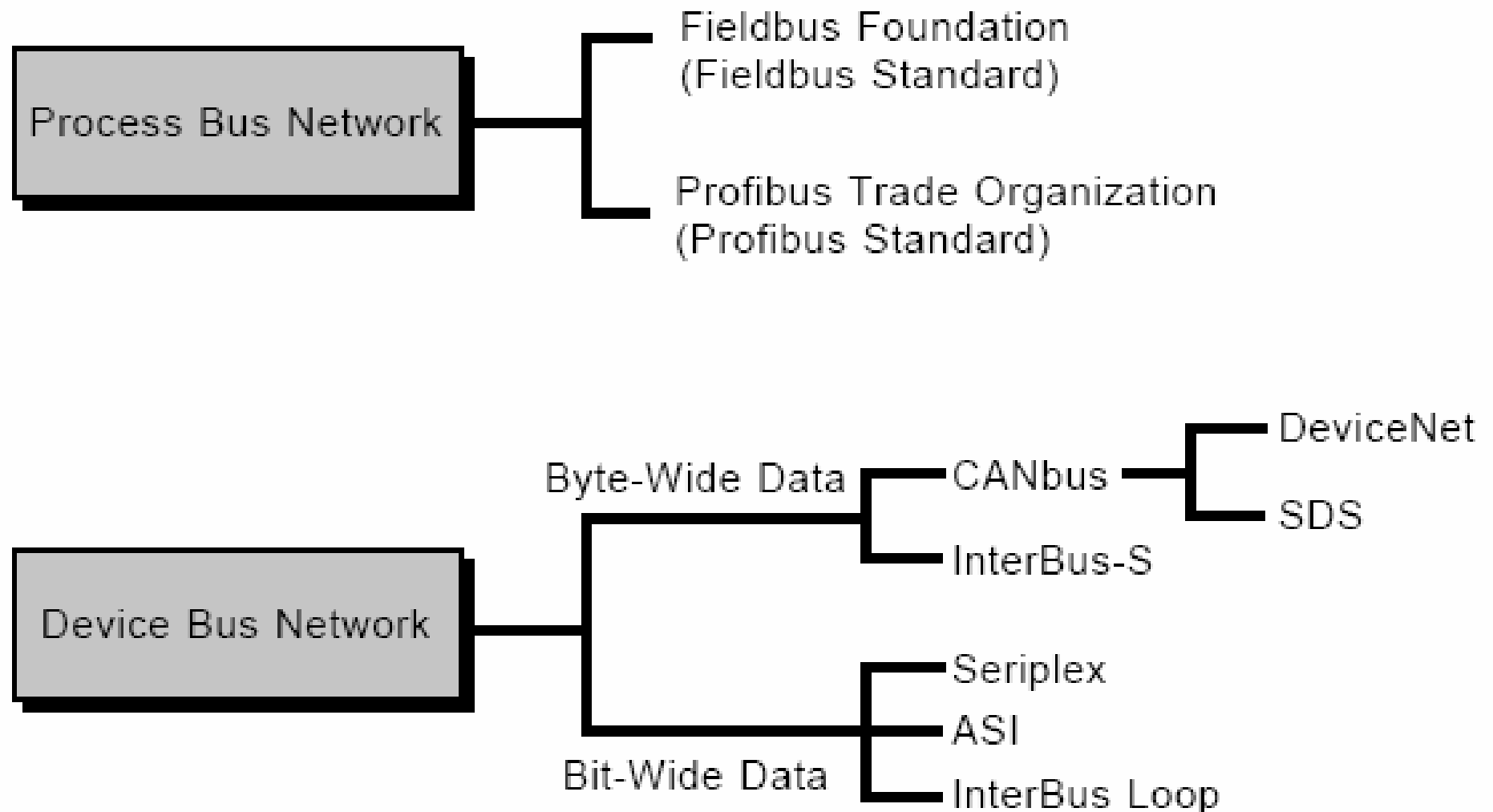
Device bus networks

- **Device bus networks** interface with low-level information devices (e.g., push buttons, limit switches, etc.), which primarily transmit data relating to the state of the device (ON/OFF) and its operational status (e.g., operating OK).
- These networks generally process only a few bits to several bytes of data at a time.

Process bus networks

- **Process bus networks** connect with high-level information devices (e.g., smart process valves, flow meters, etc.), which are typically used in process control applications.
- Process bus networks handle large amounts of data (several hundred bytes), consisting of information about the process, as well as the field devices themselves.

Network and protocol standards.

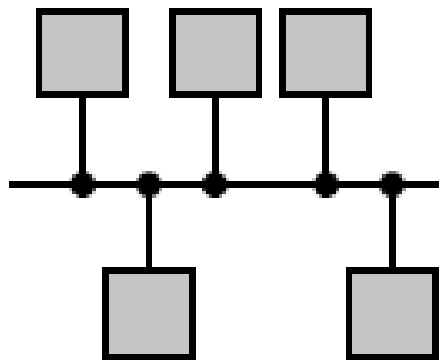


NETWORK TOPOLOGIES

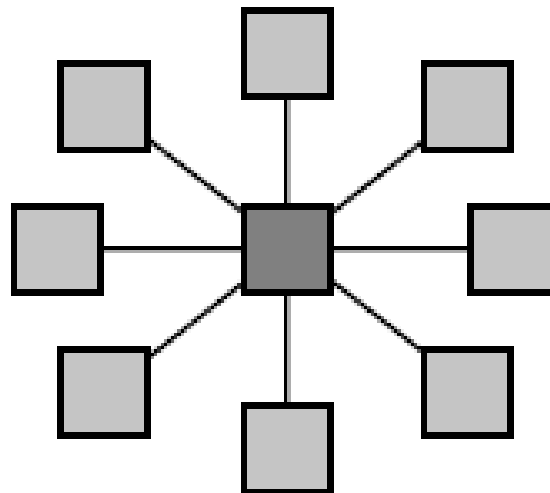
- The *topology* of a local area network is the geometry of the network, or how individual nodes are connected to it.
- A network's topology greatly affects its throughput rate, implementation cost, and reliability.
- A large network may consist of a number of interconnected topologies.

- The basic network topologies used today are:

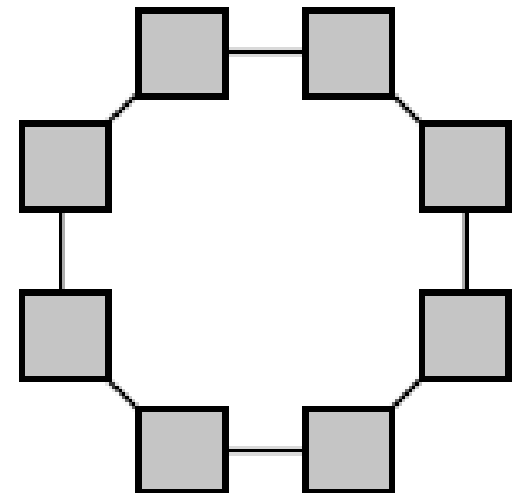
Bus



Star



Ring

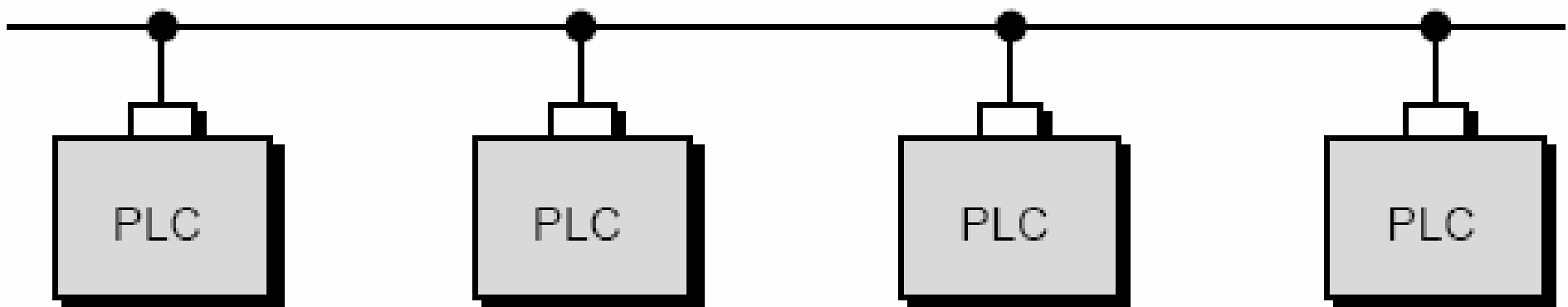


Common Bus Topology

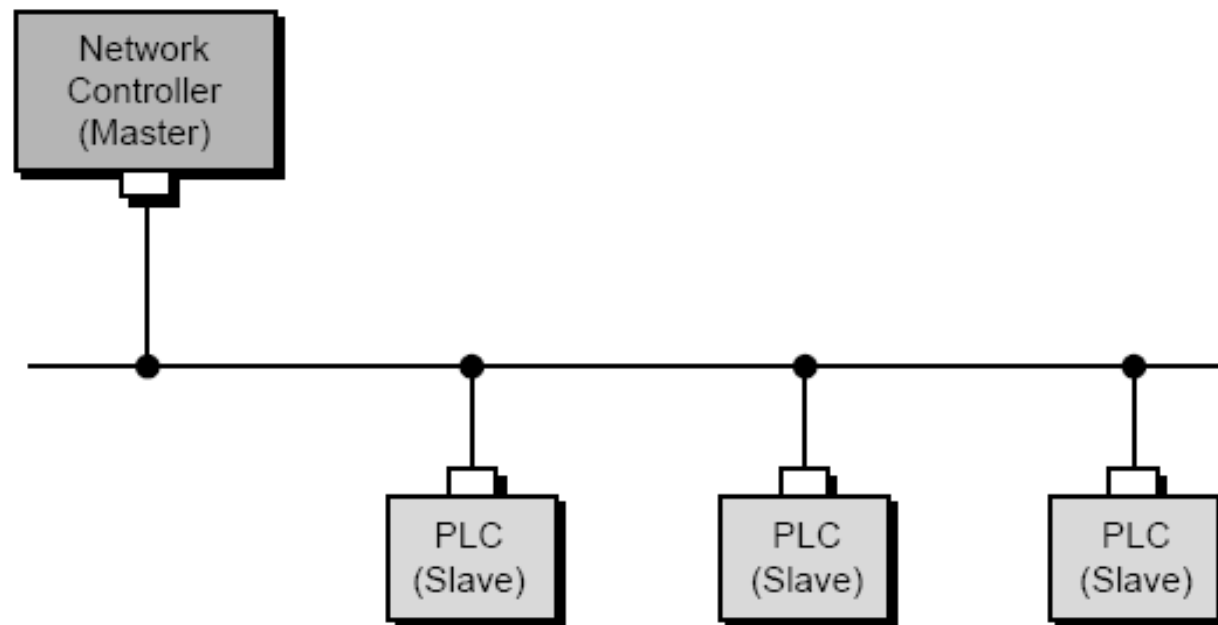
- The **common bus topology** has a main trunkline to which individual PLC nodes are connected in a multidrop fashion.
- A coaxial cable with proper terminators is typically the communication medium for the trunkline.
- Communication in a common bus network can occur between any two nodes without passing information through a network controller.

- An inherent problem of this scheme, however, is determining which node may transmit at which time, to avoid data collision.
- Several communication access methods have been developed to solve this problem. Although many access methods exist, the most commonly used ones are
 - polling (*master/slave access method*),
 - collision detection (CSMA/CD - carrier sense multiple access with collision detection)., and
 - token passing

- Common bus topologies are very useful in distributed control applications, since each station has equal independent control capability and can exchange information at any given time.



- Another configuration of the bus topology is the **master/slave bus topology**, consisting of several slave controllers and one master network controller.



- In this configuration, the master sends data to the slaves; if the master needs data from a slave, it will *poll* (address) the slave and wait for a response. No communication takes place without the master initiating it.
- The implementation of a master/slave bus topology uses two pairs of wires.
 - Through one pair of wires, the master transmits data and the slaves receive it.
 - Through the other pair of wires, the slaves transmit data and the master receives it.

- The **single highway (Bus)** and the **ring** methods are often termed **PEER TO PEER** in that each terminal has equal status.

NETWORK SPECIFICATIONS

- To determine if a particular network can support a given application (when choosing a network), the designer should examine all aspects of the network, including
 - device specifications,
 - response time,
 - maximum length,
 - throughput, and
 - Interface.

- Next Class

PLC Programming

PLC System Engineering

Dr. Mazouz Salahat

Objectives

- *Be able to select vendor, needed hardware and software;*
- *Be able to install and commission PLC system;*
- *Be able to test and maintain PLC system.*

Topics

- PLC System Selection;
- PLC Installation and Commissioning;
- PLC Troubleshooting.

Vendor selection

- The range of PLC suppliers is vast and many offer a number of alternative product ranges with any number of modules, boasting special features.
- Our choice must meet the application requirements, provide extra capacity for future development and provide a cost-effective solution.

- Most suppliers have a number of ranges of PLCs. The cost/performance of this hardware often increases in stages. The **features** available at each step vary from supplier to supplier, giving an advantage for any one application.
- Always take care not to select a PLC that will be close to its maximum potential. Choosing hardware with a similar specification from the next range will cost more but there is an easy upgrade path to enhance the performance, i.e. extra memory, faster processors, etc.

- Price is the most commonly stated reason for making a choice, but the true price of a PLC to meet the requirements of a particular application is often much the same over a wide range of supplier equipment.
- The final choice of supplier for PLC will depend upon **Vendor selection Issues:**
functionality, support available, customer preferences, user knowledge and price.

Vendor selection Issues

- *Functionality* - Match the application requirements with the features of each of the contending suppliers' equipment to identify the one that best meets our requirements.
- *Support* - Before any purchase is made the following points should be confirmed with any manufacturer:
 - Training;
 - Technical support (on site and over the phone);
 - Application support to configure and design a system;
 - Rapid exchange/repair of failed equipment;
 - Guaranteed support for any products for at least 10 years from purchase.

- *Customer acceptability* - In many cases end-users express a preference for a particular PLC because they are already using that type of equipment and have the skills available to keep it running.
- *User knowledge* - Programs and techniques that use conventional logic functions will move without any problems to most PLCs. Problems occur when the more complex instructions and methods have been used. It may even require a total change to the overall program design philosophy if certain key instructions or methods are not supported.

- *Cost* - The cost of a control system is made up of the following items:
 - Purchase price of PLC;
 - Programming cost;
 - Documentation cost;
 - Installation cost;
 - Maintenance cost.

PLC selection criteria

- *System (task) requirements.*
- *Application requirements.*
 - What input/output capacity is required?
 - What type of inputs/outputs are required?
 - What size of memory is required?
 - What speed is required of the CPU?
- *Electrical requirements.*
- *Speed of operation.*
- *Communication requirements.*
 - Software.
- *Operator interface.*
- *Physical environments.*

System requirements

- The starting point in determining any solution must be to understand what is to be achieved.
- The program design starts with breaking down the task into a number of simple understandable elements, each of which can be easily described.

Application requirements

- **Input and output device requirements.**

After determining the operation of the system, the next step is to determine what input and output devices the system requires.

- List the function required and identify a specific type of device.

- **The need for special operations in addition to discrete (On/Off) logic.**

- List the advanced functions required beside simple discrete logic.

Electrical Requirements

- The electrical requirements for **inputs, outputs, and system power**;

When determining the electrical requirements of a system, consider three items:

- Incoming power (power for the control system);
- Input device voltage; and
- Output voltage and current.

Speed of Operation

- How fast the control system must operate (speed of operation).

When determining speed of operation, consider these points:

- How fast does the process occur or machine operate?
- Are there “time critical” operations or events that must be detected?
- In what time frame must the fastest action occur (input device detection to output device activation)?
- Does the control system need to count pulses from an encoder or flow-meter and respond quickly?

Communication

- If the application requires sharing data outside the process, i.e. communication.

Communication involves sharing application data or status with another electronic device, such as a computer or a monitor in an operator's station.

Communication can take place locally through a twisted-pair wire, or remotely via telephone or radio modem.

Operator Interface

- If the system needs operator control or interaction.

In order to convey information about machine or process status, or to allow an operator to input data, many applications require **operator interfaces**.

Traditional operator interfaces include pushbuttons, pilot lights and LED numeric display.

Electronic operator interface devices display messages about machine status in descriptive text, display part count and track alarms. Also, they can be used for data input.

Physical Environment

- The physical environment in which the control system will be located.

Consider the environment where the control system will be located.

In harsh environments, house the control system in an appropriate IP-rated enclosure.

Remember to consider accessibility for maintenance, troubleshooting or reprogramming.

Hardware selection

Choosing the correct I/O hardware

By knowing the number of any type of I/O lines we need and the number of lines available on a given module, the final shopping list of modules and the size of the PLC system are determined.

In addition, build in at least 20 per cent extra capacity to allow to future modifications or to solve problems identified during commissioning.

Discrete I/O modules

Input selection

For each input we need to determine the followings:

- Voltage level.
- Response speed.

Output Selection

For each output we need to consider the followings:

- Voltage level.
- The power that PLC outputs need to switch.
- Output resistance and electrical noise can be an issue in cases where low level signals are to be switched.
- The use of a.c. outputs can often be an advantage.
 - In most cases the voltage is higher, giving a reduction in current for any particular load, the consequent reduction in the **wire size** required giving a reduction in wiring costs.
 - A second and often more important advantage is the reduction of electromagnetic interference (**EMI**).

Analog I/O modules

The following terms, common for analog input and output modules, used to describe their performance;

- *Resolution* defines how accurately the analog to digital (A/D) or digital to analog (D/A) converter within the module can represent an analog voltage as a binary number, or vice versa.
- *Isolation* refers to the ability of each input or output to work at voltage levels independent of the system ground.

Analog Input Modules

To select an analog input module the consider the followings:

- Voltage level.
- Current input.
- Conversion speed. There are two basic types of A/D converter. The first will perform a conversion every 20 ms (the period of the a.c. mains voltage), which gives a good clean reading free from worries of line frequency interference.
The second will convert in 2-20 μ s, giving the possibility of measuring transient data.
- At very high rates the PLC may only have time to act as a data logger, storing the data as it is read, and analyzing it some time after the event to report on or display it.

Analog Output Modules

- The conversion speed of an analog output is generally $<100 \mu\text{s}$ and rarely a problem.
- Once the resolution of the module is selected we have only to consider the following points:
 - Voltage level.
 - Load resistance. Typically the minimum load resistance is 300 Ohm.
 - Current output. It is often an advantage to use a current loop output (4-20 mA).

Analog closed loop control

In many cases analog inputs are used not only to **monitor** variables but also as a **feedback** to control a process by controlling relay outputs or varying an analog output.

The required control accuracy must be defined in the specification, as must the maximum required rate of change.

Counters, encoders and positioning

In order to select the correct hardware consider:

- The speed;
- The total number of pulses to be counted;
- The positioning accuracy.
 - There are many possible solutions available each of which offers a trade-off between **speed** of movement and positioning **accuracy**.

Choosing the correct processor (1)

For Selecting Modular Processors the following Criteria examined include:

- I/O points (local I/O points and expandable points).
- Each PLC processor will only be capable of working with a limited number of each type of I/O modules.
- Memory size (for data storage or program storage) and Performance (scan time depends on the processor).
 - The size of program is dependent upon the complexity of the control problem and the skill and style of the programmer.

Choosing the correct processor (2)

- The required operating speed for all the I/O must be determined, with a PLC selected to match.
 - This requires the estimation of the program size and the proportion of slow instructions.
 - The scan speed is normally expressed in terms of ms/K for a stated mix of simple and complex instructions.
 - A PLC with an appropriate memory capacity and speed can be selected.
- Advanced instruction support.
 - For any particular application it is essential to ensure that the PLC selected can handle the required operations.
- Communication options.
 - When a communications facility is required we need to determine whether the built-in port is adequate for the application, or whether a separate module will be required.

PLC Installation, Commissioning and Troubleshooting Procedures and Recommendations

Typical installation

- Typical installation (enclosure, disconnect device, fused isolation transformer, master control relay, terminal blocks and wiring ducts, suppression devices).
- Spacing controllers – follow the recommended minimum spacing to allow the convection cooling.
- Preventing excessive heat (0–60?) C
- Grounding guidelines.
- Power considerations.
- Safety considerations.
- Preventive maintenance considerations.

Commissioning and testing of a PLC system

- Checking that all cable connections between the PLC and the plant are complete, safe, and to the required specification and meeting local standards.
- Checking that all the incoming power supply matches the voltage setting for which the PLC is set.
- Checking that all protective devices are set to their appropriate trip settings.
- Checking that emergency stop button work.
- Checking that all input/output devices are connected to the correct input/output points and giving the correct signals.
- Loading and testing the software.

Testing inputs and outputs

- Input devices can be manipulated to give the open and closed contact conditions and the corresponding LED on the input module observed.
- Forcing also can be used to test inputs and outputs.

This involves software, rather than mechanical switching on or off, being used with instructions to turn off or on inputs/outputs.

Testing Software

Most PLCs contain some software checking program. This checks through the installed program and provides a list on a screen or as printout with any errors detected.

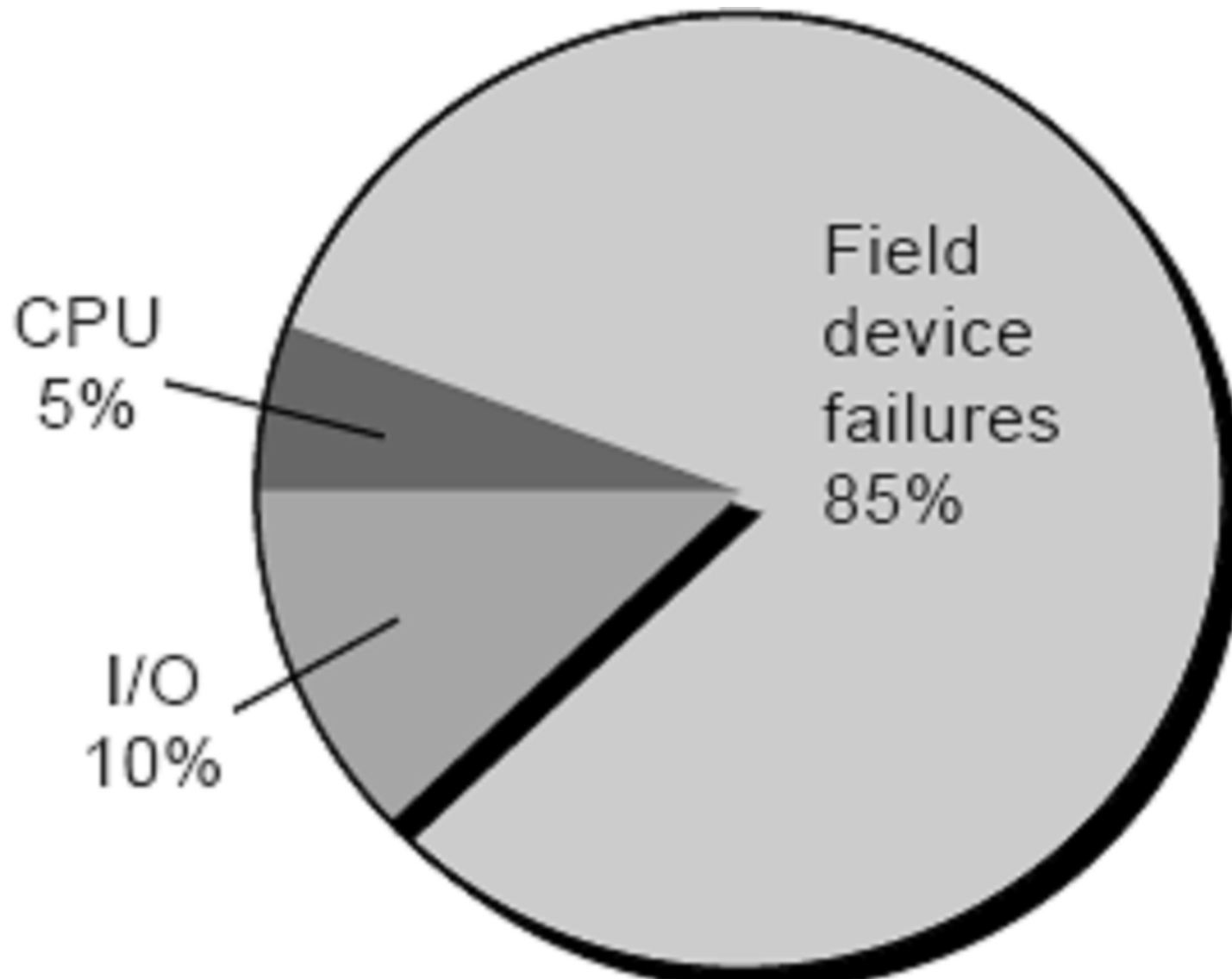
Fault Detection Techniques

Many PLCs provide built-in fault analysis procedures which carry out self-testing and display fault codes, with possibly a brief message, which can be translated by looking up code in a list to give the source of the fault and possible method of recovery.

The common fault detection techniques used are:

- Timing checks (watchdog).
- Replication (duplication).
- Expected value checks.

Failures in a PLC-based system.



Case 1

Consider a single output device failing to turn on though the output LED is on.

- If testing of the PLC output voltage indicates that it is normal then the fault might be a wiring fault or a device fault.
- If checking of the voltage at the device indicates the voltage there is normal then the fault is the device.

Case 2

Failure of an input LED to illuminate as required could be because:

- Input device is not correctly operating,
- Input device is not correctly powered,
- Incorrect wiring connections to the input module, or
- LED or input module is defective.

Program Storage

- Application programs may be loaded into battery-backed RAM in a PLC, or in EPROM.
- Against the possibility of memory failure occurring in the PLC and loss of the stored application program, a back-up copy of each application program should be kept.
- The program can then again be downloaded into the PLC without it having to be rewritten.
- The back-up may be on an EPROM cartridge, on a floppy disk or a hard disk.

Program troubleshooting

There are several causes of alteration to the user program:

- extreme environmental conditions,
- Electromagnetic Interference (EMI),
- improper grounding,
- improper wiring connections, and
- Unauthorized tampering.

If you suspect the memory has been altered, check the program against a previously saved program on an EEPROM, UVPROM or flash EPROM module.

Hardware troubleshooting

Tips for troubleshooting control system

If installation and start-up procedures were followed closely, controller will give reliable service.

If a problem should occur, the first step in the troubleshooting procedure is to

identify the problem and its source.

Do this by observing your ***machine*** or ***process*** and by ***monitoring the diagnostic LED indicators*** on the *CPU, Power Supply and I/O modules.*

Diagnostic Indicators

By observing the diagnostic indicators on the front of the processor unit and I/O modules, the majority of faults can be located and corrected.

These indicators, along with error codes identified in the programming device user manual and programmer's monitor, help trace the source of the fault to the ***user's input/output devices, wiring, or the controller.***

Troubleshooting Controller

In identifying the source of the controller's operation problem use troubleshooting considerations table including status indication, trouble description, probable causes and recommended action.

To receive the maximum benefit, follow these steps:

- Identify Power Supply and CPU LED status indicators;
- Match processor LEDs with the status LEDs located in troubleshooting tables;
- Once the status LEDs are matched to the appropriate table, simply move across the table identifying error description and probable causes;
- Follow the recommended action steps for each probable cause until the cause is identified;
- If recommended actions do not identify the cause, contact manufacturer or distributor for assistance.

Troubleshooting Input modules

An input circuit responds to an input signal in the following manner:

- An input filter removes false signals due to contact bounce or electrical interference;
- Optical isolation protects the backplane circuits by isolating logic circuits from input signals;
- Logic circuits process the signal;
- An input LED turns on or off indicating the status of the corresponding input device.
- The processor receives the input status for use in processing the program logic.

Troubleshooting Output modules

An output circuit controls the output signal in the following manner:

- The processor determines the output status;
- Logic circuits maintain the output status;
- An output LED indicates the status of the output signal;
- Optical isolation separates logic and backplane circuits from field signals;
- The output driver turns the corresponding output on or off.

Power Distribution

- The master control relay must be able to inhibit all machines motion by removing power to the machine I/O devices when the relay is de-energized.
- The DC power supply should be powered directly from the fused secondary of the transformer.
- Power to the DC input, and output, circuits is connected through a set of master control relay contacts.
- Interrupt the load side rather the AC line power. This avoids the additional delay of power supply turn-on and turn-off.

Power LED

The POWER LED on the power supply indicates that DC power is being supplied to the chassis.

This LED could be off when incoming power is present when the:

- Fuse is blown;
- Voltage drops below the normal operating range;
- Power supply is defective.

Safety Considerations

- Actively thinking about the safety of yourself and others, as well as the condition of your equipment, is of primary importance.
- When troubleshooting, pay careful attention to these general warnings:
 - Have all personnel remain clear of the controller and equipment when power is applied.
 - The problem may be intermittent and sudden unexpected machine motion could result in injury.
 - Have someone ready to operate an emergency-stop switch in case it becomes necessary to shut off power to the controller equipment.
 - Never reach into a machine to actuate a switch since unexpected machine motion can occur and cause injury.
 - Remove all electrical power at the main power disconnect switches before checking electrical connections or inputs/outputs causing machine motion.
 - Never alter safety circuits to defeat their functions. Serious injury or machine damage could result.

System documentation

The documentation is the main guide used by the users and for troubleshooting and fault finding with PLCs.

The documentation for a PLC installation should include:

- A description of the plant.
- Specification of the control requirements.
- Details of the programmable logic controller.
- Electrical installation diagrams.
- Lists of all inputs and outputs connections.
- Application program with full commentary on what it is achieving.
- Software back-ups.
- Operating manual, including details of all start up and shut down procedures and alarms.

Calling for assistance

If you need to contact manufacturer or local distributor for assistance, it is helpful to obtain the following (prior to calling):

- Processor type, series, and Part No.
- Processor LED status
- Processor error codes
- Hardware types in system (I/O modules, chassis)
- Revision of programming device (HHT or Software).

End of PLC Course